DEVOPS ENGINEERING MASTERY

CONFIGURATION MANAGEMENT & CLOUD PROVISIONING

Ansible Automation, Server Configuration, Infrastructure Management & Cloud Provisioning

VOLUME 09 • FIRST EDITION

```
nahid@technova:~$ ansible-playbook site.yml --check --diff
✓ ok=42  •  ✓ changed=3  •  ✓ failed=0
nahid@technova:~$ ansible all -m ping
✓ web-01 | SUCCESS  •  ✓ api-01 | SUCCESS  •  ✓ db-01 | SUCCESS
nahid@technova:~$ ansible-vault encrypt group_vars/prod/vault.yml
```

Ansible • Playbooks • Roles • Vault • Jinja2 • AWX

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

Companion Workbook • 24 Labs • 3 Projects • Capstone

Ansible Lab Workbook

DEVOPS ENGINEERING MASTERY — Volume 09 (Companion Workbook). First Edition, 9 November 2026. Ubuntu Server 24.04 LTS and Ansible Core 2.17+ with a MERN example application. This workbook collects all 24 guided labs, 15 assignments, the 3 major projects and the capstone, so you can work through them beside your lab virtual machines without carrying the whole book. Do the labs in order; each lab states its prerequisites. Never run a lab step on a production host. The instructor-style reference solution for the capstone is at the end — attempt the capstone before reading it.

Lab Workbook • Volume 09 — Configuration Management & Cloud Provisioning Engineering

Lab Workbook — Ansible Automation: হাতে-কলমে অনুশীলন নির্দেশিকা

এই ওয়ার্কবুকটি Volume 09-এর মূল পাঠের সহযোগী। প্রতিটি ল্যাব একটি নির্দিষ্ট দক্ষতা হাতে-কলমে শেখায় — পড়া নয়, করা। ল্যাব ১ থেকে ২৪ পর্যন্ত ধাপে ধাপে এগোলে আপনি একটি সম্পূর্ণ Ansible-চালিত অবকাঠামো গড়ে তুলবেন: ইনভেন্টরি, প্লেবুক, রোল, Vault, হার্ডেনিং, অ্যাপ ডেপ্লয়মেন্ট, ক্লাউড, Docker, Kubernetes প্রস্তুতি, CI/CD এবং মাল্টি-সার্ভার প্রোডাকশন অটোমেশন। তারপর তিনটি প্রজেক্ট ও একটি ক্যাপস্টোনে সবকিছু একত্রিত হবে।

> **CONCEPT:** প্রতিটি ল্যাব স্বয়ংসম্পূর্ণ, কিন্তু ল্যাবগুলো পরস্পরের ওপর দাঁড়িয়ে আছে। ল্যাব ৩-এর ইনভেন্টরি, ল্যাব ৯-এর প্রথম প্লেবুক এবং ল্যাব ১৪-এর রোল — এই তিনটি পরের সব ল্যাবের ভিত্তি। কোনো ল্যাব এড়িয়ে গেলে পরের ল্যাবে ফিরে এসে সেটি শেষ করুন।

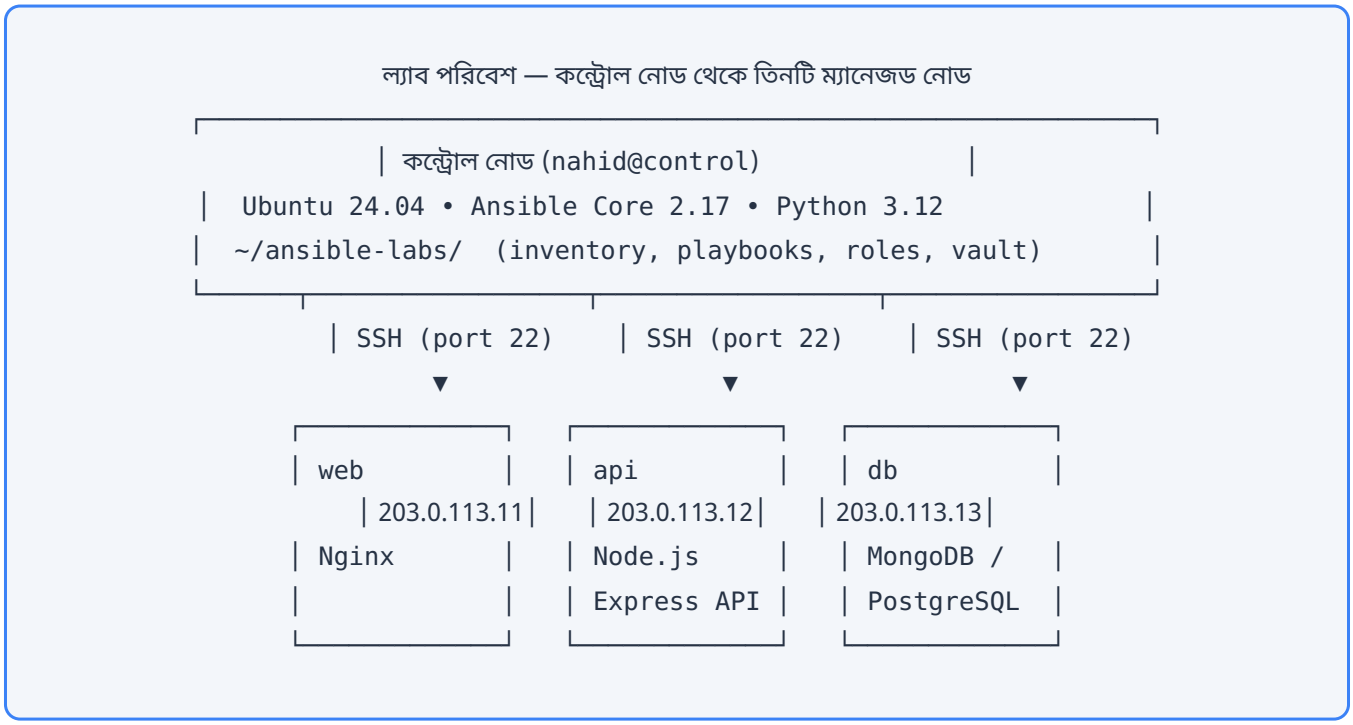
০.১ ল্যাব পরিবেশ — এক নজরে

উপাদান	বিবরণ
কন্ট্রোল নোড	Ubuntu Server 24.04 LTS, Ansible Core 2.17+, Python 3.12, ব্যবহারকারী `nahid`
ম্যানেজড নোড — web	203.0.113.11 — Nginx ওয়েব সার্ভার
ম্যানেজড নোড — api	203.0.113.12 — Node.js API সার্ভার
ম্যানেজড নোড — db	203.0.113.13 — MongoDB / PostgreSQL ডেটাবেস সার্ভার
কোম্পানি / ডোমেইন	TechNova (কাল্পনিক), technova.example
নিরাপত্তা নিয়ম	আসল পাসওয়ার্ড বা সিক্রেট কখনো উদাহরণে লিখবেন না; সব জায়গায় `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করুন

> **WARNING:** এই বইয়ের সব IP ঠিকানা (203.0.113.x) ডকুমেন্টেশন-উদ্দেশ্যে সংরক্ষিত TEST-NET ঠিকানা। নিজের ল্যাবে নিজের VM-এর আসল IP ব্যবহার করুন। ল্যাব ২০ (ক্লাউড VM) ছাড়া বাকি সব ল্যাব লোকাল VM (VirtualBox / KVM / Multipass)-এ বিনামূল্যে করা যায় — ক্লাউডে করার আগে খরচের সতর্কতা পড়ুন।

০.২ ল্যাব করার সাধারণ নিয়ম

(১) প্রতিটি ল্যাব শুরু করার আগে আগের ল্যাবের **Verification** ধাপটি পাস হয়েছে কি না যাচাই করুন। (২) প্রতিটি কমান্ড চালানোর পর আউটপুটটি ল্যাবের **Expected Output**-এর সঙ্গে মিলিয়ে দেখুন; না মিললে **Troubleshooting** দেখুন। (৩) কাজ শেষে **Cleanup** ধাপটি করুন, যাতে পরের ল্যাব পরিষ্কার পরিবেশে শুরু হয়। (৪) প্রতিটি ল্যাব শেষে **Review Questions**-এর উত্তর নিজে লিখে রাখুন — এগুলোই আপনার নোট। (৫) সব ফাইল `~/ansible-labs/` ডিরেক্টরিতে সংরক্ষণ করুন, যাতে প্রজেক্ট ও ক্যাপস্টোনে সেগুলো পুনঃব্যবহার করা যায়।



Lab ১ • Ansible পরিচিতি

Lab ১ — Ansible ইনস্টলেশন

১.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) Ubuntu 24.04-এ Ansible Core 2.17+ ইনস্টল করতে; (২) `ansible --version` দিয়ে ইনস্টলেশন যাচাই করতে; (৩) Ansible-এর কনফিগারেশন ফাইল (`ansible.cfg`) তৈরি ও বুঝতে; (৪) একটি পরিষ্কার ওয়ার্কিং ডিরেক্টরি কাঠামো গড়তে।

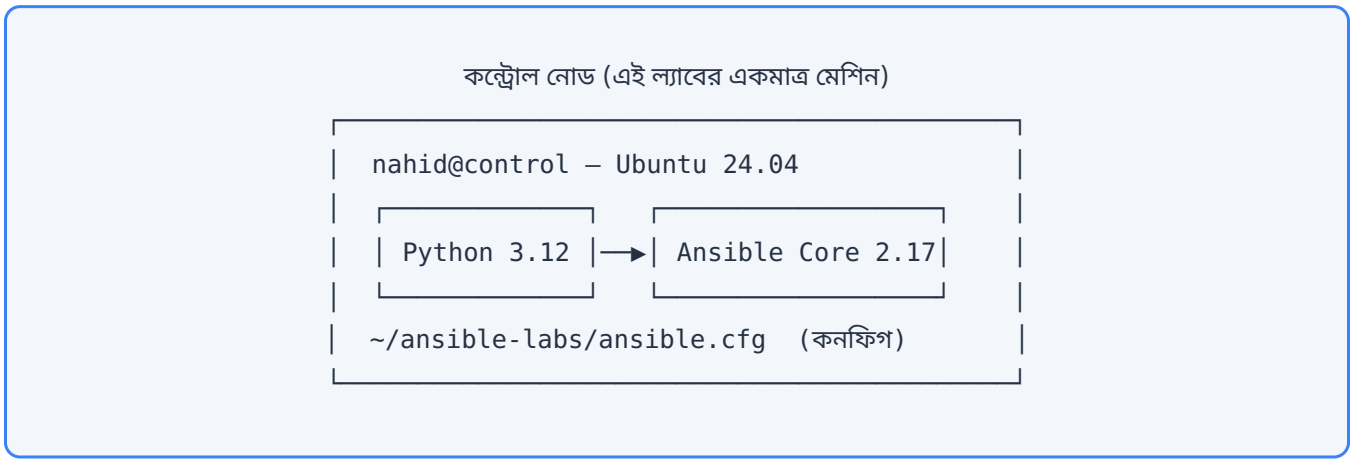
১.২ প্রয়োজনীয় টুলস

Ubuntu Server 24.04 LTS (কন্ট্রোল নোড), Python 3.12, `pip`, ইন্টারনেট সংযোগ, `sudo` অধিকারসহ ব্যবহারকারী `nahid`।

১.৩ পূর্বশর্ত

Linux টার্মিনালে কাজের অভিজ্ঞতা, `apt` দিয়ে প্যাকেজ ইনস্টলের ধারণা, এবং একটি আপডেটেড Ubuntu সিস্টেম। Ansible-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই।

১.৪ আর্কিটেকচার



১.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — সিস্টেম আপডেট করুন। পুরোনো প্যাকেজ তালিকা দিয়ে ইনস্টল করলে নির্ভরতা-সমস্যা হতে পারে।

```
sudo apt update && sudo apt upgrade -y
```

ধাপ ২ — Python ও pip ইনস্টল করুন। Ansible Core Python-ভিত্তিক; Ubuntu 24.04-এ Python 3.12 ডিফল্ট থাকে, তবে `pip` ও `venv` আলাদাভাবে লাগে।

```
sudo apt install -y python3 python3-pip python3-venv python3-full
python3 --version   # Python 3.12.x দেখা যাওয়া উচিত
```

ধাপ ৩ — Ansible Core ইনস্টল করুন। দুটি নির্ভরযোগ্য পথ আছে; ল্যাবের জন্য `pipx` পথটি সুপারিশ করা হয়, কারণ এতে সিস্টেম Python নোংরা হয় না।

```
sudo apt install -y pipx
pipx install --include-deps ansible-core
pipx ensurepath
# বিকল্প (apt পথ): sudo apt install -y ansible-core
```

ধাপ ৪ — ওয়ার্কিং ডিরেক্টরি ও কনফিগ তৈরি করুন।

```
mkdir -p ~/ansible-labs/{inventory,playbooks,roles,group_vars,host_vars}
cd ~/ansible-labs
cat > ansible.cfg << 'EOF'
[defaults]
inventory = ./inventory/hosts.ini
host_key_checking = False
retry_files_enabled = False
deprecation_warnings = False
collections_path = ~/.ansible/collections
roles_path = ./roles
EOF
cat ansible.cfg
```

> **TIP:** `host_key_checking = False` শুধু ল্যাবের জন্য। প্রোডাকশনে এটি `True` রাখুন এবং ম্যানেজড নোডের SSH হোস্ট কী আগেই `known_hosts`-এ যোগ করুন — নইলে মাঝখানের আক্রমণ (Man-in-the-Middle) ধরা পড়বে না।

ধাপ ৫ — ইনস্টলেশন যাচাই করুন।

```
ansible --version
ansible-config view
```

১.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible --version
ansible [core 2.17.x]
  config file = /home/nahid/ansible-labs/ansible.cfg
  python version = 3.12.x
  jinja version = 3.1.x
```

১.৭ যাচাইকরণ (Verification)

`ansible --version`-এ `core 2.17` বা তার বেশি এবং `config file` আপনার `~/ansible-labs/ansible.cfg`-কে নির্দেশ করছে কি না দেখুন। `ansible-config view`-এ `inventory` ও `host_key_checking` মানগুলো মিলছে কি না যাচাই করুন।

১.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`ansible: command not found`	`pipx ensurepath` চালিয়ে নতুন টার্মিনাল খুলুন, অথবা `~/.local/bin` PATH-এ আছে কি না দেখুন
ভুল `config file` দেখাচ্ছে	আপনি `~/ansible-labs` ডিরেক্টরির বাইরে আছেন; `cd ~/ansible-labs` করে আবার চালান
pip ইনস্টলে অনুমতি ত্রুটি	সিস্টেম pip-এ `--break-system-packages` ব্যবহার করবেন না; `pipx` বা `apt` পথ ব্যবহার করুন

১.৯ ক্লিনআপ

এই ল্যাবে কিছু মুছতে হবে না — তৈরি করা ডিরেক্টরি ও কনফিগ পরের সব ল্যাবে ব্যবহৃত হবে। শুধু `sudo apt autoremove -y` দিয়ে অপ্রয়োজনীয় প্যাকেজ পরিষ্কার করতে পারেন।

১.১০ রিভিউ প্রশ্ন

(১) Ansible Core ও Ansible Community Package-এর পার্থক্য কী? (২) `ansible.cfg` কোন কোন জায়গা থেকে পড়া হয়, এবং অগ্রাধিকার ক্রম কী? (৩) কন্ট্রোল নোডে এজেন্ট ইনস্টলের প্রয়োজন হয় না কেন? (৪) `host_key_checking = False` প্রোডাকশনে ঝুঁকিপূর্ণ কেন? (৫) `pipx` দিয়ে ইনস্টল করার সুবিধা কী?

Lab ২ • সংযোগ

Lab ২ — ম্যানেজড নোডে SSH সংযোগ

২.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) SSH কী-জোড়া তৈরি করতে; (২) পাসওয়ার্ড ছাড়া তিনটি ম্যানেজড নোডে পাবলিক-কী প্রমাণীকরণ সেট করতে; (৩) `ansible` দিয়ে সংযোগ পরীক্ষা করতে; (৪) SSH সংযোগের সাধারণ ত্রুটি নির্ণয় করতে।

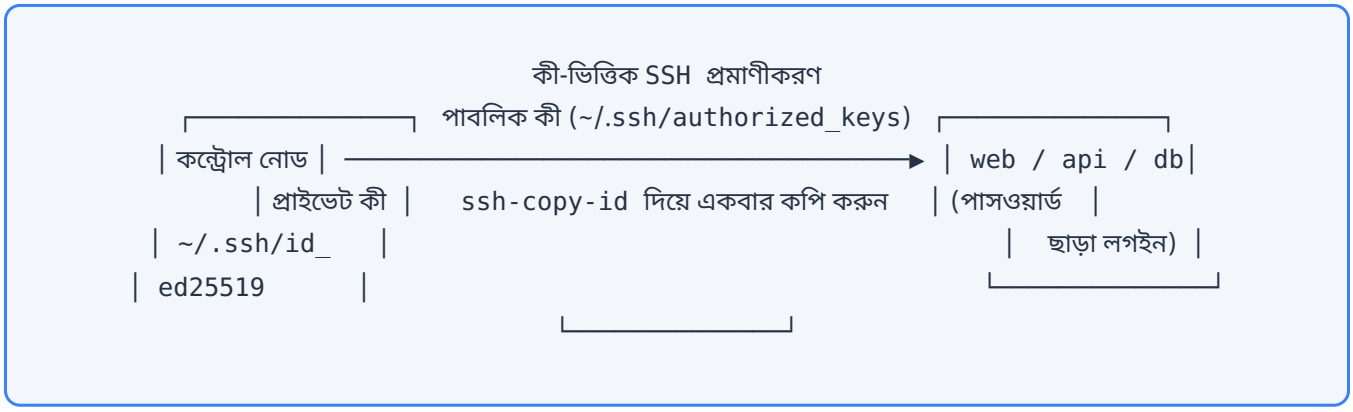
২.২ প্রয়োজনীয় টুলস

কন্ট্রোল নোড (Lab ১ সম্পন্ন), তিনটি ল্যাব VM (web 203.0.113.11, api 203.0.113.12, db 203.0.113.13), প্রতিটি VM-এ `nahid` ব্যবহারকারী ও `sudo` অধিকার।

২.৩ পূর্বশর্ত

Lab ১ সম্পন্ন; তিনটি VM চালু ও একই নেটওয়ার্কে পৌঁছানো যায়; প্রতিটি VM-এ OpenSSH সার্ভার চলছে (`sudo apt install -y openssh-server`)।

২.৪ আর্কিটেকচার



২.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — SSH কী-জোড়া তৈরি করুন। Ed25519 আধুনিক, ছোট ও নিরাপদ; পাসফ্রেজ দিন (ল্যাবে খালি রাখলেও চলবে, তবে প্রোডাকশনে নয়)।

```
ssh-keygen -t ed25519 -C "nahid@technova.example" -f ~/.ssh/id_ed25519
ls -l ~/.ssh/   # id_ed25519 (600) ও id_ed25519.pub (644) থাকবে
```

ধাপ ২ — তিনটি নোডে পাবলিক কী কপি করুন।

```
ssh-copy-id -i ~/.ssh/id_ed25519.pub nahid@203.0.113.11
ssh-copy-id -i ~/.ssh/id_ed25519.pub nahid@203.0.113.12
ssh-copy-id -i ~/.ssh/id_ed25519.pub nahid@203.0.113.13
```

ধাপ ৩ — পাসওয়ার্ড ছাড়া লগইন পরীক্ষা করুন।

```
ssh nahid@203.0.113.11 'hostname; whoami; uptime'
ssh nahid@203.0.113.12 'hostname; whoami'
ssh nahid@203.0.113.13 'hostname; whoami'
```

ধাপ ৪ — Ansible দিয়ে সংযোগ পরীক্ষা করুন। ইনভেন্টরি এখনো তৈরি হয়নি, তাই কমান্ড লাইনে হোস্ট দিন (পূর্ণ ইনভেন্টরি Lab ৩-এ আসবে)।

```
ansible all -i 203.0.113.11, -m ping -u nahid
ansible all -i 203.0.113.11,203.0.113.12,203.0.113.13, -m ping -u nahid
```

> **CONCEPT:** Ansible এজেন্টলেস — ম্যানেজড নোডে কোনো Ansible ইনস্টল করতে হয় না। কন্ট্রোল নোড SSH দিয়ে সংযোগ করে, Python মডিউল পাঠায়, চালায়, ফলাফল নিয়ে আসে। তাই ম্যানেজড নোডে শুধু SSH ও Python 3 থাকলেই চলে।

ধাপ ৫ — SSH কনফিগ ফাইল (ঐচ্ছিক কিন্তু সুপারিশকৃত)। প্রতিবার IP মনে রাখতে হবে না।

```
cat >> ~/.ssh/config << 'EOF'
Host web
    HostName 203.0.113.11
    User nahid
    IdentityFile ~/.ssh/id_ed25519
Host api
    HostName 203.0.113.12
    User nahid
    IdentityFile ~/.ssh/id_ed25519
Host db
    HostName 203.0.113.13
    User nahid
    IdentityFile ~/.ssh/id_ed25519
EOF
chmod 600 ~/.ssh/config
ssh web 'hostname'
```

২.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible all -i 203.0.113.11,203.0.113.12,203.0.113.13, -m ping -u nahid
203.0.113.11 | SUCCESS => {"changed": false, "ping": "pong"}
203.0.113.12 | SUCCESS => {"changed": false, "ping": "pong"}
203.0.113.13 | SUCCESS => {"changed": false, "ping": "pong"}
```

২.৭ যাচাইকরণ (Verification)

তিনটি নোডেই `ping: pong` আসতে হবে। `ssh web 'hostname'` পাসওয়ার্ড না চেয়ে সঙ্গে সঙ্গে চলতে হবে। `ls -l ~/.ssh/id_ed25519`-এর অনুমতি `600` হতে হবে।

২.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`Permission denied (publickey)`	পাবলিক কী কপি হয়নি — `ssh-copy-id` আবার চালান; ম্যানেজড নোডে `~/.ssh` 700 ও `authorized_keys` 600 কি না দেখুন
`Connection timed out`	VM বন্ধ, ভুল IP, বা ফায়ারওয়ালে পোর্ট 22 বন্ধ — `ping` ও `ss -tlnp | grep 22` দিয়ে যাচাই করুন
`WARNING: UNPROTECTED PRIVATE KEY FILE`	`chmod 600 ~/.ssh/id_ed25519` চালান
`Host key verification failed`	VM পুনর্নির্মাণ হলে হোস্ট কী বদলায় — `ssh-keygen -R 203.0.113.11` দিয়ে পুরোনো কী মুছুন

২.৯ ক্লিনআপ

কী মুছবেন না — এটি পরের সব ল্যাবে লাগবে। শুধু পরীক্ষার জন্য তৈরি অস্থায়ী ফাইল থাকলে মুছে ফেলুন।

২.১০ রিভিউ প্রশ্ন

(১) পাসওয়ার্ডের বদলে SSH কী ব্যবহারের নিরাপত্তা-সুবিধা কী? (২) Ed25519 ও RSA কী-এর পার্থক্য কী? (৩) `ssh-copy-id` আসলে কী করে? (৪) প্রাইভেট কী-এর অনুমতি 600 হওয়া জরুরি কেন? (৫) Ansible-এর `ping` মডিউল কি আসলে ICMP ping পাঠায়?

Lab ৩ • ইনভেন্টরি

Lab ৩ — স্ট্যাটিক ইনভেন্টরি

৩.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) INI ও YAML ফরম্যাটে স্ট্যাটিক ইনভেন্টরি লিখতে; (২) হোস্টকে গ্রুপে সাজাতে (`webservers`, `appservers`, `dbservers`); (৩) গ্রুপ ও হোস্ট ভেরিয়েবল ব্যবহার করতে; (৪) `ansible-inventory` দিয়ে ইনভেন্টরি যাচাই করতে।

৩.২ প্রয়োজনীয় টুলস

Lab ১–২ সম্পন্ন কন্ট্রোল নোড, টেক্সট এডিটর (`nano` বা `vim`)।

৩.৩ পূর্বশর্ত

তিনটি ম্যানেজড নোডে SSH কী-প্রমাণীকরণ কাজ করছে (Lab ২)।

৩.৪ আর্কিটেকচার



৩.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — INI ইনভেন্টরি লিখুন।

```
cat > ~/ansible-labs/inventory/hosts.ini << 'EOF'
[webservers]
web ansible_host=203.0.113.11

[appservers]
api ansible_host=203.0.113.12

[dbservers]
db ansible_host=203.0.113.13

[technova:children]
webservers
appservers
dbservers
```

```
[all:vars]
ansible_user=nahid
ansible_python_interpreter=/usr/bin/python3
EOF
```

ধাপ ২ — YAML ইনভেন্টরি (বিকল্প) লিখুন। YAML পড়তে সহজ এবং ভেরিয়েবলের সঙ্গে ভালো মেশে।

```
cat > ~/ansible-labs/inventory/hosts.yml << 'EOF'
all:
  vars:
    ansible_user: nahid
    ansible_python_interpreter: /usr/bin/python3
  children:
    technova:
      children:
        webservers:
          hosts:
            web:
              ansible_host: 203.0.113.11
        appservers:
          hosts:
            api:
              ansible_host: 203.0.113.12
        dbservers:
          hosts:
            db:
              ansible_host: 203.0.113.13
EOF
```

ধাপ ৩ — গ্রুপ ভেরিয়েবল ফাইল তৈরি করুন।

```
mkdir -p ~/ansible-labs/group_vars ~/ansible-labs/host_vars
cat > ~/ansible-labs/group_vars/webservers.yml << 'EOF'
http_port: 80
https_port: 443
server_name: web.technova.example
EOF
cat > ~/ansible-labs/group_vars/dbservers.yml << 'EOF'
db_port: 27017
db_name: technova_app
EOF
cat > ~/ansible-labs/group_vars/all.yml << 'EOF'
company: TechNova
domain: technova.example
timezone: Asia/Dhaka
EOF
```

ধাপ ৪ — ইনভেন্টরি যাচাই করুন।

```
cd ~/ansible-labs
ansible-inventory --list -y
ansible-inventory --graph
ansible webservers --list-hosts
ansible all -m ping
```

> **TIP:** `ansible.cfg`-এ `inventory = ./inventory/hosts.ini` দেওয়া আছে, তাই প্রতিবার `-i` দিতে হয় না। YAML ইনভেন্টরি ব্যবহার করতে চাইলে `ansible.cfg`-এর পথটি `hosts.yml`-এ বদলে নিন।

৩.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible-inventory --graph
@all:
  |--@technova:
  |  |--@appservers:
  |  |  |--api
  |  |--@dbservers:
  |  |  |--db
  |  |--@webservers:
  |  |  |--web
$ ansible all -m ping
web | SUCCESS => {"ping": "pong"}
api | SUCCESS => {"ping": "pong"}
db  | SUCCESS => {"ping": "pong"}
```

৩.৭ যাচাইকরণ (Verification)

`ansible-inventory --graph`-এ তিনটি হোস্ট সঠিক গ্রুপে দেখা যাবে; `ansible webservers --list-hosts`-এ শুধু `web` আসবে; `ansible all -m ping`-এ তিনটিতেই `pong` আসবে।

৩.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`WARNING: provided hosts list is empty`	`ansible.cfg`-এর inventory পথ ভুল, অথবা ভুল ডিরেক্টরি থেকে চালাচ্ছেন
হোস্ট গ্রুপে দেখা যাচ্ছে না	INI-তে `[group:children]` বানান ও YAML-এ ইন্ডেন্টেশন যাচাই করুন; `ansible-inventory --list -y` দিয়ে দেখুন
ভেরিয়েবল কাজ করছে না	`group_vars/` ফাইলের নাম গ্রুপের নামের সঙ্গে হুবহু মিলতে হবে; YAML সিনট্যাক্স যাচাই করুন

৩.৯ ক্লিনআপ

ইনভেন্টরি ফাইলগুলো রেখে দিন — পরের সব ল্যাব এগুলো ব্যবহার করবে। শুধু পরীক্ষামূলক ডুপ্লিকেট ফাইল থাকলে মুছে ফেলুন।

৩.১০ রিভিউ প্রশ্ন

(১) স্ট্যাটিক ও ডায়নামিক ইনভেন্টরির পার্থক্য কী, এবং কখন কোনটি ব্যবহার করবেন? (২) `[group:children]` কী কাজে লাগে? (৩) একই ভেরিয়েবল হোস্ট ও গ্রুপ — দুই জায়গায় থাকলে কোনটি জেতে? (৪) `ansible_host` ও ইনভেন্টরি হোস্টনামের পার্থক্য কী? (৫) YAML ইনভেন্টরিতে ইন্ডেন্টেশন ভুল হলে কী হয়?

Lab ৪ • অ্যাড-হক কমান্ড

Lab ৪ — অ্যাড-হক কমান্ড

৪.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) প্লেবুক না লিখে একক কাজের জন্য অ্যাড-হক কমান্ড চালাতে; (২) সাধারণ মডিউল (`ping`, `command`, `shell`, `apt`, `copy`, `file`, `service`) ব্যবহার করতে; (৩) প্যাটার্ন দিয়ে হোস্ট বাছাই করতে; (৪) `--check` ও `--diff` দিয়ে নিরাপদে পরীক্ষা করতে।

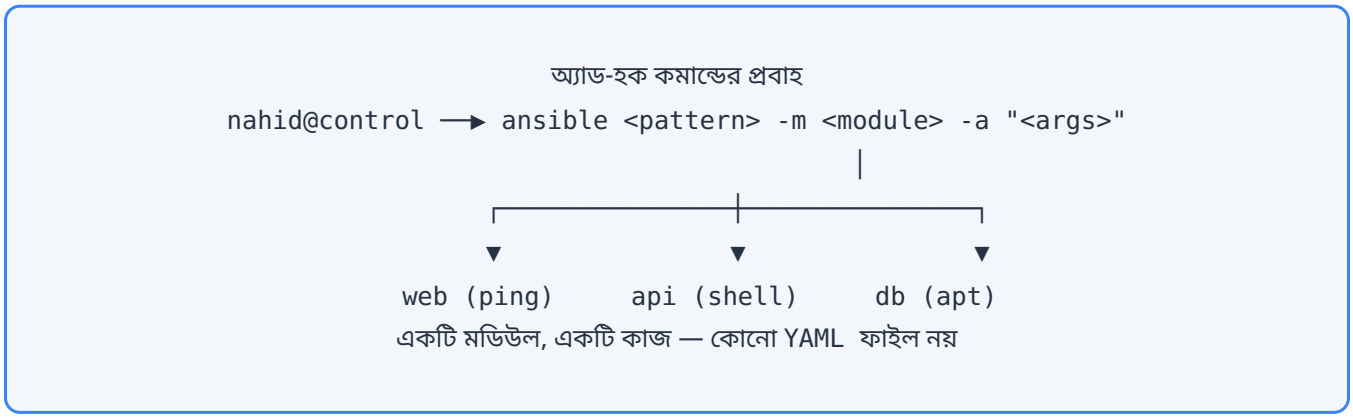
৪.২ প্রয়োজনীয় টুলস

Lab ১–৩ সম্পন্ন কন্ট্রোল নোড ও ইনভেন্টরি।

৪.৩ পূর্বশর্ত

ইনভেন্টরি থেকে তিনটি নোডেই `ping` সফল হচ্ছে।

৪.৪ আর্কিটেকচার



৪.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — তথ্য সংগ্রহের মডিউল চালান।

```
ansible all -m ping
ansible all -m setup -a 'filter=ansible_distribution*' | head -40
ansible web -m command -a 'uptime'
```

ধাপ ২ — `shell` বনাম `command` বুঝুন। `command` পাইপ বা রিডাইরেক্ট বোঝে না; `shell` বোঝে।

```
ansible all -m shell -a 'df -h / | tail -1'
ansible all -m shell -a 'free -h | grep Mem'
```

ধাপ ৩ — প্যাকেজ ও ফাইল মডিউল ব্যবহার করুন।

```
ansible all -m apt -a 'name=htop state=present update_cache=yes' --become
ansible web -m copy -a 'content="Hello TechNova\n" dest=/tmp/hello.txt mode=0644'
ansible web -m file -a 'path=/tmp/hello.txt state=file'
ansible web -m command -a 'cat /tmp/hello.txt'
```

ধাপ ৪ — সার্ভিস মডিউল ও প্যাটার্ন অনুশীলন করুন।

```
ansible webservers -m service -a 'name=ssh state=started' --become
ansible 'webservers:appservers' -m shell -a 'hostname'
ansible 'all:!dbservers' -m shell -a 'hostname'
ansible all -m shell -a 'echo test' --check --diff
```

> **WARNING:** `shell` মডিউলে ব্যবহারকারীর ইনপুট সরাসরি বসাবেন না — কমান্ড ইনজেকশন হতে পারে। যেখানে সম্ভব `command` বা নির্দিষ্ট মডিউল (`apt`, `copy`, `file`) ব্যবহার করুন; `shell` শুধু পাইপ/রিডাইরেক্ট দরকার হলে।

ধাপ ৫ — ফলাফলের রঙ ও পরিবর্তন বুঝুন। সবুজ = কোনো পরিবর্তন হয়নি (ok), হলুদ = পরিবর্তন হয়েছে (changed), লাল = ব্যর্থ (failed)। একই কমান্ড দ্বিতীয়বার চালালে `changed` কমে যাওয়া উচিত — এটাই Idempotency।

```
ansible web -m copy -a 'content="Hello TechNova\n" dest=/tmp/hello.txt mode=0644'
ansible web -m copy -a 'content="Hello TechNova\n" dest=/tmp/hello.txt mode=0644'
```

৪.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible all -m shell -a 'df -h / | tail -1'
web | CHANGED | rc=0 >> /dev/sda1  20G  5.1G  14G  27% /
api | CHANGED | rc=0 >> /dev/sda1  20G  4.8G  14G  26% /
db  | CHANGED | rc=0 >> /dev/sda1  20G  6.2G  13G  32% /
```

৪.৭ যাচাইকরণ (Verification)

`/tmp/hello.txt` ফাইলটি web নোডে আছে কি না `ansible web -m command -a 'cat /tmp/hello.txt'` দিয়ে দেখুন। `htop` তিনটি নোডেই ইনস্টল হয়েছে কি না `ansible all -m command -a 'which htop'` দিয়ে যাচাই করুন। দ্বিতীয়বার `copy` চালালে `ok` (সবুজ) আসা উচিত।

৪.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`Failed to lock /var/lib/dpkg`	অন্য apt প্রক্রিয়া চলছে — অপেক্ষা করুন বা `ps aux | grep apt` দিয়ে দেখুন
`shell` কমান্ডে পাইপ কাজ করছে না	ভুলে `command` ব্যবহার করেছেন — `shell` ব্যবহার করুন
প্যাটার্নে কোনো হোস্ট মিলছে না	

	`ansible-inventory --graph` দিয়ে গ্রুপের নাম যাচাই করুন; কোলন ও বিস্ময়চিহ্নের ব্যবহার দেখুন

৪.৯ ক্লিনআপ

```
ansible web -m file -a 'path=/tmp/hello.txt state=absent'
ansible all -m apt -a 'name=htop state=absent' --become
```

৪.১০ রিভিউ প্রশ্ন

(১) অ্যাড-হক কমান্ড কখন যথেষ্ট, আর কখন প্লেবুক লিখতে হয়? (২) `command` ও `shell` মডিউলের পার্থক্য কী? (৩) `--check` মোড কী করে, এবং এর সীমা কী? (৪) Idempotency বলতে কী বোঝায়? (৫) `all:!dbservers` প্যাটার্নটি কী বোঝায়?

Lab ৫ • প্যাকেজ অটোমেশন

Lab ৫ — প্যাকেজ অটোমেশন

৫.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) `apt` মডিউল দিয়ে প্যাকেজ ইনস্টল, আপগ্রেড ও অপসারণ করতে; (২) একাধিক প্যাকেজ একসঙ্গে লুপে ইনস্টল করতে; (৩) `apt` ক্যাশ আপডেটের সময় নিয়ন্ত্রণ করতে; (৪) প্যাকেজের অবস্থা যাচাই করতে।

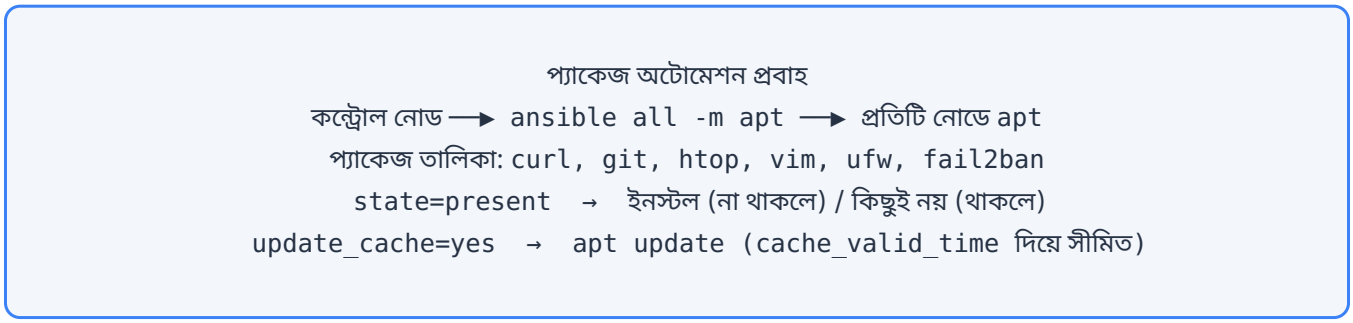
৫.২ প্রয়োজনীয় টুলস

Lab ১–৪ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি, `sudo` অধিকার।

৫.৩ পূর্বশর্ত

তিনটি নোডেই SSH সংযোগ ও `ping` সফল; `apt` মডিউলের প্রাথমিক ধারণা (Lab ৪)।

৫.৪ আর্কিটেকচার



৫.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — একটি প্যাকেজ ইনস্টল করুন।

```
ansible all -m apt -a 'name=curl state=present update_cache=yes cache_valid_time=3600' --
become
```

ধাপ ২ — একাধিক প্যাকেজ একসঙ্গে ইনস্টল করুন।

```
ansible all -m apt -a 'name={{ item }} state=present' --become --extra-vars '{"item": "git"}'
# লুপের সঠিক রূপ প্লেবুকে (Lab ৯-এ); অ্যাড-হকে তালিকা দিন:
ansible all -m apt -a 'name=git,htop,vim,ufw state=present' --become
```

ধাপ ৩ — প্যাকেজ আপগ্রেডের নীতি পরীক্ষা করুন। সব প্যাকেজ একসঙ্গে আপগ্রেড করা ঝুঁকিপূর্ণ; প্রথমে `--check` দিয়ে দেখুন।

```
ansible all -m apt -a 'upgrade=dist update_cache=yes' --become --check
# সন্তুষ্ট হলে --check ছাড়া চালান (ল্যাব VM-এ নিরাপদ, প্রোডাকশনে সাবধানে)
```

ধাপ ৪ — প্যাকেজের অবস্থা যাচাই করুন।

```
ansible all -m shell -a 'dpkg -l | grep -E "curl|git|htop" | awk "{print \$2, \$3}"'
ansible all -m apt -a 'name=nginx state=present' --become --check
```

> **CONCEPT:** `cache_valid_time=3600` মানে ক্যাশ এক ঘণ্টার মধ্যে আপডেট হয়ে থাকলে আবার `apt update` চালাবে না — এতে প্রতিটি রানে সময় বাঁচে। `state=present` Idempotent: প্যাকেজ থাকলে কিছু করে না, না থাকলে ইনস্টল করে।

ধাপ ৫ — একটি প্যাকেজ অপসারণের অনুশীলন করুন।

```
ansible all -m apt -a 'name=htop state=absent' --become
ansible all -m apt -a 'name=htop state=present' --become  # আবার ইনস্টল
```

৫.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible all -m apt -a 'name=curl state=present update_cache=yes cache_valid_time=3600' --
become
web | CHANGED => {"cache_update": true, "changed": true}
api | CHANGED => {"cache_update": true, "changed": true}
db  | CHANGED => {"changed": false, "msg": "curl is already installed"}
```

৫.৭ যাচাইকরণ (Verification)

`ansible all -m command -a 'curl --version | head -1'` ও `git --version` তিনটি নোডেই সফল হতে হবে। দ্বিতীয়বার একই কমান্ড চালালে `changed: false` আসা উচিত।

৫.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`Could not get lock /var/lib/dpkg/lock`	অন্য apt প্রক্রিয়া চলছে — `ps aux | grep -i apt` দিয়ে দেখে অপেক্ষা করুন
`Unable to locate package`	ক্যাশ পুরোনো — `update_cache=yes` যোগ করুন
আপগ্রেডে কার্নেল বদলে গেল	`upgrade=dist` কার্নেলও বদলাতে পারে; প্রোডাকশনে নির্দিষ্ট প্যাকেজ আপগ্রেড করুন, রিবুটের পরিকল্পনা রাখুন

৫.৯ ক্লিনআপ

ইনস্টল করা বেস প্যাকেজগুলো (curl, git, vim, ufw) রেখে দিন — পরের ল্যাবে লাগবে। শুধু পরীক্ষার প্যাকেজ থাকলে `state=absent` দিয়ে মুছে ফেলুন।

৫.১০ রিভিউ প্রশ্ন

(১) `state=present`, `state=latest` ও `state=absent`-এর পার্থক্য কী? (২) `cache_valid_time` কেন ব্যবহার করবেন? (৩) `upgrade=dist` ও `upgrade=safe`-এর পার্থক্য কী? (৪) `apt` মডিউল `shell`-এ `apt install` চালানোর চেয়ে ভালো কেন? (৫) প্যাকেজ ইনস্টলে `--become` কেন লাগে?

Lab ৬ • ব্যবহারকারী ব্যবস্থাপনা

Lab ৬ — ব্যবহারকারী ও গ্রুপ ব্যবস্থাপনা

৬.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) `user` ও `group` মডিউল দিয়ে ব্যবহারকারী ও গ্রুপ তৈরি করতে; (২) ব্যবহারকারীকে গ্রুপে যোগ ও `sudo` অধিকার দিতে; (৩) SSH পাবলিক কী (`authorized_key`) বসাতে; (৪) ব্যবহারকারী নিষ্ক্রিয় বা মুছে ফেলতে।

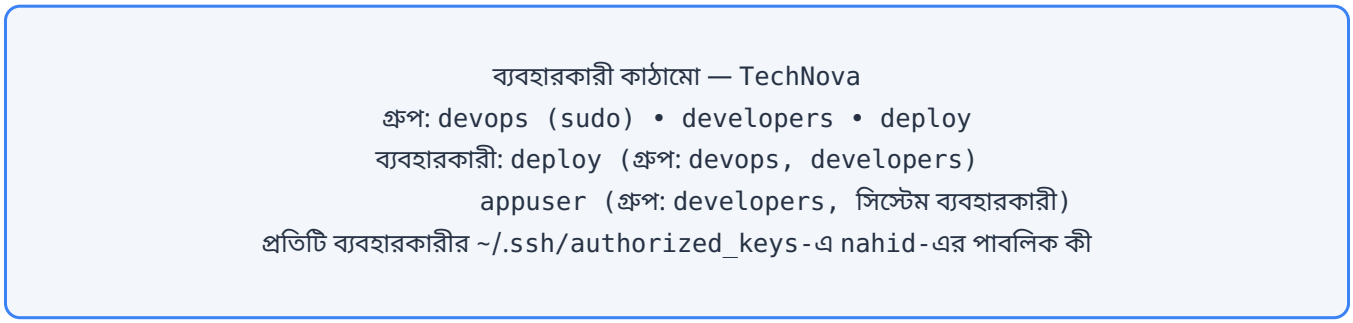
৬.২ প্রয়োজনীয় টুলস

Lab ১–৫ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি।

৬.৩ পূর্বশর্ত

SSH কী-প্রমাণীকরণ ও `--become` (sudo) কাজ করছে।

৬.৪ আর্কিটেকচার



৬.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — গ্রুপ তৈরি করুন।

```
ansible all -m group -a 'name=devops state=present' --become
ansible all -m group -a 'name=developers state=present' --become
ansible all -m group -a 'name=deploy state=present' --become
```

ধাপ ২ — ব্যবহারকারী তৈরি করুন।

```
ansible all -m user -a 'name=deploy groups=devops,developers append=yes shell=/bin/bash
create_home=yes' --become
ansible all -m user -a 'name=appuser groups=developers append=yes shell=/usr/sbin/nologin
system=yes create_home=no' --become
```

ধাপ ৩ — `sudo` অধিকার দিন (পাসওয়ার্ড ছাড়া, শুধু ল্যাবে)।

```
ansible all -m copy -a 'content="deploy ALL=(ALL) NOPASSWD:ALL\n" dest=/etc/sudoers.d/deploy
mode=0440 validate="visudo -cf %s"' --become
```

ধাপ ৪ — SSH পাবলিক কী বসান।

```
ansible all -m authorized_key -a 'user=deploy state=present key="{{ lookup("file", "~/.ssh/
id_ed25519.pub") }}"' --become
```

> **WARNING:** `NOPASSWD:ALL` শুধু ল্যাবের জন্য। প্রোডাকশনে পাসওয়ার্ড-সহ sudo দিন এবং sudoers ফাইলে শুধু প্রয়োজনীয় কমান্ডের অনুমতি দিন। `validate="visudo -cf %s"` ভুল sudoers ফাইল বসানো থেকে বাঁচায় — এটি কখনো বাদ দেবেন না।

ধাপ ৫ — যাচাই করুন।

```
ansible all -m shell -a 'id deploy; id appuser; sudo -l -U deploy' --become
ssh -i ~/.ssh/id_ed25519 deploy@203.0.113.11 'whoami; hostname'
```

৬.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible all -m shell -a 'id deploy' --become
web | CHANGED | rc=0 >> uid=1001(deploy) gid=1001(deploy) groups=1001(deploy),1002(devops),
1003(developers)
```

৬.৭ যাচাইকরণ (Verification)

`id deploy`-এ `devops` ও `developers` গ্রুপ দেখা যাবে; `sudo -l -U deploy`-এ NOPASSWD দেখা যাবে; `deploy` ব্যবহারকারী হিসেবে SSH লগইন পাসওয়ার্ড ছাড়া হবে।

৬.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`user is currently used by process`	ব্যবহারকারী লগইন অবস্থায় মুছতে গেলে হয় — প্রথমে লগআউট করান, তারপর মুছুন
sudoers ফাইল ভুল হলে sudo বন্ধ	`validate` ব্যবহার করলে এটি হবে না; হলে রিকভারি মোডে `/etc/sudoers.d/deploy` মুছুন
`append=yes` না দিলে গ্রুপ হারায়	`groups` দিলে পুরোনো গ্রুপ বদলে যায় — সবসময় `append=yes` দিন

৬.৯ ক্লিনআপ

```
ansible all -m user -a 'name=appuser state=absent remove=yes' --become
# deploy ব্যবহারকারী রেখে দিন — পরের ল্যাবে (Lab ১৮) অ্যাপ ডেপ্লয়ে লাগবে
```

৬.১০ রিভিউ প্রশ্ন

(১) `append=yes` না দিলে কী হয়? (২) সিস্টেম ব্যবহারকারী (`system=yes`) কখন ব্যবহার করবেন? (৩) `authorized_key` মডিউল ম্যানুয়ালি কী কপি করার চেয়ে ভালো কেন? (৪) sudoers ফাইলে `validate` কেন জরুরি? (৫) ব্যবহারকারী মুছতে `remove=yes` কী করে?

Lab ৭ • সার্ভিস ব্যবস্থাপনা

Lab ৭ — সার্ভিস ব্যবস্থাপনা

৭.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) `service` ও `systemd` মডিউল দিয়ে সার্ভিস চালু, বন্ধ, পুনরায় চালু ও সক্রিয় করতে; (২) সার্ভিসের অবস্থা যাচাই করতে; (৩) কাস্টম systemd ইউনিট ফাইল বসাতে; (৪) সার্ভিস ব্যর্থ হলে লগ দেখে নির্ণয় করতে।

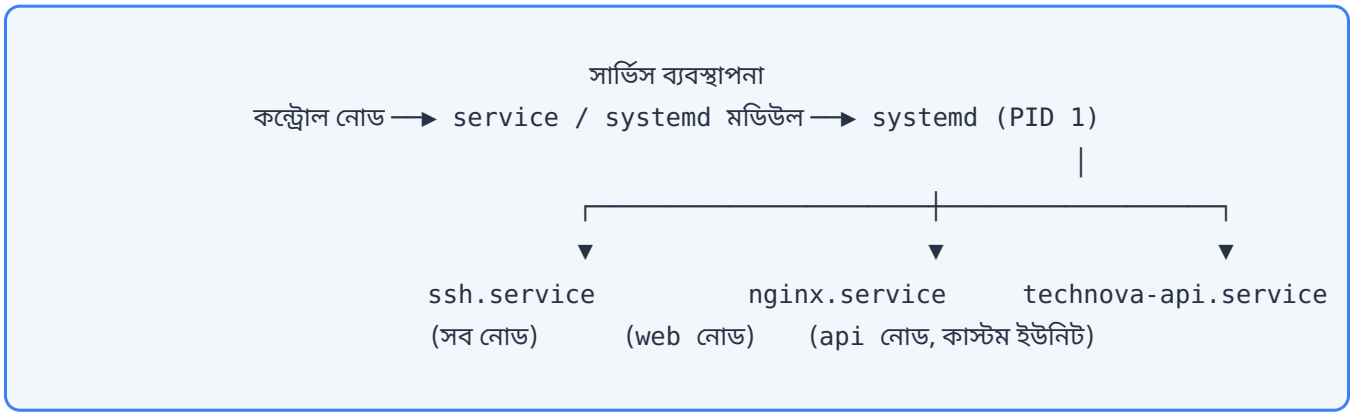
৭.২ প্রয়োজনীয় টুলস

Lab ১–৬ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি, Nginx (Lab ৫-এ ইনস্টল না থাকলে এখন ইনস্টল করুন)।

৭.৩ পূর্বশর্ত

তিনটি নোডে `systemd` চলছে; `ssh` সার্ভিস সক্রিয় আছে।

৭.৪ আর্কিটেকচার



৭.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — সার্ভিসের বর্তমান অবস্থা দেখুন।

```
ansible all -m service_facts
ansible all -m shell -a 'systemctl is-active ssh; systemctl is-enabled ssh'
```

ধাপ ২ — Nginx ইনস্টল ও চালু করুন।

```
ansible web -m apt -a 'name=nginx state=present' --become
ansible web -m service -a 'name=nginx state=started enabled=yes' --become
ansible web -m shell -a 'systemctl status nginx --no-pager | head -8'
```

ধাপ ৩ — কাস্টম systemd ইউনিট তৈরি করুন (api নোডে)।

```
cat > /tmp/technova-api.service << 'EOF'
[Unit]
Description=TechNova API (Node.js)
After=network.target

[Service]
Type=simple
User=deploy
WorkingDirectory=/opt/technova-api
ExecStart=/usr/bin/node /opt/technova-api/server.js
Restart=on-failure
RestartSec=5
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
EOF
ansible api -m copy -a 'src=/tmp/technova-api.service dest=/etc/systemd/system/technova-api.service mode=0644' --become
ansible api -m systemd -a 'name=technova-api daemon_reload=yes enabled=yes state=started' --become
```

> **CONCEPT:** `service` মডিউল সাধারণ কাজের জন্য যথেষ্ট; `systemd` মডিউলে `daemon_reload`, `masked` ও `scope`-এর মতো অতিরিক্ত সুবিধা আছে। ইউনিট ফাইল বদলালে `daemon_reload=yes` না দিলে systemd নতুন কনফিগ পড়ে না।

ধাপ ৪ — সার্ভিস পুনরায় চালু ও বন্ধের অনুশীলন করুন।

```
ansible web -m service -a 'name=nginx state=restarted' --become
ansible web -m shell -a 'curl -s -o /dev/null -w "%{http_code}\n" http://localhost/'
ansible api -m systemd -a 'name=technova-api state=stopped' --become
ansible api -m systemd -a 'name=technova-api state=started' --become
```

৭.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible web -m shell -a 'curl -s -o /dev/null -w "%{http_code}\n" http://localhost/'
web | CHANGED | rc=0 >> 200
```

৭.৭ যাচাইকরণ (Verification)

`systemctl is-active nginx` = `active`, `is-enabled nginx` = `enabled`; `curl http://203.0.113.11/` কন্ট্রোল নোড থেকে 200 দেবে; `technova-api` ইউনিটটি `loaded` অবস্থায় থাকবে (অ্যাপ ফাইল না থাকলে ব্যর্থ হতে পারে — সেটি Lab ১৮-এ ঠিক হবে)।

৭.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান

`Failed to start nginx: port 80 in use`	অন্য ওয়েব সার্ভার চলছে — `ss -tlnp | grep 80` দিয়ে দেখে বন্ধ করুন
ইউনিট ফাইল বদলেও পুরোনো কনফিগ চলছে	`daemon_reload=yes` দিয়ে `systemd` মডিউল চালান
সার্ভিস `failed` অবস্থায়	`journalctl -u technova-api -n 50 --no-pager` দিয়ে লগ দেখুন; `ExecStart` -এর পথ ও অনুমতি যাচাই করুন

৭.৯ ক্লিনআপ

Nginx চালু রেখে দিন — Lab ৮-এ এর কনফিগারেশন করা হবে। `technova-api` সার্ভিসটি বন্ধ করে রাখুন (`state=stopped`), কারণ অ্যাপ এখনো ডেপ্লয় হয়নি।

৭.১০ রিভিউ প্রশ্ন

(১) `state=started` ও `enabled=yes` -এর পার্থক্য কী? (২) `restarted` ও `reloaded` -এর পার্থক্য কখন গুরুত্বপূর্ণ? (৩) `daemon_reload` কখন লাগে? (৪) `Restart=on-failure` কী করে? (৫) `service_facts` কী তথ্য দেয়?

Lab ৮ • Nginx কনফিগারেশন

Lab ৮ — Nginx কনফিগারেশন

৮.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) Nginx সাইট কনফিগারেশন ফাইল তৈরি ও সক্রিয় করতে; (২) রিভার্স প্রক্সি হিসেবে Nginx সাজাতে; (৩) কনফিগ পরীক্ষা (`nginx -t`) করে নিরাপদে রিলোড করতে; (৪) স্ট্যাটিক ওয়েবসাইট পরিবেশন করতে।

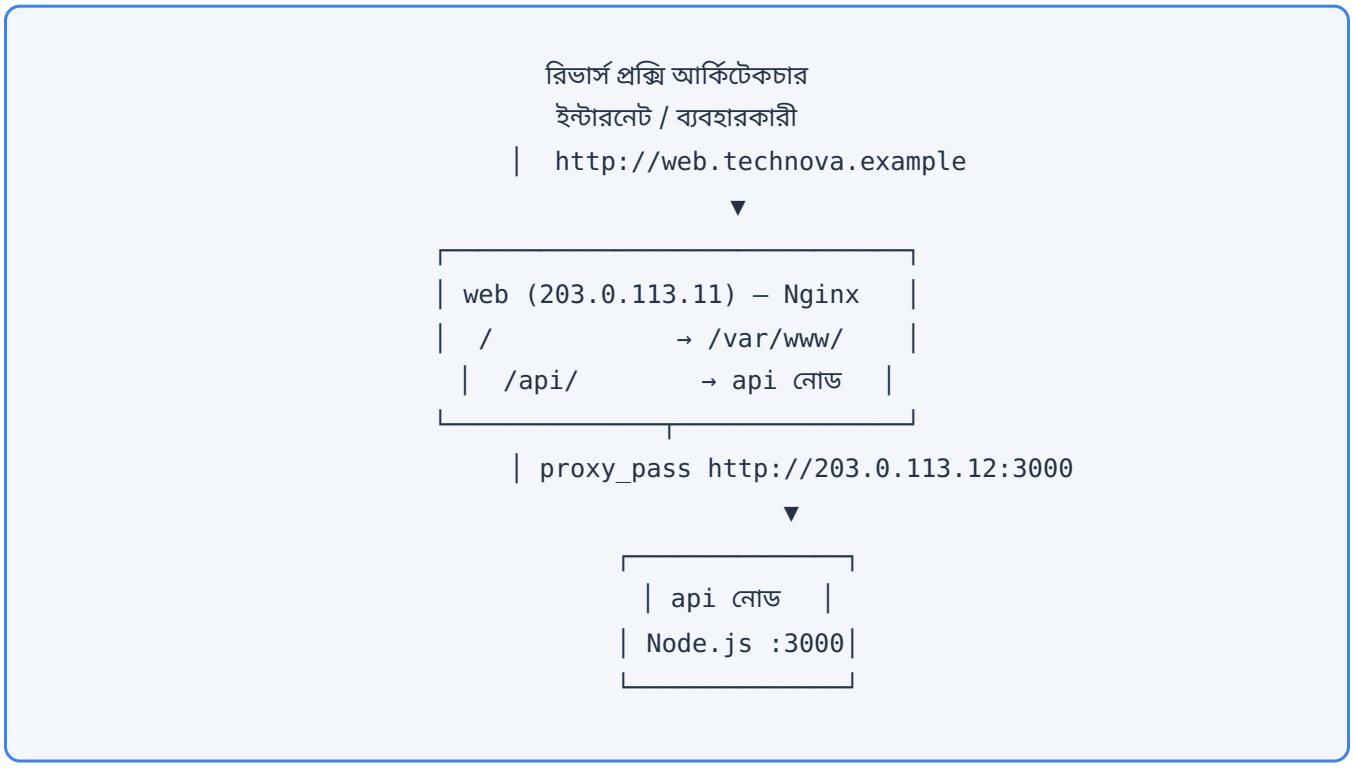
৮.২ প্রয়োজনীয় টুলস

Lab ১–৭ সম্পন্ন কন্ট্রোল নোড, web নোডে Nginx ইনস্টল ও চলমান।

৮.৩ পূর্বশর্ত

Lab ৭-এ Nginx চালু আছে; web নোডের পোর্ট 80 পৌঁছানো যায়।

৮.৪ আর্কিটেকচার



৮.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — স্ট্যাটিক ওয়েব রুট তৈরি করুন।

```
ansible web -m file -a 'path=/var/www/technova state=directory owner=www-data group=www-data
mode=0755' --become
```

```
ansible web -m copy -a 'content="<h1>TechNova — স্বাগতম</h1>\n<p>Ansible দিয়ে কনফিগার করা হয়েছে।</
p>\n" dest=/var/www/technova/index.html mode=0644' --become
```

ধাপ ২ — সাইট কনফিগারেশন লিখুন।

```
cat > /tmp/technova.conf << 'EOF'
server {
    listen 80;
    server_name web.technova.example 203.0.113.11;

    root /var/www/technova;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location /api/ {
        proxy_pass http://203.0.113.12:3000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
EOF
ansible web -m copy -a 'src=/tmp/technova.conf dest=/etc/nginx/sites-available/technova.conf
mode=0644 validate="nginx -t -c /etc/nginx/nginx.conf"' --become
```

ধাপ ৩ — সাইট সক্রিয় ও ডিফল্ট সাইট নিষ্ক্রিয় করুন।

```
ansible web -m file -a 'src=/etc/nginx/sites-available/technova.conf dest=/etc/nginx/sites-
enabled/technova.conf state=link' --become
ansible web -m file -a 'path=/etc/nginx/sites-enabled/default state=absent' --become
ansible web -m shell -a 'nginx -t' --become
```

ধাপ ৪ — Nginx রিলোড করুন ও পরীক্ষা করুন।

```
ansible web -m service -a 'name=nginx state=reloaded' --become
curl -s http://203.0.113.11/
curl -s -o /dev/null -w "API প্রক্সি: %{http_code}\n" http://203.0.113.11/api/health
```

> **TIP:** কনফিগ বদলে `restarted`-এর বদলে `reloaded` ব্যবহার করুন — এতে চলমান সংযোগ ছিন্ন হয় না। আর সবসময় `nginx -t` দিয়ে পরীক্ষা করে তারপর রিলোড করুন; ভুল কনফিগে রিলোড করলে Nginx পুরোনো কনফিগেই চলতে থাকে, কিন্তু রিস্টার্ট করলে বন্ধ হয়ে যেতে পারে।

৮.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible web -m shell -a 'nginx -t' --become
web | CHANGED | rc=0 >> nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

```
$ curl -s http://203.0.113.11/
<h1>TechNova — স্বাগতম</h1>
```

৮.৭ যাচাইকরণ (Verification)

`curl http://203.0.113.11/`-এ TechNova পাতা আসবে; `nginx -t` সফল হবে; `/etc/nginx/sites-enabled/`-এ শুধু `technova.conf` লিংক থাকবে; `/api/` পথটি api নোডের দিকে প্রক্সি হবে (অ্যাপ না থাকলে 502 আসা স্বাভাবিক — Lab ১৮-এ ঠিক হবে)।

৮.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`nginx: [emerg] unknown directive`	সিনট্যাক্স ভুল — `nginx -t`-এর লাইন নম্বর দেখে ঠিক করুন; সেমিকোলন ভুলে গেছেন কি না দেখুন
403 Forbidden	`/var/www/technova`-এর অনুমতি বা মালিকানা ভুল — `www-data` পড়তে পারে কি না যাচাই করুন
`/api/`-এ 502 Bad Gateway	api নোডে অ্যাপ চলছে না — এটি এখন স্বাভাবিক; Lab ১৮-এ অ্যাপ ডেপ্লয় হলে ঠিক হবে

৮.৯ ক্লিনআপ

কনফিগ রেখে দিন — Lab ১২ (টেমপ্লেট) ও Lab ১৮-এ এটিকে টেমপ্লেটে রূপান্তর করা হবে। `/tmp/technova.conf` কন্ট্রোল নোড থেকে মুছে ফেলতে পারেন।

৮.১০ রিভিউ প্রশ্ন

(১) `sites-available` ও `sites-enabled`-এর পার্থক্য কী? (২) `try_files $uri $uri/ =404;` কী করে? (৩) রিভার্স প্রক্সিতে `X-Forwarded-For` হেডার কেন লাগে? (৪) `reload` ও `restart`-এর পার্থক্য কী? (৫) `validate` প্যারামিটার কীভাবে ভুল কনফিগ বসানো ঠেকায়?

Lab ৯ • প্রথম প্লেবুক

Lab ৯ — প্রথম প্লেবুক

৯.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) YAML প্লেবুকের কাঠামো (play, tasks, modules) লিখতে; (২) `ansible-playbook` চালাতে; (৩) `--check`, `--diff` ও `--limit` ব্যবহার করতে; (৪) প্লেবুকের সিনট্যাক্স যাচাই করতে।

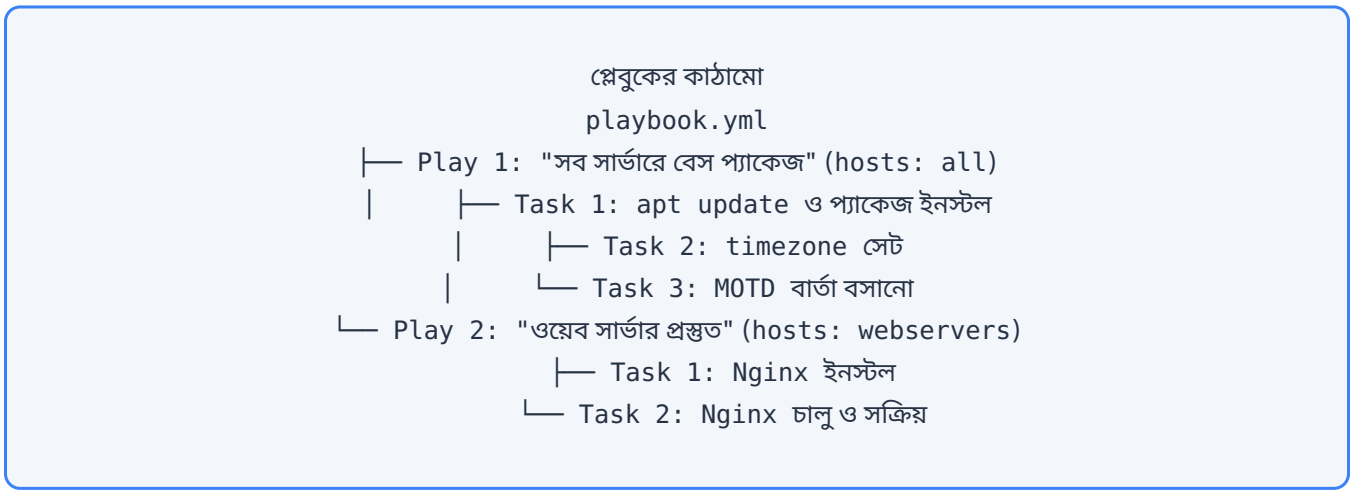
৯.২ প্রয়োজনীয় টুলস

Lab ১–৮ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি, টেক্সট এডিটর।

৯.৩ পূর্বশর্ত

অ্যাড-হক কমান্ডের অভিজ্ঞতা (Lab ৪); YAML-এর প্রাথমিক ধারণা।

৯.৪ আর্কিটেকচার



৯.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — প্রথম প্লেবুক লিখুন।

```
cat > ~/ansible-labs/playbooks/site-baseline.yml << 'EOF'
---
- name: সব সার্ভারে বেস প্যাকেজ ও সেটিংস
  hosts: all
  become: true
  tasks:
    - name: apt ক্যাশ আপডেট ও বেস প্যাকেজ ইনস্টল
      ansible.builtin.apt:
        name:
          - curl
          - git
          - vim
          - htop
```

```
          - ufw
        state: present
        update_cache: true
        cache_valid_time: 3600

    - name: টাইমজোন Asia/Dhaka সেট করুন
      community.general.timezone:
        name: Asia/Dhaka

    - name: স্বাগত বার্তা (MOTD) বসান
      ansible.builtin.copy:
        content: "TechNova managed server — Ansible দিয়ে পরিচালিত\n"
        dest: /etc/motd
        mode: "0644"

- name: ওয়েব সার্ভার প্রস্তুত করুন
  hosts: webservers
  become: true
  tasks:
    - name: Nginx ইনস্টল করুন
      ansible.builtin.apt:
        name: nginx
        state: present

    - name: Nginx চালু ও বুটে সক্রিয় করুন
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
EOF
```

ধাপ ২ — সিনট্যাক্স যাচাই করুন।

```
cd ~/ansible-labs
ansible-playbook playbooks/site-baseline.yml --syntax-check
ansible-playbook playbooks/site-baseline.yml --check --diff
```

ধাপ ৩ — প্লেবুক চালান।

```
ansible-playbook playbooks/site-baseline.yml
```

ধাপ ৪ — Idempotency পরীক্ষা করুন। একই প্লেবুক দ্বিতীয়বার চালান — সব টাস্ক `ok` আসা উচিত, `changed=0` হওয়া উচিত।

```
ansible-playbook playbooks/site-baseline.yml | tail -12
```

> **CONCEPT:** প্লেবুক হলো "ঘোষণামূলক" (Declarative) — আপনি বলেন কী অবস্থা চাই, Ansible নিজে ঠিক করে কী করতে হবে। তাই দ্বিতীয়বার চালালে কিছু বদলায় না। এটি শেল স্ক্রিপ্টের সঙ্গে মূল পার্থক্য: স্ক্রিপ্ট বলে কী করতে হবে, আর প্রতিবার চালালে একই কাজ আবার করে।

ধাপ ৫ — সীমিত হোস্টে চালান।

```
ansible-playbook playbooks/site-baseline.yml --limit webservers
ansible-playbook playbooks/site-baseline.yml --tags "none" --list-tasks
```

৯.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
PLAY [সব সার্ভারে বেস প্যাকেজ ও সেটিংস] *****************
TASK [apt ক্যাশ আপডেট ও বেস প্যাকেজ ইনস্টল] *************
changed: [web]
changed: [api]
changed: [db]
PLAY RECAP *********************************************
web  : ok=5  changed=3  unreachable=0  failed=0
api  : ok=4  changed=3  unreachable=0  failed=0
db   : ok=4  changed=3  unreachable=0  failed=0
```

৯.৭ যাচাইকরণ (Verification)

PLAY RECAP-এ `failed=0` ও `unreachable=0` হতে হবে। `ansible all -m shell -a 'cat /etc/motd; date'` দিয়ে MOTD ও টাইমজোন (+06) যাচাই করুন। দ্বিতীয় রানে `changed=0` হতে হবে।

৯.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`ERROR! syntax error in YAML`	ইন্ডেন্টেশন ভুল — ট্যাব নয়, স্পেস ব্যবহার করুন; `yamllint` বা অনলাইন YAML যাচাইকারী ব্যবহার করুন
`community.general.timezone not found`	কমিউনিটি কালেকশন ইনস্টল নেই — `ansible-galaxy collection install community.general` চালান (Lab ১৫-এ বিস্তারিত)
PLAY RECAP-এ `failed=1`	উপরের লাল টাস্কের বার্তা পড়ুন; `-v` দিয়ে বিস্তারিত দেখুন

৯.৯ ক্লিনআপ

প্লেবুকটি রেখে দিন — এটি Project ০১ ও ক্যাপস্টোনের বেসলাইন হবে। `/etc/motd` পরীক্ষার জন্য বদলাতে চাইলে প্লেবুকেই বদলে আবার চালান, হাতে নয়।

৯.১০ রিভিউ প্রশ্ন

(১) Play, Task ও Module-এর সম্পর্ক কী? (২) `--check` মোডের সীমা কী — কোন টাস্কে এটি ভুল ফল দিতে পারে? (৩) Idempotency যাচাই করতে প্লেবুক দুবার চালানো কেন জরুরি? (৪) `become: true` প্লে-স্তরে দিলে কী হয়? (৫) `--limit` কখন ব্যবহার করবেন?

Lab ১০ • ভেরিয়েবল ও ফ্যাক্ট

Lab ১০ — ভেরিয়েবল ও ফ্যাক্ট

১০.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) ভেরিয়েবলের অগ্রাধিকার (Precedence) বুঝতে ও ব্যবহার করতে; (২) `group_vars`, `host_vars`, প্লে ভেরিয়েবল ও `extra_vars` ব্যবহার করতে; (৩) Ansible Facts (`ansible_facts`) পড়ে সিদ্ধান্ত নিতে; (৪) `register` দিয়ে টাস্কের ফলাফল সংরক্ষণ করতে।

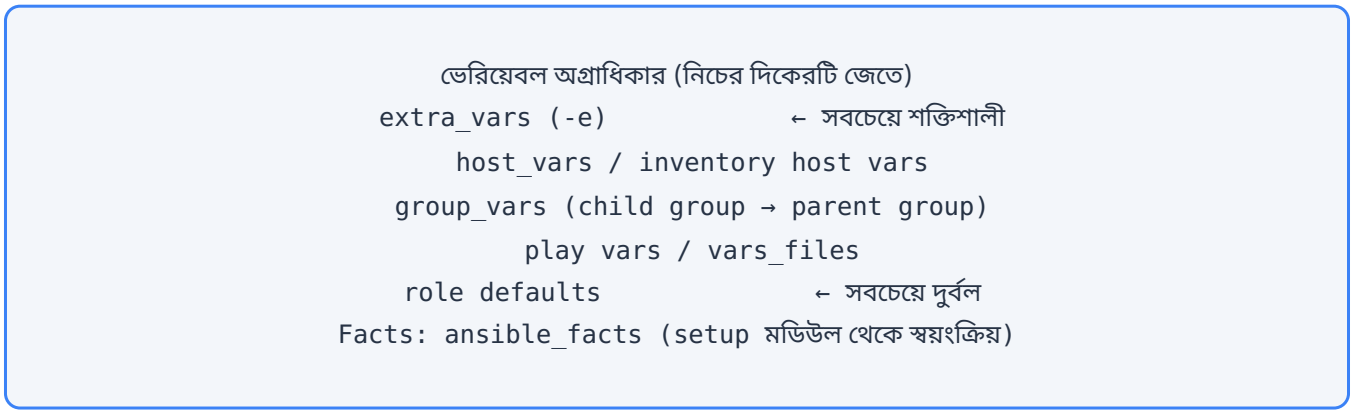
১০.২ প্রয়োজনীয় টুলস

Lab ১–৯ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি, প্লেবুকের অভিজ্ঞতা।

১০.৩ পূর্বশর্ত

Lab ৩-এর `group_vars` ফাইলগুলো আছে; Lab ৯-এর প্লেবুক চলে।

১০.৪ আর্কিটেকচার



১০.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — হোস্ট ভেরিয়েবল তৈরি করুন।

```
cat > ~/ansible-labs/host_vars/web.yml << 'EOF'
app_name: technova-web
app_port: 8080
admin_email: admin@technova.example
EOF
cat > ~/ansible-labs/host_vars/db.yml << 'EOF'
app_name: technova-db
db_engine: postgresql
EOF
```

ধাপ ২ — Facts দেখুন ও ব্যবহার করুন।

```
ansible all -m setup -a 'filter=ansible_facts' | grep -E 'ansible_distribution|
ansible_memtotal_mb|ansible_processor_vcpus'
ansible web -m debug -a 'msg="OS: {{ ansible_facts.distribution }}
{{ ansible_facts.distribution_version }}, RAM: {{ ansible_facts.memtotal_mb }} MB, CPU:
{{ ansible_facts.processor_vcpus }}"'
```

ধাপ ৩ — ভেরিয়েবল ও Facts একসঙ্গে ব্যবহার করে প্লেবুক লিখুন।

```
cat > ~/ansible-labs/playbooks/variables-demo.yml << 'EOF'
---
- name: ভেরিয়েবল ও ফ্যাক্ট প্রদর্শন
  hosts: all
  gather_facts: true
  vars:
    environment: lab
    welcome_text: "স্বাগতম {{ app_name | default('technova') }} — {{ environment }}"
  tasks:
    - name: হোস্টের তথ্য দেখান
      ansible.builtin.debug:
        msg: "{{ welcome_text }} | OS {{ ansible_facts['distribution'] }}
{{ ansible_facts['distribution_version'] }} | IP {{ ansible_facts['default_ipv4']
['address'] }}"

    - name: ডিস্ক ব্যবহার পরীক্ষা করে সংরক্ষণ করুন
      ansible.builtin.shell: df -h / | tail -1 | awk '{print $5}'
      register: disk_usage
      changed_when: false

    - name: ডিস্ক ব্যবহার ৮০%-এর বেশি হলে সতর্ক করুন
      ansible.builtin.debug:
        msg: "সতর্কতা: {{ inventory_hostname }}-এ ডিস্ক {{ disk_usage.stdout }} পূর্ণ"
      when: disk_usage.stdout | replace('%', '') | int > 80
EOF
ansible-playbook ~/ansible-labs/playbooks/variables-demo.yml
```

ধাপ ৪ — `extra_vars` দিয়ে অগ্রাধিকার পরীক্ষা করুন।

```
ansible-playbook ~/ansible-labs/playbooks/variables-demo.yml -e "environment=staging"
ansible-playbook ~/ansible-labs/playbooks/variables-demo.yml -e "app_name=override-test"
```

> **TIP:** গোপন তথ্য (পাসওয়ার্ড, টোকেন) কখনো `group_vars` -এ সরাসরি লিখবেন না — Lab ১৬-এর Vault ব্যবহার করুন। আর `extra_vars` কমান্ড লাইনে লিখলে তা শেলের ইতিহাসে থেকে যায় — সিক্রেটের জন্য এটি ব্যবহার করবেন না।

১০.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
ok: [web] => {"msg": "স্বাগতম technova-web — lab | OS Ubuntu 24.04 | IP 203.0.113.11"}
ok: [api] => {"msg": "স্বাগতম technova — lab | OS Ubuntu 24.04 | IP 203.0.113.12"}
ok: [db]  => {"msg": "স্বাগতম technova-db — lab | OS Ubuntu 24.04 | IP 203.0.113.13"}
```

১০.৭ যাচাইকরণ (Verification)

web-এ `technova-web`, db-তে `technova-db`, api-তে ডিফল্ট `technova` দেখা যাবে। `-e environment=staging` দিলে বার্তায় `staging` আসবে — এতে অগ্রাধিকার প্রমাণিত হবে।

১০.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`undefined variable`	ভেরিয়েবলটি সংজ্ঞায়িত হয়নি — `| default('মান')` ফিল্টার ব্যবহার করুন, অথবা ফাইলের নাম/গ্রুপ যাচাই করুন
Facts খালি আসছে	`gather_facts: false` থাকলে Facts সংগ্রহ হয় না; প্লেতে `gather_facts: true` রাখুন
স্ট্রিং তুলনা ভুল ফল দিচ্ছে	`disk_usage.stdout` স্ট্রিং — সংখ্যায় রূপান্তর করতে `| int` ব্যবহার করুন

১০.৯ ক্লিনআপ

পরীক্ষার `override-test` মান কোথাও সংরক্ষিত হয়নি — `extra_vars` শুধু ওই রানের জন্য। `host_vars` ফাইলগুলো রেখে দিন।

১০.১০ রিভিউ প্রশ্ন

(১) `extra_vars` সবচেয়ে শক্তিশালী কেন, এবং এতে কী ঝুঁকি আছে? (২) Facts সংগ্রহ বন্ধ করলে কী সুবিধা ও অসুবিধা? (৩) `register` ও `set_fact`-এর পার্থক্য কী? (৪) `default()` ফিল্টার কখন লাগে? (৫) `group_vars/all.yml` ও `group_vars/webservers.yml`-এর মধ্যে কোনটি জেতে?

Lab ১১ • কন্ডিশনাল ও লুপ

Lab ১১ — কন্ডিশনাল ও লুপ

১১.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) `when` দিয়ে শর্তসাপেক্ষ টাস্ক চালাতে; (২) `loop` দিয়ে তালিকার ওপর পুনরাবৃত্তি করতে; (৩) `loop`-এর সঙ্গে `dict` ও শর্ত মেশাতে; (৪) `block` / `rescue` দিয়ে ত্রুটি সামলাতে।

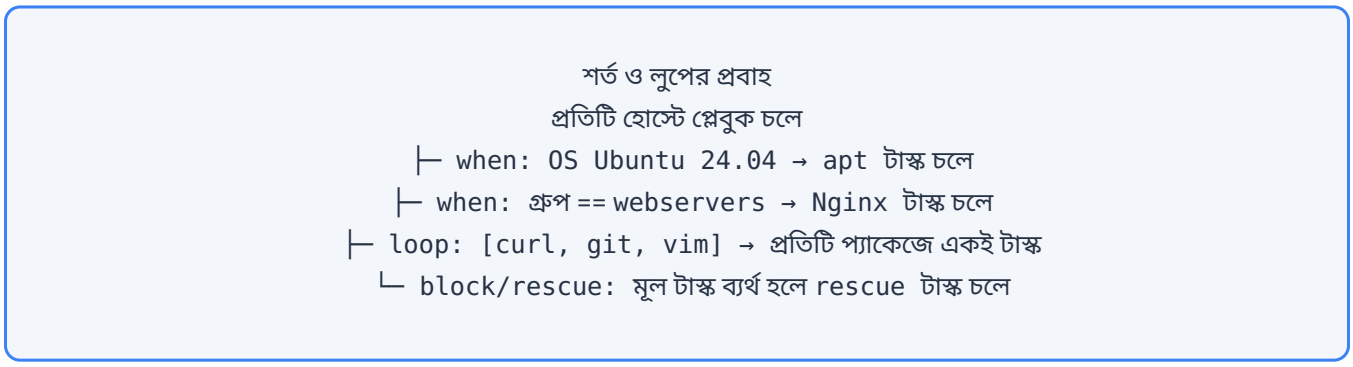
১১.২ প্রয়োজনীয় টুলস

Lab ১–১০ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি।

১১.৩ পূর্বশর্ত

ভেরিয়েবল ও Facts-এর ধারণা (Lab ১০); Jinja2-এর প্রাথমিক ধারণা।

১১.৪ আর্কিটেকচার



১১.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — শর্তসাপেক্ষ প্লেবুক লিখুন।

```
cat > ~/ansible-labs/playbooks/conditionals-loops.yml << 'EOF'
---
- name: শর্ত ও লুপের অনুশীলন
  hosts: all
  become: true
  vars:
    base_packages:
      - curl
      - git
      - vim
    web_packages:
      - nginx
      - certbot
  tasks:
    - name: OS অনুযায়ী বেস প্যাকেজ ইনস্টল (শুধু Ubuntu)
      ansible.builtin.apt:
        name: "{{ item }}"
```

```
        state: present
      loop: "{{ base_packages }}"
      when: ansible_facts['distribution'] == 'Ubuntu'

    - name: শুধু ওয়েব সার্ভারে ওয়েব প্যাকেজ ইনস্টল
      ansible.builtin.apt:
        name: "{{ item }}"
        state: present
      loop: "{{ web_packages }}"
      when: "'webservers' in group_names"

    - name: ডিস্ক ৮০%-এর বেশি পূর্ণ হলে সতর্কতা লিখুন
      ansible.builtin.copy:
        content: "সতর্কতা: ডিস্ক প্রায় পূর্ণ — {{ inventory_hostname }}\n"
        dest: /tmp/disk-warning.txt
        mode: "0644"
      when: ansible_facts['mounts'] | selectattr('mount', 'equalto', '/') |
map(attribute='size_total') | first > 0
EOF
ansible-playbook ~/ansible-labs/playbooks/conditionals-loops.yml
```

ধাপ ২ — ডিকশনারি লুপ ও `loop_control` ব্যবহার করুন।

```
cat > ~/ansible-labs/playbooks/users-loop.yml << 'EOF'
---
- name: একাধিক ব্যবহারকারী তৈরি
  hosts: all
  become: true
  vars:
    lab_users:
      - { name: "alice", groups: "developers", shell: "/bin/bash" }
      - { name: "bob", groups: "developers", shell: "/bin/bash" }
      - { name: "carol", groups: "devops", shell: "/bin/bash" }
  tasks:
    - name: ব্যবহারকারী তৈরি করুন
      ansible.builtin.user:
        name: "{{ item.name }}"
        groups: "{{ item.groups }}"
        append: true
        shell: "{{ item.shell }}"
        create_home: true
      loop: "{{ lab_users }}"
      loop_control:
        label: "{{ item.name }}"
EOF
ansible-playbook ~/ansible-labs/playbooks/users-loop.yml
```

ধাপ ৩ — `block` / `rescue` দিয়ে ত্রুটি সামলান।

```
cat > ~/ansible-labs/playbooks/block-rescue.yml << 'EOF'
---
- name: ত্রুটি সামলানোর অনুশীলন
  hosts: webservers
  become: true
  tasks:
    - name: Nginx কনফিগ আপডেটের চেষ্টা
      block:
```

```
        - name: ভুল কনফিগ ইচ্ছাকৃতভাবে বসান (পরীক্ষা)
          ansible.builtin.copy:
            content: "server { invalid directive here"
            dest: /etc/nginx/sites-available/broken.conf
            mode: "0644"
        - name: কনফিগ পরীক্ষা করুন
          ansible.builtin.command: nginx -t
          changed_when: false
      rescue:
        - name: ভাঙা কনফিগ মুছে ফেলুন
          ansible.builtin.file:
            path: /etc/nginx/sites-available/broken.conf
            state: absent
        - name: ব্যর্থতার বার্তা দেখান
          ansible.builtin.debug:
            msg: "কনফিগ ভুল ছিল — ভাঙা ফাইল মুছে ফেলা হয়েছে, Nginx নিরাপদ"
      always:
        - name: Nginx চলছে কি না নিশ্চিত করুন
          ansible.builtin.service:
            name: nginx
            state: started
EOF
ansible-playbook ~/ansible-labs/playbooks/block-rescue.yml
```

> **WARNING:** `when`-এ Jinja2 ডিলিমিটার (`{{ }}`) ব্যবহার করবেন না — `when: ansible_facts['distribution'] == 'Ubuntu'` লিখুন, `when: "{{ ... }}"` নয়। এটি সবচেয়ে সাধারণ নতুনদের ভুল।

১১.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
TASK [ব্যবহারকারী তৈরি করুন] ********************************
ok: [web] => (item=alice)
ok: [web] => (item=bob)
ok: [web] => (item=carol)
TASK [ব্যর্থতার বার্তা দেখান] ********************************
ok: [web] => {"msg": "কনফিগ ভুল ছিল — ভাঙা ফাইল মুছে ফেলা হয়েছে, Nginx নিরাপদ"}
```

১১.৭ যাচাইকরণ (Verification)

`ansible all -m shell -a 'id alice; id bob; id carol'`-এ তিনজন ব্যবহারকারী দেখা যাবে। `/etc/nginx/sites-available/broken.conf` ফাইলটি থাকবে না। Nginx এখনো চলছে কি না `curl http://203.0.113.11/` দিয়ে যাচাই করুন।

১১.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
	`{{ }}` ব্যবহার করেছেন কি না দেখুন; স্ট্রিং বনাম সংখ্যা তুলনায় `| int` ব্যবহার করুন

`when` শর্ত সবসময় সত্য/মিথ্যা	
লুপে `item` undefined	`loop` প্যারামিটারটি টাস্ক-স্তরে আছে কি না দেখুন; `with_items` (পুরোনো) ও `loop` মেশাবেন না
`rescue` চলছে না	`block`-এর ভেতরের টাস্কটি `failed` না হয়ে `changed` হয়েছে কি না দেখুন — `failed_when` দিয়ে ব্যর্থতা সংজ্ঞায়িত করুন

১১.৯ ক্লিনআপ

```
ansible all -m user -a 'name={{ item }} state=absent remove=yes' --become --extra-vars
'{"item": "alice"}'
ansible all -m user -a 'name=bob state=absent remove=yes' --become
ansible all -m user -a 'name=carol state=absent remove=yes' --become
```

১১.১০ রিভিউ প্রশ্ন

(১) `when`-এ `{{ }}` ব্যবহার করা যায় না কেন? (২) `loop` ও `with_items`-এর পার্থক্য কী? (৩) `block` / `rescue` / `always`-এর প্রত্যেকটি কখন চলে? (৪) `loop_control.label` কী সুবিধা দেয়? (৫) শর্তে `group_names` ব্যবহারের সুবিধা কী?

Lab ১২ • Jinja2 টেমপ্লেট

Lab ১২ — Jinja2 টেমপ্লেট

১২.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) Jinja2 টেমপ্লেট (`.j2`) লিখতে; (২) ভেরিয়েবল, লুপ ও শর্ত টেমপ্লেটে ব্যবহার করতে; (৩) `template` মডিউল দিয়ে কনফিগ ফাইল তৈরি করতে; (৪) টেমপ্লেটের ভুল নির্ণয় করতে।

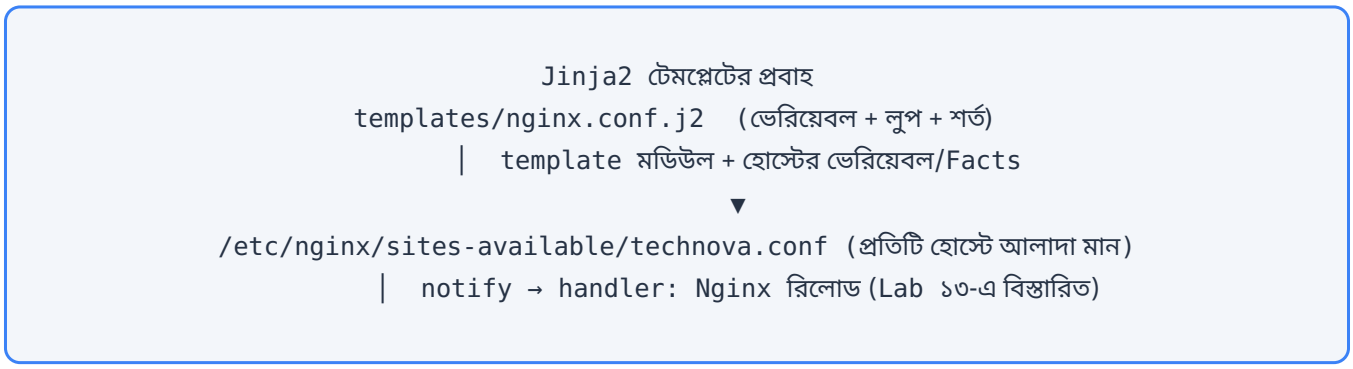
১২.২ প্রয়োজনীয় টুলস

Lab ১–১১ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি।

১২.৩ পূর্বশর্ত

ভেরিয়েবল, Facts ও লুপের ধারণা (Lab ১০–১১); Nginx কনফিগের ধারণা (Lab ৮)।

১২.৪ আর্কিটেকচার



১২.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Nginx টেমপ্লেট লিখুন।

```
mkdir -p ~/ansible-labs/templates
cat > ~/ansible-labs/templates/nginx.conf.j2 << 'EOF'
# TechNova Nginx কনফিগ — {{ inventory_hostname }}-এর জন্য তৈরি
# কোম্পানি: {{ company }} | পরিবেশ: {{ environment | default('lab') }}
server {
    listen {{ http_port | default(80) }};
    server_name {{ server_name | default(inventory_hostname + '.' + domain) }};

    root /var/www/technova;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

{% if inventory_hostname in groups['appservers'] or enable_api_proxy | default(false) %}
    location /api/ {
```

```
        proxy_pass http://{{ api_host | default('203.0.113.12') }}:{{ api_port |
default(3000) }}/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
{% endif %}

{% for path in extra_locations | default([]) %}
    location {{ path.url }} {
        proxy_pass {{ path.backend }};
    }
{% endfor %}
}
EOF
```

ধাপ ২ — টেমপ্লেট ব্যবহার করে প্লেবুক লিখুন।

```
cat > ~/ansible-labs/playbooks/nginx-template.yml << 'EOF'
---
- name: টেমপ্লেট দিয়ে Nginx কনফিগ তৈরি
  hosts: webservers
  become: true
  vars:
    enable_api_proxy: true
    api_host: 203.0.113.12
    api_port: 3000
    extra_locations:
      - { url: "/health", backend: "http://127.0.0.1:3000/health" }
  tasks:
    - name: Nginx কনফিগ টেমপ্লেট থেকে তৈরি করুন
      ansible.builtin.template:
        src: ../templates/nginx.conf.j2
        dest: /etc/nginx/sites-available/technova.conf
        mode: "0644"
        validate: "nginx -t -c /etc/nginx/nginx.conf"
      notify: Nginx রিলোড করুন

    - name: সাইট সক্রিয় করুন
      ansible.builtin.file:
        src: /etc/nginx/sites-available/technova.conf
        dest: /etc/nginx/sites-enabled/technova.conf
        state: link

  handlers:
    - name: Nginx রিলোড করুন
      ansible.builtin.service:
        name: nginx
        state: reloaded
EOF
ansible-playbook ~/ansible-labs/playbooks/nginx-template.yml
```

ধাপ ৩ — তৈরি হওয়া ফাইলটি দেখুন।

```
ansible web -m shell -a 'cat /etc/nginx/sites-available/technova.conf'
ansible web -m shell -a 'nginx -t' --become
```

> **TIP:** টেমপ্লেটে Nginx-এর `$uri`, `$host` লিখতে Jinja2-এর সঙ্গে সংঘাত হয় না, কারণ Jinja2 শুধু `{{ }}` ও `{% %}` খোঁজে। কিন্তু যদি কনফিগে আক্ষরিক `{{ }}` দরকার হয়, তবে `{% raw %}...{% endraw %}` ব্যবহার করুন।

১২.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট — web নোডে তৈরি ফাইল
# TechNova Nginx কনফিগ — web-এর জন্য তৈরি
server {
    listen 80;
    server_name web.technova.example;
    location /api/ {
        proxy_pass http://203.0.113.12:3000/;
    }
    location /health {
        proxy_pass http://127.0.0.1:3000/health;
    }
}
```

১২.৭ যাচাইকরণ (Verification)

তৈরি ফাইলে `web.technova.example`, `203.0.113.12:3000` ও `/health` লোকেশন থাকবে; `nginx -t` সফল হবে; `curl http://203.0.113.11/` পাতা দেবে। ভেরিয়েবল বদলে (যেমন `api_port: 4000`) আবার চালালে ফাইল বদলে যাবে — এটাই টেমপ্লেটের শক্তি।

১২.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`AnsibleUndefinedVariable`	ভেরিয়েবল সংজ্ঞায়িত হয়নি — `| default(...)` ব্যবহার করুন, অথবা `group_vars`-এ যোগ করুন
`TemplateSyntaxError`	`{% if %}`-এর সঙ্গে `{% endif %}` মিলছে কি না দেখুন; `ansible-playbook --syntax-check` চালান
টেমপ্লেটে `$` ভেরিয়েবল বদলে যাচ্ছে	Jinja2 `$` ছোঁয় না — সমস্যা হলে `{% raw %}` ব্যবহার করুন

১২.৯ ক্লিনআপ

টেমপ্লেট ও প্লেবুক রেখে দিন — Lab ১৪-এর রোলে এগুলো যাবে। `extra_locations`-এর পরীক্ষামূলক মান প্লেবুকেই আছে, আলাদা করে মুছতে হবে না।

১২.১০ রিভিউ প্রশ্ন

(১) `copy` ও `template` মডিউলের পার্থক্য কী? (২) টেমপ্লেটে `{% if %}` ও `{% for %}` কখন ব্যবহার করবেন? (৩) `validate` প্যারামিটার টেমপ্লেটে কেন বিশেষভাবে জরুরি? (৪) `{{ }}` ও `{% %}`-এর পার্থক্য কী? (৫) `default()` ফিল্টার টেমপ্লেটে কেন বারবার লাগে?

Lab ১৩ • হ্যান্ডলার

Lab ১৩ — হ্যান্ডলার ও নোটিফিকেশন

১৩.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) হ্যান্ডলার লিখতে ও `notify` দিয়ে ট্রিগার করতে; (২) বুঝতে কখন হ্যান্ডলার চলে আর কখন চলে না; (৩) একাধিক টাস্ক থেকে একই হ্যান্ডলার ট্রিগার করতে; (৪) `flush_handlers` ও `listen` ব্যবহার করতে।

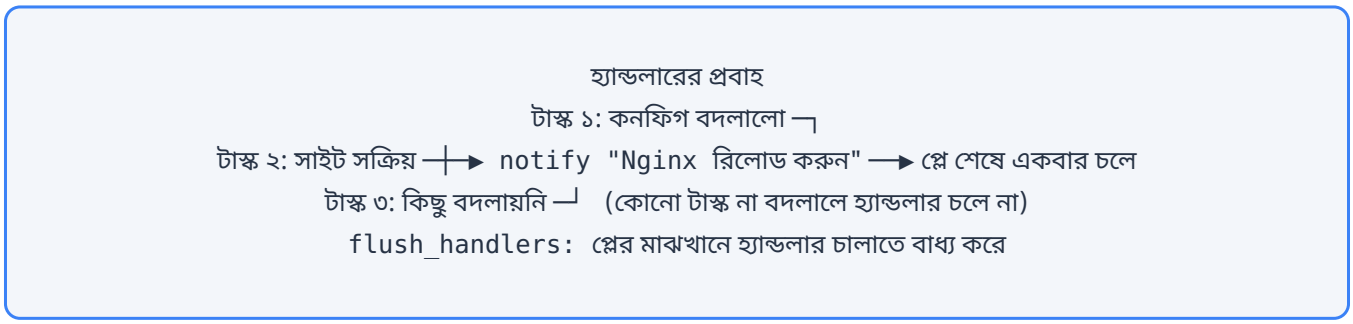
১৩.২ প্রয়োজনীয় টুলস

Lab ১–১২ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি, Nginx টেমপ্লেট (Lab ১২)।

১৩.৩ পূর্বশর্ত

টেমপ্লেট মডিউল ও প্লেবুকের ধারণা; web নোডে Nginx চলছে।

১৩.৪ আর্কিটেকচার



১৩.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — হ্যান্ডলারসহ প্লেবুক লিখুন।

```
cat > ~/ansible-labs/playbooks/handlers-demo.yml << 'EOF'
---
- name: হ্যান্ডলারের অনুশীলন
  hosts: webservers
  become: true
  tasks:
    - name: Nginx প্রধান কনফিগ আপডেট করুন
      ansible.builtin.template:
        src: ../templates/nginx.conf.j2
        dest: /etc/nginx/sites-available/technova.conf
        mode: "0644"
        validate: "nginx -t -c /etc/nginx/nginx.conf"
      notify:
        - Nginx রিলোড করুন
        - Nginx অবস্থা যাচাই করুন

    - name: ওয়েব রুটের সূচক পাতা আপডেট করুন
      ansible.builtin.copy:
```

```
        content: "<h1>TechNova — হ্যান্ডলার সংস্করণ</h1>\n"
        dest: /var/www/technova/index.html
        mode: "0644"
      notify: Nginx রিলোড করুন

    - name: হ্যান্ডলার এখনই চালান (পরের টাস্কের আগে দরকার)
      ansible.builtin.meta: flush_handlers

    - name: ওয়েবসাইট সাড়া দিচ্ছে কি না পরীক্ষা করুন
      ansible.builtin.uri:
        url: http://localhost/
        status_code: 200
      register: site_check

    - name: পরীক্ষার ফল দেখান
      ansible.builtin.debug:
        msg: "সাইটের অবস্থা: {{ site_check.status }}"

  handlers:
    - name: Nginx রিলোড করুন
      ansible.builtin.service:
        name: nginx
        state: reloaded

    - name: Nginx অবস্থা যাচাই করুন
      ansible.builtin.command: nginx -t
      changed_when: false
EOF
ansible-playbook ~/ansible-labs/playbooks/handlers-demo.yml
```

ধাপ ২ — হ্যান্ডলার না চলার পরীক্ষা করুন। একই প্লেবুক আবার চালান — কোনো টাস্ক `changed` হবে না, তাই কোনো হ্যান্ডলারও চলবে না। আউটপুটে `RUNNING HANDLER` দেখা যাবে না।

```
ansible-playbook ~/ansible-labs/playbooks/handlers-demo.yml | grep -E 'RUNNING HANDLER|PLAY RECAP' -A 2
```

ধাপ ৩ — `listen` দিয়ে একাধিক নামে এক হ্যান্ডলার ব্যবহার করুন।

```
cat > ~/ansible-labs/playbooks/handlers-listen.yml << 'EOF'
---
- name: listen-এর অনুশীলন
  hosts: webservers
  become: true
  tasks:
    - name: কনফিগ বদলান
      ansible.builtin.copy:
        content: "# TechNova — {{ ansible_date_time.date }}\n"
        dest: /etc/nginx/conf.d/technova-note.conf
        mode: "0644"
      notify: ওয়েব সার্ভিস পুনরায় চালু করুন

  handlers:
    - name: Nginx রিলোড করুন
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

```
      listen: ওয়েব সার্ভিস পুনরায় চালু করুন
EOF
ansible-playbook ~/ansible-labs/playbooks/handlers-listen.yml
```

> **CONCEPT:** হ্যান্ডলার একটি প্লে-তে যতবারই নোটিফাই হোক না কেন, প্লে শেষে মাত্র একবার চলে। এতে সার্ভিস বারবার রিস্টার্ট হয় না — দশটি কনফিগ ফাইল বদলালেও Nginx একবারই রিলোড হয়। হ্যান্ডলার কেবল তখনই চলে যখন নোটিফাই করা টাস্কটি `changed` হয়।

১৩.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
RUNNING HANDLER [Nginx রিলোড করুন] **********************
changed: [web]
RUNNING HANDLER [Nginx অবস্থা যাচাই করুন] ***************
ok: [web]
PLAY RECAP *********************************************
web  : ok=6  changed=3  unreachable=0  failed=0
```

১৩.৭ যাচাইকরণ (Verification)

প্রথম রানে `RUNNING HANDLER` দেখা যাবে; দ্বিতীয় রানে দেখা যাবে না। `curl http://203.0.113.11/`-এ "হ্যান্ডলার সংস্করণ" পাতা আসবে। `flush_handlers`-এর পরের `uri` টাস্কটি 200 পাবে।

১৩.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
হ্যান্ডলার চলছে না	নোটিফাই করা টাস্কটি `changed` হয়নি — টাস্কটি আসলেই কিছু বদলেছে কি না দেখুন; নামের বানান হুবহু মিলতে হবে
প্লে ব্যর্থ হলে হ্যান্ডলার চলল না	প্লে ব্যর্থ হলে হ্যান্ডলার চলে না — `--force-handlers` ফ্ল্যাগ ব্যবহার করুন, অথবা `flush_handlers` দিয়ে আগেই চালান
হ্যান্ডলারে ভুল নাম	`listen` ব্যবহার করলে টাস্কে `listen`-এর নাম দিতে হয়, হ্যান্ডলারের `name` নয়

১৩.৯ ক্লিনআপ

```
ansible web -m file -a 'path=/etc/nginx/conf.d/technova-note.conf state=absent' --become
ansible web -m service -a 'name=nginx state=reloaded' --become
```

১৩.১০ রিভিউ প্রশ্ন

(১) হ্যান্ডলার কখন চলে, আর কখন চলে না? (২) একই হ্যান্ডলার দশবার নোটিফাই হলে কতবার চলবে, এবং কেন? (৩) `flush_handlers` কখন দরকার? (৪) `--force-handlers` কখন ব্যবহার করবেন? (৫) `listen` কী সুবিধা দেয়?

Lab ১৪ • রোল

Lab ১৪ — Ansible রোল

১৪.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) `ansible-galaxy init` দিয়ে রোলের কাঠামো তৈরি করতে; (২) রোলের ডিরেক্টরি (`tasks`, `handlers`, `templates`, `defaults`, `vars`, `files`, `meta`) বুঝতে ও ব্যবহার করতে; (৩) প্লেবুক থেকে রোল কল করতে; (৪) রোলের ডিফল্ট ভেরিয়েবল ওভাররাইড করতে।

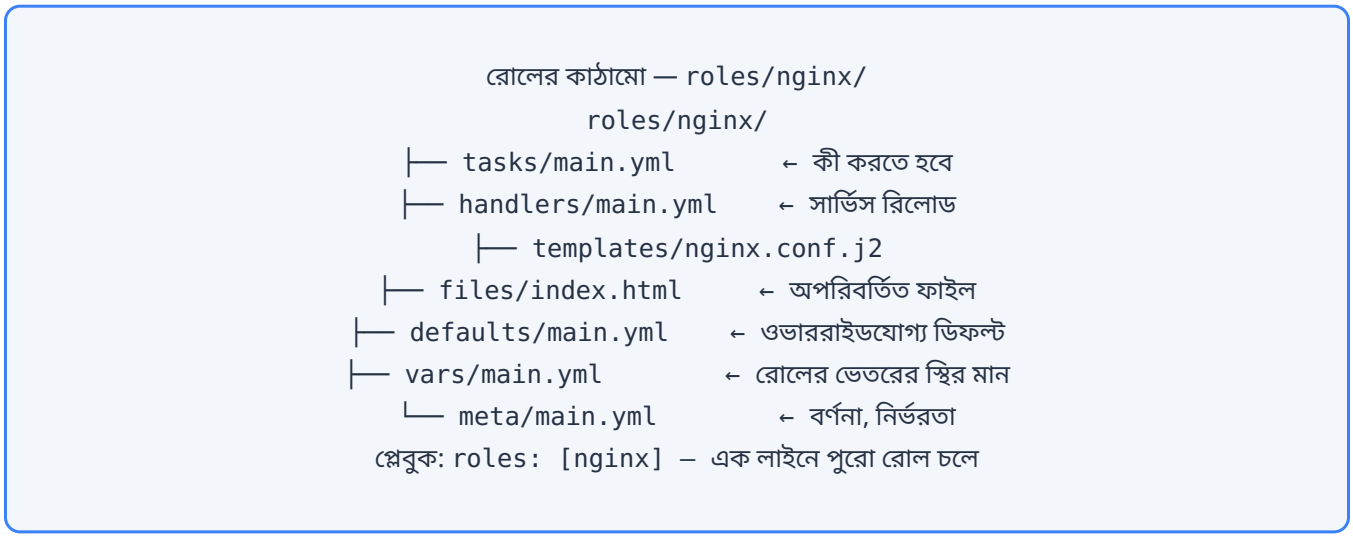
১৪.২ প্রয়োজনীয় টুলস

Lab ১–১৩ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি।

১৪.৩ পূর্বশর্ত

প্লেবুক, টেমপ্লেট ও হ্যান্ডলারের অভিজ্ঞতা (Lab ৯, ১২, ১৩)।

১৪.৪ আর্কিটেকচার



১৪.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — রোল তৈরি করুন।

```
cd ~/ansible-labs
ansible-galaxy init roles/nginx
ansible-galaxy init roles/common
find roles/nginx -type f | sort
```

ধাপ ২ — `common` রোল লিখুন (সব সার্ভারের বেসলাইন)।

```
cat > roles/common/defaults/main.yml << 'EOF'
common_packages:
```

```
  - curl
  - git
  - vim
  - htop
  - ufw
common_timezone: Asia/Dhaka
EOF
cat > roles/common/tasks/main.yml << 'EOF'
---
- name: বেস প্যাকেজ ইনস্টল করুন
  ansible.builtin.apt:
    name: "{{ common_packages }}"
    state: present
    update_cache: true
    cache_valid_time: 3600

- name: টাইমজোন সেট করুন
  community.general.timezone:
    name: "{{ common_timezone }}"
EOF
```

ধাপ ৩ — `nginx` রোল লিখুন।

```
cat > roles/nginx/defaults/main.yml << 'EOF'
nginx_http_port: 80
nginx_server_name: "{{ inventory_hostname }}.technova.example"
nginx_enable_api_proxy: true
nginx_api_host: 203.0.113.12
nginx_api_port: 3000
EOF
cat > roles/nginx/tasks/main.yml << 'EOF'
---
- name: Nginx ইনস্টল করুন
  ansible.builtin.apt:
    name: nginx
    state: present

- name: Nginx কনফিগ টেমপ্লেট থেকে তৈরি করুন
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/sites-available/technova.conf
    mode: "0644"
    validate: "nginx -t -c /etc/nginx/nginx.conf"
  notify: Nginx রিলোড করুন

- name: সাইট সক্রিয় করুন
  ansible.builtin.file:
    src: /etc/nginx/sites-available/technova.conf
    dest: /etc/nginx/sites-enabled/technova.conf
    state: link
  notify: Nginx রিলোড করুন

- name: Nginx চালু ও সক্রিয় করুন
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
EOF
```

```
cat > roles/nginx/handlers/main.yml << 'EOF'
---
- name: Nginx রিলোড করুন
  ansible.builtin.service:
    name: nginx
    state: reloaded
EOF
cp ~/ansible-labs/templates/nginx.conf.j2 roles/nginx/templates/nginx.conf.j2
# টেমপ্লেটের ভেরিয়েবল নাম রোলের ডিফল্টের সঙ্গে মেলান
sed -i 's/{{ http_port | default(80) }}/{{ nginx_http_port }}/; s/{{ server_name |
default(inventory_hostname + '"'"'.'"'"' + domain) }}/{{ nginx_server_name }}/; s/{{ api_host
| default(.203.0.113.12.) }}/{{ nginx_api_host }}/; s/{{ api_port | default(3000) }}/
{{ nginx_api_port }}/; s/enable_api_proxy | default(false)/nginx_enable_api_proxy/' roles/
nginx/templates/nginx.conf.j2
cat roles/nginx/templates/nginx.conf.j2
```

ধাপ ৪ — রোল ব্যবহার করে প্লেবুক লিখুন ও চালান।

```
cat > ~/ansible-labs/playbooks/site.yml << 'EOF'
---
- name: সব সার্ভারে বেসলাইন
  hosts: all
  become: true
  roles:
    - common

- name: ওয়েব সার্ভার
  hosts: webservers
  become: true
  roles:
    - role: nginx
      vars:
        nginx_server_name: web.technova.example
EOF
ansible-playbook ~/ansible-labs/playbooks/site.yml
```

> **TIP:** `defaults/main.yml`-এর মান সবচেয়ে দুর্বল — প্লেবুক, গ্রুপ ভেরিয়েবল বা `extra_vars` দিয়ে সহজে ওভাররাইড করা যায়। `vars/main.yml`-এর মান শক্তিশালী — রোলের ভেতরের স্থির মান রাখুন, ব্যবহারকারী বদলাবে এমন মান `defaults`-এ রাখুন।

১৪.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
PLAY [সব সার্ভারে বেসলাইন] *******************************
TASK [common : বেস প্যাকেজ ইনস্টল করুন] ******************
ok: [web]
ok: [api]
ok: [db]
PLAY [ওয়েব সার্ভার] ***************************************
TASK [nginx : Nginx কনফিগ টেমপ্লেট থেকে তৈরি করুন] ********
ok: [web]
```

```
PLAY RECAP *********************************************
web  : ok=8  changed=0  unreachable=0  failed=0
```

১৪.৭ যাচাইকরণ (Verification)

PLAY RECAP-এ `failed=0` হতে হবে; `curl http://203.0.113.11/` পাতা দেবে; `ansible-playbook playbooks/site.yml` দ্বিতীয়বার চালালে `changed=0` হবে। `roles/nginx/`-এর প্রতিটি ডিরেক্টরির কাজ ব্যাখ্যা করতে পারা উচিত।

১৪.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`ERROR! the role 'nginx' was not found`	`ansible.cfg`-এ `roles_path = ./roles` আছে কি না দেখুন; প্লেবুকটি `~/ansible-labs` থেকে চালান
টেমপ্লেটে ভেরিয়েবল খালি আসছে	`defaults/main.yml`-এর নাম ও টেমপ্লেটের নাম মিলছে কি না দেখুন; `sed` বদলের পর ফাইলটি পড়ে দেখুন
হ্যান্ডলার চলছে না	হ্যান্ডলারটি `roles/nginx/handlers/main.yml`-এ আছে কি না দেখুন; রোলের হ্যান্ডলার স্বয়ংক্রিয়ভাবে লোড হয়

১৪.৯ ক্লিনআপ

রোলগুলো রেখে দিন — Project ০১–০৩ ও ক্যাপস্টোনে এগুলোই ব্যবহৃত হবে। পরীক্ষার জন্য তৈরি অস্থায়ী রোল থাকলে মুছে ফেলুন।

১৪.১০ রিভিউ প্রশ্ন

(১) রোল ব্যবহারের সুবিধা প্লেবুকের তুলনায় কী? (২) `defaults` ও `vars` ডিরেক্টরির পার্থক্য কী? (৩) `tasks/main.yml` ছাড়া অন্য টাস্ক ফাইল কীভাবে অন্তর্ভুক্ত করবেন? (৪) রোলের হ্যান্ডলার কীভাবে প্লেবুকের হ্যান্ডলারের সঙ্গে মেশে? (৫) `meta/main.yml`-এ কী লেখা হয়, এবং কেন?

Lab ১৫ • Galaxy কালেকশন

Lab ১৫ — Ansible Galaxy কালেকশন

১৫.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) Galaxy থেকে কালেকশন খুঁজে ও ইনস্টল করতে; (২) `requirements.yml` দিয়ে কালেকশন পিন করতে; (৩) কালেকশনের মডিউল (যেমন `community.general`, `community.docker`) ব্যবহার করতে; (৪) কালেকশনের সংস্করণ পরিচালনা করতে।

১৫.২ প্রয়োজনীয় টুলস

Lab ১–১৪ সম্পন্ন কন্ট্রোল নোড, ইন্টারনেট সংযোগ।

১৫.৩ পূর্বশর্ত

`ansible-galaxy` কমান্ড কাজ করছে (Lab ১৪); রোলের ধারণা আছে।

১৫.৪ আর্কিটেকচার



১৫.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — কালেকশন খুঁজুন ও ইনস্টল করুন।

```
ansible-galaxy collection search community.general
ansible-galaxy collection install community.general
ansible-galaxy collection install community.docker community.postgresql
ansible-galaxy collection list
```

ধাপ ২ — `requirements.yml` দিয়ে সংস্করণ পিন করুন।

```
cat > ~/ansible-labs/collections/requirements.yml << 'EOF'
---
collections:
```

```
  - name: community.general
    version: ">=9.0.0"
  - name: community.docker
    version: ">=3.0.0"
  - name: community.postgresql
    version: ">=3.0.0"
  - name: ansible.posix
    version: ">=1.5.0"
EOF
ansible-galaxy collection install -r ~/ansible-labs/collections/requirements.yml
```

ধাপ ৩ — কালেকশনের মডিউল ব্যবহার করুন।

```
cat > ~/ansible-labs/playbooks/collections-demo.yml << 'EOF'
---
- name: কালেকশন মডিউলের অনুশীলন
  hosts: all
  become: true
  tasks:
    - name: UFW ফায়ারওয়াল ইনস্টল ও সক্রিয় করুন
      community.general.ufw:
        state: enabled
        policy: deny

    - name: SSH পোর্ট খুলুন
      community.general.ufw:
        rule: allow
        port: "22"
        proto: tcp

    - name: HTTP পোর্ট খুলুন (শুধু ওয়েব সার্ভারে)
      community.general.ufw:
        rule: allow
        port: "80"
        proto: tcp
      when: "'webservers' in group_names"
EOF
ansible-playbook ~/ansible-labs/playbooks/collections-demo.yml
```

> **WARNING:** UFW সক্রিয় করার আগে SSH পোর্ট (22) খোলা নিশ্চিত করুন — নইলে নিজেই নিজেকে সার্ভার থেকে বন্ধ করে দেবেন। এই প্লেবুকে SSH নিয়মটি UFW সক্রিয় করার আগেই লেখা হয়েছে, তবে `ufw` মডিউল নিয়মের ক্রম নিশ্চিত করে না — প্রথমে `rule: allow port 22` আলাদা রানে চালিয়ে তারপর `state: enabled` করুন, অথবা কনসোল অ্যাক্সেস হাতে রাখুন।

ধাপ ৪ — কালেকশনের ডকুমেন্টেশন দেখুন।

```
ansible-doc community.general.ufw | head -30
ansible-doc community.docker.docker_container | head -20
```

১৫.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible-galaxy collection list
Collection        Version
----------------- -------
community.docker  3.12.0
community.general 9.5.0
community.postgresql 3.7.0
ansible.posix     1.6.2
```

১৫.৭ যাচাইকরণ (Verification)

`ansible-galaxy collection list`-এ চারটি কালেকশন দেখা যাবে। `ansible all -m shell -a 'sudo ufw status verbose'`-এ `Status: active` ও পোর্ট 22 খোলা দেখা যাবে। `ansible-doc` দিয়ে মডিউলের ডকুমেন্টেশন পড়া যাবে।

১৫.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`Failed to resolve collection`	ইন্টারনেট বা Galaxy সার্ভার সমস্যা — আবার চেষ্টা করুন; প্রক্সি থাকলে `https_proxy` সেট করুন
মডিউল খুঁজে পাওয়া যাচ্ছে না	কালেকশনের পুরো নাম (FQCN) ব্যবহার করুন: `community.general.ufw`, শুধু `ufw` নয়
UFW সক্রিয় করে SSH বন্ধ হয়ে গেল	VM কনসোল থেকে `sudo ufw allow 22/tcp && sudo ufw enable` করুন; ল্যাবে স্ন্যাপশট নিয়ে রাখুন

১৫.৯ ক্লিনআপ

কালেকশনগুলো রেখে দিন — Lab ১৯ ও ২১-এ লাগবে। UFW সক্রিয় রেখে দিন; Lab ১৭-এ হার্ডেনিংয়ে এটি আরও কঠোর করা হবে।

১৫.১০ রিভিউ প্রশ্ন

(১) কালেকশন ও রোলের পার্থক্য কী? (২) `requirements.yml` কেন ব্যবহার করবেন? (৩) FQCN (পুরো নাম) ব্যবহারের সুবিধা কী? (৪) কালেকশনের সংস্করণ পিন না করলে কী ঝুঁকি? (৫) Galaxy ছাড়া নিজের কালেকশন কীভাবে বানানো যায়?

Lab ১৬ • Ansible Vault

Lab ১৬ — Ansible Vault

১৬.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) `ansible-vault` দিয়ে গোপন ভেরিয়েবল এনক্রিপ্ট করতে; (২) ভল্ট পাসওয়ার্ড ফাইল নিরাপদে পরিচালনা করতে; (৩) এনক্রিপ্ট করা ভেরিয়েবল প্লেবুকে ব্যবহার করতে; (৪) ভল্ট ফাইল সম্পাদনা, পুনঃকী ও দেখার নিয়ম জানতে।

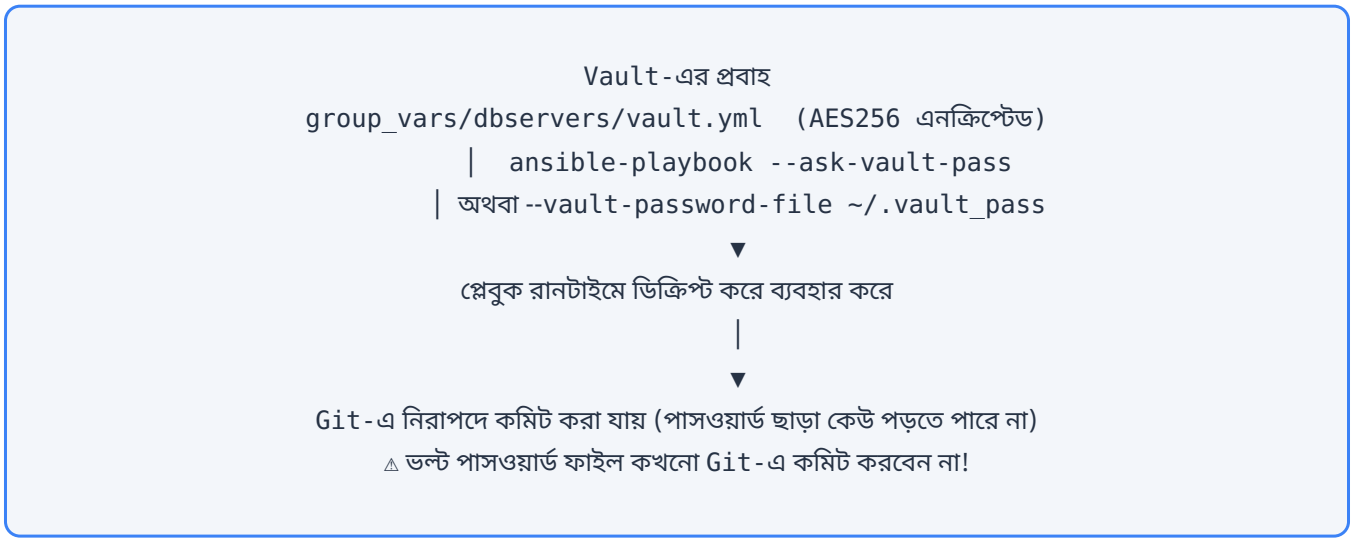
১৬.২ প্রয়োজনীয় টুলস

Lab ১–১৫ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি।

১৬.৩ পূর্বশর্ত

ভেরিয়েবলের ধারণা (Lab ১০); Git-এর প্রাথমিক ধারণা থাকলে ভালো।

১৬.৪ আর্কিটেকচার



১৬.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — ভল্ট পাসওয়ার্ড ফাইল তৈরি করুন।

```
echo 'CHANGE-ME-VAULT-PASSWORD' > ~/.vault_pass
chmod 600 ~/.vault_pass
# ansible.cfg-এ যোগ করুন (ঐচ্ছিক):
cat >> ~/ansible-labs/ansible.cfg << 'EOF'
vault_password_file = ~/.vault_pass
EOF
```

ধাপ ২ — এনক্রিপ্ট করা সিক্রেট ফাইল তৈরি করুন।

```
cat > /tmp/vault-secrets.yml << 'EOF'
db_admin_password: "CHANGE-ME-DB-ADMIN-PASSWORD"
db_app_password: "CHANGE-ME-DB-APP-PASSWORD"
api_jwt_secret: "CHANGE-ME-JWT-SECRET"
EOF
ansible-vault encrypt /tmp/vault-secrets.yml --vault-password-file ~/.vault_pass
cp /tmp/vault-secrets.yml ~/ansible-labs/group_vars/dbservers/vault.yml
ansible-vault view ~/ansible-labs/group_vars/dbservers/vault.yml --vault-password-file
~/.vault_pass
```

ধাপ ৩ — একক মান এনক্রিপ্ট করুন (inline)।

```
ansible-vault encrypt_string 'CHANGE-ME-API-KEY' --name 'api_key' --vault-password-file
~/.vault_pass
# আউটপুটটি group_vars/all.yml-এ বসান:
cat >> ~/ansible-labs/group_vars/all.yml << 'EOF'
# নিচের মানটি ansible-vault encrypt_string দিয়ে তৈরি — সরাসরি সম্পাদনা করবেন না
EOF
echo "এনক্রিপ্ট করা স্ট্রিংটি উপরের কমান্ডের আউটপুট থেকে group_vars/all.yml-এ বসান।"
```

ধাপ ৪ — ভল্ট ভেরিয়েবল ব্যবহার করে প্লেবুক চালান।

```
cat > ~/ansible-labs/playbooks/vault-demo.yml << 'EOF'
---
- name: ভল্ট সিক্রেট ব্যবহারের অনুশীলন
  hosts: dbservers
  become: true
  tasks:
    - name: ডেটাবেস অ্যাপ ব্যবহারকারীর পাসওয়ার্ড ফাইলে লিখুন (পরীক্ষা)
      ansible.builtin.copy:
        content: "app_password={{ db_app_password }}\n"
        dest: /tmp/db-app-credentials.txt
        owner: root
        group: root
        mode: "0600"
      no_log: true

    - name: ফাইলটি আছে কি না যাচাই করুন (মান না দেখিয়ে)
      ansible.builtin.stat:
        path: /tmp/db-app-credentials.txt
      register: cred_file

    - name: যাচাইয়ের ফল দেখান
      ansible.builtin.debug:
        msg: "ক্রেডেনশিয়াল ফাইল তৈরি হয়েছে: {{ cred_file.stat.exists }}"
EOF
ansible-playbook ~/ansible-labs/playbooks/vault-demo.yml --vault-password-file ~/.vault_pass
```

WARNING: `no_log: true` সিক্রেট ব্যবহার করা টাস্কে সবসময় দিন — নইলে পাসওয়ার্ড লগে বা `-v` আউটপুটে দেখা যাবে। ভল্ট পাসওয়ার্ড ফাইল (`~/.vault_pass`) কখনো Git-এ কমিট করবেন না; `.gitignore`-এ যোগ করুন। ভল্ট পাসওয়ার্ড হারালে এনক্রিপ্ট করা ফাইল আর খোলা যাবে না — পাসওয়ার্ড নিরাপদে সংরক্ষণ করুন।

ধাপ ৫ — ভল্ট ফাইল সম্পাদনা ও পুনঃকী করুন।

```
ansible-vault edit ~/ansible-labs/group_vars/dbservers/vault.yml --vault-password-file
~/.vault_pass
ansible-vault rekey ~/ansible-labs/group_vars/dbservers/vault.yml --vault-password-file
~/.vault_pass --new-vault-password-file /tmp/new-vault-pass
# পরীক্ষা শেষে পুরোনো পাসওয়ার্ড ফাইলেই ফিরে যান (ল্যাবের ধারাবাহিকতার জন্য)
```

১৬.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible-vault view ~/ansible-labs/group_vars/dbservers/vault.yml
db_admin_password: "CHANGE-ME-DB-ADMIN-PASSWORD"
db_app_password: "CHANGE-ME-DB-APP-PASSWORD"
api_jwt_secret: "CHANGE-ME-JWT-SECRET"
$ ansible-playbook playbooks/vault-demo.yml
ok: [db] => {"msg": "ক্রেডেনশিয়াল ফাইল তৈরি হয়েছে: True"}
```

১৬.৭ যাচাইকরণ (Verification)

`cat group_vars/dbservers/vault.yml`-এ শুধু `$ANSIBLE_VAULT;1.1;AES256` হেডার ও এনক্রিপ্টেড লেখা দেখা যাবে — আসল পাসওয়ার্ড দেখা যাবে না। `/tmp/db-app-credentials.txt` ফাইলটি db নোডে 600 অনুমতিতে থাকবে। প্লেবুকের আউটপুটে পাসওয়ার্ডের মান কোথাও দেখা যাবে না।

১৬.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`ERROR! Decryption failed`	ভুল ভল্ট পাসওয়ার্ড — সঠিক `--vault-password-file` বা `--ask-vault-pass` ব্যবহার করুন
পাসওয়ার্ড লগে দেখা যাচ্ছে	টাস্কে `no_log: true` যোগ করুন; `debug`-এ সিক্রেট ভেরিয়েবল ছাপাবেন না
`vault.yml` Git-এ প্লেইন টেক্সটে গেল	এনক্রিপ্ট করতে ভুলে গেছেন — `ansible-vault encrypt` চালান; Git ইতিহাস থেকে সরাতে `git filter-repo` লাগবে

১৬.৯ ক্লিনআপ

```
ansible db -m file -a 'path=/tmp/db-app-credentials.txt state=absent' --become
rm -f /tmp/vault-secrets.yml /tmp/new-vault-pass
```

১৬.১০ রিভিউ প্রশ্ন

(১) `ansible-vault encrypt` ও `encrypt_string`-এর পার্থক্য কী? (২) ভল্ট পাসওয়ার্ড ফাইল Git-এ কমিট করা যায় না কেন? (৩) `no_log: true` কী করে? (৪) ভল্ট পাসওয়ার্ড হারালে কী হবে? (৫) প্রোডাকশনে ভল্টের বদলে আর কী ব্যবহার করা যায় (যেমন HashiCorp Vault)?

Lab ১৭ • সার্ভার হার্ডেনিং

Lab ১৭ — Linux সার্ভার হার্ডেনিং

১৭.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) SSH সার্ভার কঠোর করতে (পাসওয়ার্ড লগইন বন্ধ, রুট লগইন বন্ধ); (২) UFW ফায়ারওয়াল নীতি প্রয়োগ করতে; (৩) স্বয়ংক্রিয় নিরাপত্তা আপডেট চালু করতে; (৪) `fail2ban` দিয়ে ব্রুট-ফোর্স আক্রমণ ঠেকাতে; (৫) হার্ডেনিং যাচাই করতে।

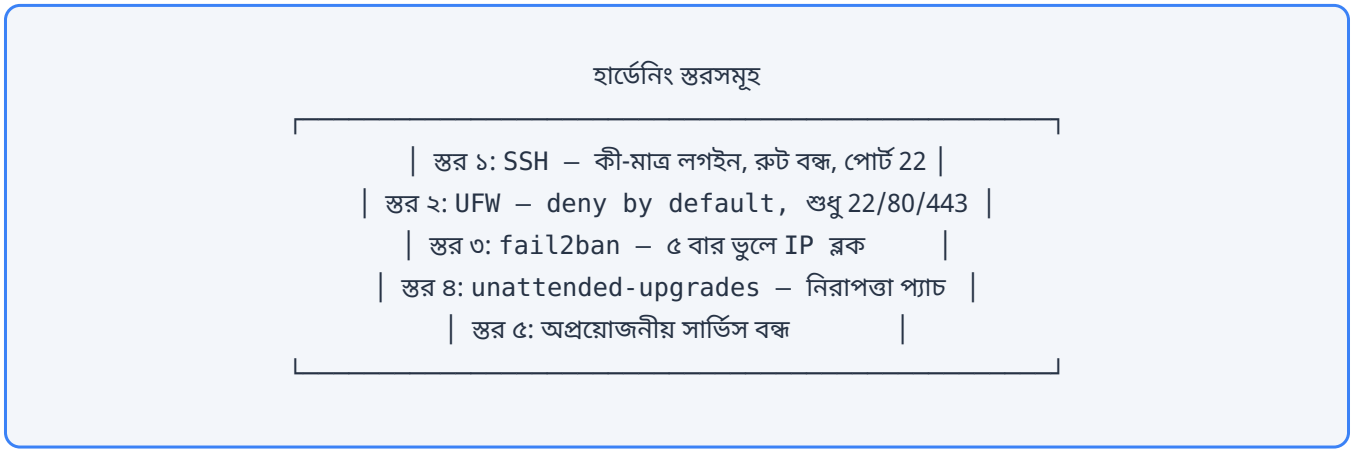
১৭.২ প্রয়োজনীয় টুলস

Lab ১–১৬ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি, Vault (Lab ১৬)।

১৭.৩ পূর্বশর্ত

SSH কী-লগইন নির্ভরযোগ্যভাবে কাজ করছে — পাসওয়ার্ড লগইন বন্ধ করার আগে এটি নিশ্চিত করা অপরিহার্য; নইলে সার্ভারে ঢোকা যাবে না।

১৭.৪ আর্কিটেকচার



১৭.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — SSH কনফিগ কঠোর করুন। `lineinfile` দিয়ে বিদ্যমান `sshd_config` বদলান — পুরো ফাইল হাতে লেখার চেয়ে নিরাপদ।

```
cat > ~/ansible-labs/playbooks/hardening.yml << 'EOF'
---
- name: Linux সার্ভার হার্ডেনিং
  hosts: all
  become: true
  tasks:
    - name: SSH-এ পাসওয়ার্ড প্রমাণীকরণ বন্ধ করুন
      ansible.builtin.lineinfile:
        path: /etc/ssh/sshd_config
        regexp: "^#?PasswordAuthentication"
```

```
      line: "PasswordAuthentication no"
      validate: "/usr/sbin/sshd -t -f %s"
    notify: SSH পুনরায় চালু করুন

  - name: রুট লগইন বন্ধ করুন
    ansible.builtin.lineinfile:
      path: /etc/ssh/sshd_config
      regexp: "^#?PermitRootLogin"
      line: "PermitRootLogin no"
      validate: "/usr/sbin/sshd -t -f %s"
    notify: SSH পুনরায় চালু করুন

  - name: খালি পাসওয়ার্ড বন্ধ করুন
    ansible.builtin.lineinfile:
      path: /etc/ssh/sshd_config
      regexp: "^#?PermitEmptyPasswords"
      line: "PermitEmptyPasswords no"
      validate: "/usr/sbin/sshd -t -f %s"
    notify: SSH পুনরায় চালু করুন

  - name: UFW ইনস্টল করুন
    ansible.builtin.apt:
      name: ufw
      state: present

  - name: SSH পোর্ট খুলুন (ফায়ারওয়াল সক্রিয় করার আগে)
    community.general.ufw:
      rule: allow
      port: "22"
      proto: tcp

  - name: HTTP/HTTPS পোর্ট খুলুন (শুধু ওয়েব সার্ভারে)
    community.general.ufw:
      rule: allow
      port: "{{ item }}"
      proto: tcp
    loop: ["80", "443"]
    when: "'webservers' in group_names"

  - name: UFW ডিফল্ট নীতি deny ও সক্রিয় করুন
    community.general.ufw:
      state: enabled
      policy: deny

  - name: fail2ban ইনস্টল ও চালু করুন
    ansible.builtin.apt:
      name: fail2ban
      state: present
  - name: fail2ban চালু ও সক্রিয় করুন
    ansible.builtin.service:
      name: fail2ban
      state: started
      enabled: true

  - name: স্বয়ংক্রিয় নিরাপত্তা আপডেট ইনস্টল করুন
    ansible.builtin.apt:
      name: unattended-upgrades
      state: present
```

```
  handlers:
    - name: SSH পুনরায় চালু করুন
      ansible.builtin.service:
        name: ssh
        state: restarted
EOF
ansible-playbook ~/ansible-labs/playbooks/hardening.yml
```

> **WARNING:** এই প্লেবুক চালানোর আগে একটি আলাদা টার্মিনালে SSH সংযোগ খোলা রাখুন। SSH পুনরায় চালু করার সময় কোনো ভুল হলে ওই সংযোগ দিয়েই ফিরে ঢুকতে পারবেন। আর পাসওয়ার্ড লগইন বন্ধ করার আগে নিশ্চিত হোন যে কী-লগইন কাজ করছে — নইলে সার্ভার লক হয়ে যাবে।

ধাপ ২ — হার্ডেনিং যাচাই করুন।

```
ansible all -m shell -a 'grep -E "^(PasswordAuthentication|PermitRootLogin|
PermitEmptyPasswords)" /etc/ssh/sshd_config'
ansible all -m shell -a 'sudo ufw status verbose; sudo fail2ban-client status sshd'
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no nahid@203.0.113.11 'echo
SHOULD-FAIL' || echo "পাসওয়ার্ড লগইন বন্ধ — সঠিক"
```

১৭.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible all -m shell -a 'sudo ufw status verbose'
web | CHANGED | rc=0 >> Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing)
22/tcp  ALLOW IN  Anywhere
80/tcp  ALLOW IN  Anywhere
443/tcp ALLOW IN  Anywhere
```

১৭.৭ যাচাইকরণ (Verification)

তিনটি SSH সেটিংই `no` হবে; UFW `active` ও ডিফল্ট `deny` হবে; `fail2ban-client status sshd` জেলটি দেখাবে; পাসওয়ার্ড দিয়ে লগইনের চেষ্টা ব্যর্থ হবে; কী দিয়ে লগইন এখনো কাজ করবে।

১৭.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
SSH পুনরায় চালু হলে সংযোগ বন্ধ	স্বাভাবিক — নতুন সংযোগ করুন; কী-লগইন কাজ না করলে VM কনসোল থেকে `sshd_config` ঠিক করুন
UFW সক্রিয় করে নিজে ব্লক	কনসোল থেকে `sudo ufw allow 22/tcp` করুন; Lab ১৫-এর সতর্কতা দেখুন
`sshd -t` ভুল দিচ্ছে	`lineinfile`-এর `validate` ভুল কনফিগ বসতে দেয়নি — `sshd -t` চালিয়ে লাইনটি দেখুন

১৭.৯ ক্লিনআপ

হার্ডেনিং স্থায়ী — মুছবেন না। এটি Project ০১ ও ক্যাপস্টোনের বেসলাইন নিরাপত্তা। ল্যাব শেষে শুধু নিশ্চিত হোন যে কী-লগইন ও Nginx এখনো কাজ করছে।

১৭.১০ রিভিউ প্রশ্ন

(১) পাসওয়ার্ড লগইন বন্ধ করার আগে কী নিশ্চিত করতে হবে, এবং কেন? (২) UFW-এ ডিফল্ট `deny` নীতির অর্থ কী? (৩) `fail2ban` কীভাবে ব্রুট-ফোর্স ঠেকায়? (৪) `lineinfile`-এ `validate` কেন জরুরি? (৫) `unattended-upgrades` কী ঝুঁকি তৈরি করতে পারে?

Lab ১৮ • Node.js অ্যাপ ডেপ্লয়মেন্ট

Lab ১৮ — Node.js অ্যাপ্লিকেশন ডেপ্লয়মেন্ট

১৮.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) Node.js 20 LTS রানটাইম ইনস্টল করতে; (২) Express অ্যাপের কোড ডেপ্লয় করতে; (৩) `.env` পরিবেশ ফাইল টেমপ্লেট থেকে তৈরি করতে; (৪) systemd সার্ভিস দিয়ে অ্যাপ চালাতে; (৫) হেলথ চেক দিয়ে ডেপ্লয় যাচাই করতে।

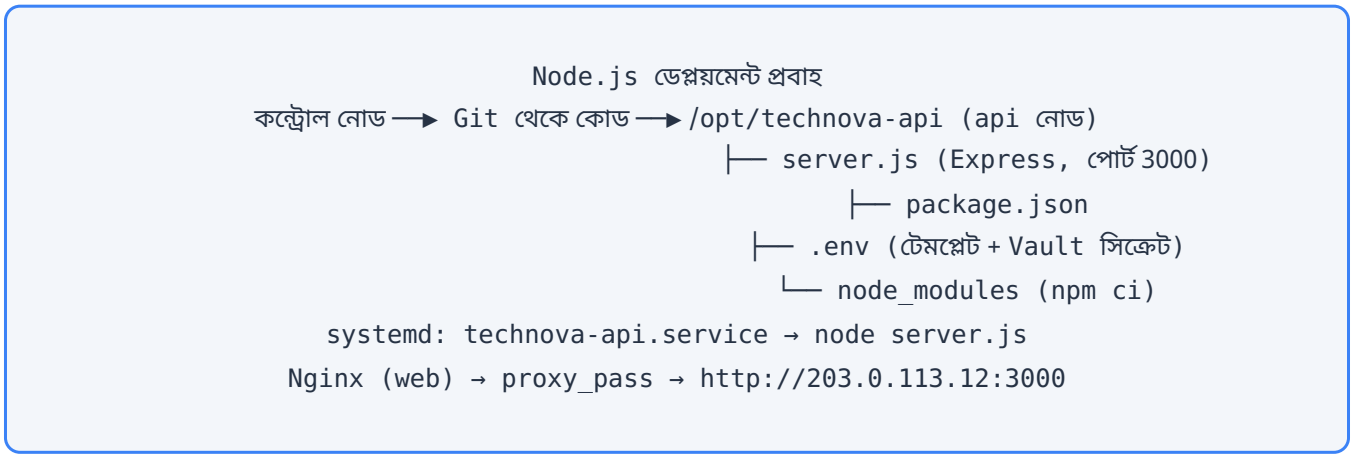
১৮.২ প্রয়োজনীয় টুলস

Lab ১–১৭ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি, api নোডে Node.js, Vault (Lab ১৬)।

১৮.৩ পূর্বশর্ত

Lab ৭-এর `technova-api.service` ইউনিট ফাইল আছে; Lab ৮-এর Nginx প্রক্সি আছে; Git-এর ধারণা আছে।

১৮.৪ আর্কিটেকচার



১৮.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Node.js 20 LTS ইনস্টল করুন।

```
cat > ~/ansible-labs/playbooks/nodejs-setup.yml << 'EOF'
---
- name: Node.js রানটাইম সেটআপ
  hosts: appservers
  become: true
  vars:
    nodejs_version: "20"
  tasks:
    - name: NodeSource GPG কী যোগ করুন
      ansible.builtin.apt_key:
        url: https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key
        state: present
        keyring: /etc/apt/keyrings/nodesource.gpg
```

```
    - name: NodeSource রিপোজিটরি যোগ করুন
      ansible.builtin.apt_repository:
        repo: "deb [signed-by=/etc/apt/keyrings/nodesource.gpg] https://deb.nodesource.com/
node_{{ nodejs_version }}.x nodistro main"
        state: present
        filename: nodesource

    - name: Node.js ইনস্টল করুন
      ansible.builtin.apt:
        name: nodejs
        state: present
        update_cache: true

    - name: Node.js সংস্করণ যাচাই করুন
      ansible.builtin.command: node --version
      register: node_version
      changed_when: false

    - name: সংস্করণ দেখান
      ansible.builtin.debug:
        msg: "Node.js সংস্করণ: {{ node_version.stdout }}"
EOF
ansible-playbook ~/ansible-labs/playbooks/nodejs-setup.yml
```

ধাপ ২ — নমুনা Express অ্যাপ তৈরি করুন (কন্ট্রোল নোডে)।

```
mkdir -p /tmp/technova-api
cat > /tmp/technova-api/package.json << 'EOF'
{
  "name": "technova-api",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": { "start": "node server.js" },
  "dependencies": { "express": "^4.19.0" }
}
EOF
cat > /tmp/technova-api/server.js << 'EOF'
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;
app.get("/health", (req, res) => res.json({ status: "ok", app: "technova-api" }));
app.get("/api/hello", (req, res) => res.json({ message: "TechNova API থেকে স্বাগতম" }));
app.listen(PORT, () => console.log(`TechNova API চলছে পোর্ট ${PORT}-এ`));
EOF
cat > ~/ansible-labs/templates/dotenv.j2 << 'EOF'
NODE_ENV={{ node_env | default('production') }}
PORT={{ api_port | default(3000) }}
DB_HOST={{ db_host | default('203.0.113.13') }}
DB_NAME={{ db_name | default('technova_app') }}
JWT_SECRET={{ api_jwt_secret }}
EOF
```

ধাপ ৩ — অ্যাপ ডেপ্লয় প্লেবুক লিখুন ও চালান।

```
cat > ~/ansible-labs/playbooks/deploy-nodejs.yml << 'EOF'
---
```

```
- name: TechNova API ডেপ্লয় করুন
  hosts: appservers
  become: true
  vars:
    app_dir: /opt/technova-api
    app_user: deploy
  tasks:
    - name: অ্যাপ ডিরেক্টরি তৈরি করুন
      ansible.builtin.file:
        path: "{{ app_dir }}"
        state: directory
        owner: "{{ app_user }}"
        group: "{{ app_user }}"
        mode: "0755"

    - name: অ্যাপের কোড কপি করুন
      ansible.builtin.copy:
        src: /tmp/technova-api/
        dest: "{{ app_dir }}/"
        owner: "{{ app_user }}"
        group: "{{ app_user }}"
        mode: "0644"
      notify: API পুনরায় চালু করুন

    - name: নির্ভরতা ইনস্টল করুন (npm ci)
      community.general.npm:
        path: "{{ app_dir }}"
        state: present
      become_user: "{{ app_user }}"

    - name: পরিবেশ ফাইল (.env) তৈরি করুন
      ansible.builtin.template:
        src: ../templates/dotenv.j2
        dest: "{{ app_dir }}/.env"
        owner: "{{ app_user }}"
        group: "{{ app_user }}"
        mode: "0600"
      no_log: true
      notify: API পুনরায় চালু করুন

    - name: systemd ইউনিট নিশ্চিত করুন
      ansible.builtin.copy:
        src: /tmp/technova-api.service
        dest: /etc/systemd/system/technova-api.service
        mode: "0644"
      notify: API পুনরায় চালু করুন

    - name: API সার্ভিস চালু ও সক্রিয় করুন
      ansible.builtin.systemd:
        name: technova-api
        state: started
        enabled: true
        daemon_reload: true

    - name: হেলথ চেক করুন
      ansible.builtin.uri:
        url: http://localhost:3000/health
        status_code: 200
```

```
      register: health
      retries: 5
      delay: 3
      until: health.status == 200

  handlers:
    - name: API পুনরায় চালু করুন
      ansible.builtin.systemd:
        name: technova-api
        state: restarted
EOF
ansible-playbook ~/ansible-labs/playbooks/deploy-nodejs.yml --vault-password-file
~/.vault_pass
```

> **TIP:** `.env` ফাইলের অনুমতি `0600` রাখুন এবং `no_log: true` দিন — এতে JWT সিক্রেট লগে যাবে না। প্রোডাকশনে কোড Git থেকে `git` মডিউল দিয়ে টানুন; `copy` শুধু ল্যাবের জন্য।

১৮.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ curl -s http://203.0.113.12:3000/health
{"status":"ok","app":"technova-api"}
$ curl -s http://203.0.113.11/api/hello
{"message":"TechNova API থেকে স্বাগতম"}
```

১৮.৭ যাচাইকরণ (Verification)

`curl http://203.0.113.12:3000/health` 200 দেবে; Nginx প্রক্সি দিয়ে `curl http://203.0.113.11/api/hello` বার্তা দেবে; `systemctl status technova-api` = `active`; `/opt/technova-api/.env` 600 অনুমতিতে থাকবে।

১৮.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`Cannot find module 'express'`	`npm ci` হয়নি — `ls /opt/technova-api/node_modules` দেখুন; নেটওয়ার্ক বা npm রেজিস্ট্রি যাচাই করুন
হেলথ চেক ব্যর্থ (connection refused)	সার্ভিস চালু হয়নি — `journalctl -u technova-api -n 50` দিয়ে লগ দেখুন; পোর্ট 3000 খোলা কি না দেখুন
`/api/` -এ 502	Nginx প্রক্সি ঠিক আছে কি না দেখুন (Lab ৮); api নোডে অ্যাপ চলছে কি না দেখুন
Vault ভেরিয়েবল undefined	`--vault-password-file` দিতে ভুলে গেছেন — আবার চালান

১৮.৯ ক্লিনআপ

অ্যাপ চালু রেখে দিন — Lab ২৩–২৪-এ এটিই ডেপ্লয় করা হবে। রোলব্যাকের অনুশীলন করতে চাইলে `systemctl stop technova-api` দিয়ে বন্ধ করে আবার প্লেবুক চালান।

১৮.১০ রিভিউ প্রশ্ন

(১) `npm ci` ও `npm install`-এর পার্থক্য কী? (২) `.env` ফাইলে 600 অনুমতি কেন জরুরি? (৩) হেলথ চেকে `retries` ও `until` কেন লাগে? (৪) systemd-এ `Restart=on-failure` কী সুবিধা দেয়? (৫) কোড ডেপ্লয়ে `copy`-এর বদলে `git` মডিউল কেন ভালো?

Lab ১৯ • ডেটাবেস কনফিগারেশন

Lab ১৯ — MongoDB বা PostgreSQL কনফিগারেশন

১৯.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) PostgreSQL ইনস্টল ও প্রাথমিক কনফিগারেশন করতে; (২) ডেটাবেস, ব্যবহারকারী ও অধিকার তৈরি করতে; (৩) MongoDB-এর ক্ষেত্রে একই কাজের ধারণা পেতে; (৪) ডেটাবেস সার্ভিস যাচাই করতে।

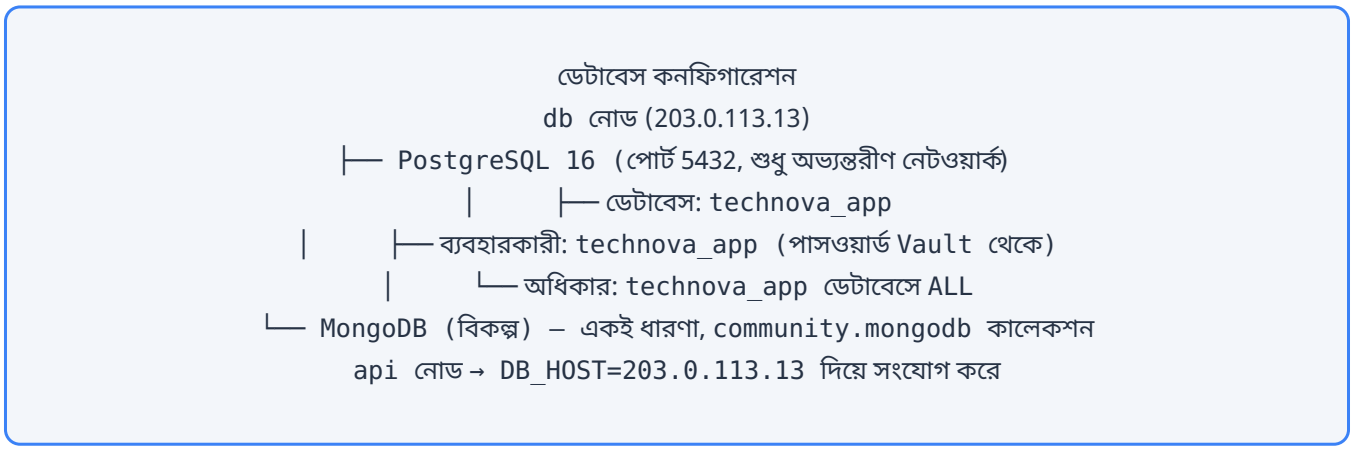
১৯.২ প্রয়োজনীয় টুলস

Lab ১–১৮ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি, `community.postgresql` কালেকশন (Lab ১৫), Vault।

১৯.৩ পূর্বশর্ত

db নোডে SSH ও sudo কাজ করছে; Vault সিক্রেট (`db_app_password`) তৈরি আছে।

১৯.৪ আর্কিটেকচার



১৯.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — PostgreSQL ইনস্টল করুন।

```
cat > ~/ansible-labs/playbooks/postgresql-setup.yml << 'EOF'
---
- name: PostgreSQL সেটআপ
  hosts: dbservers
  become: true
  vars:
    pg_version: "16"
  tasks:
    - name: PostgreSQL ও Python ড্রাইভার ইনস্টল করুন
      ansible.builtin.apt:
        name:
          - "postgresql-{{ pg_version }}"
          - python3-psycopg2
        state: present
```

```
        update_cache: true

    - name: PostgreSQL চালু ও সক্রিয় করুন
      ansible.builtin.service:
        name: postgresql
        state: started
        enabled: true

    - name: অ্যাপ ডেটাবেস তৈরি করুন
      community.postgresql.postgresql_db:
        name: "{{ db_name }}"
        state: present
      become_user: postgres

    - name: অ্যাপ ব্যবহারকারী তৈরি করুন
      community.postgresql.postgresql_user:
        name: "{{ db_name }}"
        password: "{{ db_app_password }}"
        db: "{{ db_name }}"
        priv: "ALL"
        state: present
      become_user: postgres
      no_log: true

    - name: শুধু অভ্যন্তরীণ নেটওয়ার্ক থেকে সংযোগের অনুমতি দিন
      ansible.builtin.lineinfile:
        path: /etc/postgresql/{{ pg_version }}/main/pg_hba.conf
        line: "host {{ db_name }} {{ db_name }} 203.0.113.0/24 scram-sha-256"
        state: present
      notify: PostgreSQL পুনরায় চালু করুন

  handlers:
    - name: PostgreSQL পুনরায় চালু করুন
      ansible.builtin.service:
        name: postgresql
        state: restarted
EOF
ansible-playbook ~/ansible-labs/playbooks/postgresql-setup.yml --vault-password-file
~/.vault_pass
```

ধাপ ২ — সংযোগ পরীক্ষা করুন।

```
ansible db -m shell -a 'sudo -u postgres psql -lqt | cut -d \| -f 1 | grep -w technova_app' --
become
ansible api -m shell -a 'PGPASSWORD="CHANGE-ME-DB-APP-PASSWORD" psql -h 203.0.113.13 -U
technova_app -d technova_app -c "SELECT version();"' --become
# উপরের কমান্ডে আসল পাসওয়ার্ডের বদলে Vault মান ব্যবহার করুন; লগে পাসওয়ার্ড যাবে না তা নিশ্চিত করুন
```

> **CONCEPT:** MongoDB ব্যবহার করতে চাইলে ধারণা একই: `community.mongodb` কালেকশনের `mongodb_user` ও `mongodb_db` মডিউল দিয়ে ব্যবহারকারী ও ডেটাবেস তৈরি করুন, `mongod.conf`-এ `bindIp` শুধু অভ্যন্তরীণ নেটওয়ার্কে সীমিত রাখুন, এবং প্রমাণীকরণ (`authorization: enabled`) চালু করুন। MERN অ্যাপে MongoDB-ই সাধারণ পছন্দ; এই ল্যাবে PostgreSQL দেখানো হয়েছে কারণ এর Ansible মডিউল বেশি পরিপক্ক।

ধাপ ৩ — ব্যাকআপের প্রাথমিক ব্যবস্থা করুন।

```
ansible db -m shell -a 'sudo -u postgres pg_dump technova_app | head -5' --become
# নিয়মিত ব্যাকআপের জন্য cron-এ pg_dump যোগ করুন (Project ০২-এ বিস্তারিত)
```

১৯.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ sudo -u postgres psql -lqt | grep technova_app
 technova_app | postgres | UTF8 | ...
$ psql -h 203.0.113.13 -U technova_app -d technova_app -c "SELECT version();"
 PostgreSQL 16.x on x86_64-pc-linux-gnu ...
```

১৯.৭ যাচাইকরণ (Verification)

`technova_app` ডেটাবেস তালিকায় থাকবে; api নোড থেকে পাসওয়ার্ড দিয়ে সংযোগ সফল হবে; বাইরের নেটওয়ার্ক থেকে (উদাহরণ: কন্ট্রোল নোডের বাইরের IP) সংযোগ ব্যর্থ হওয়া উচিত — `pg_hba.conf` শুধু 203.0.113.0/24 অনুমতি দেয়।

১৯.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`Peer authentication failed`	`become_user: postgres` ছাড়া `psql` চালিয়েছেন — মডিউলে `become_user` দিন
`password authentication failed`	Vault থেকে পাসওয়ার্ড ঠিক আসছে কি না দেখুন; `no_log` থাকায় লগে মান দেখা যাবে না — Vault ফাইলটি যাচাই করুন
api নোড থেকে সংযোগ ব্যর্থ	`pg_hba.conf` -এর লাইন ও UFW-এ পোর্ট 5432 খোলা কি না দেখুন; `listen_addresses` যাচাই করুন

১৯.৯ ক্লিনআপ

ডেটাবেস ও ব্যবহারকারী রেখে দিন — Lab ২৪ ও Project ০২-এ লাগবে। পরীক্ষার ডাম্প ফাইল থাকলে মুছে ফেলুন।

১৯.১০ রিভিউ প্রশ্ন

(১) `pg_hba.conf` কী নিয়ন্ত্রণ করে? (২) ডেটাবেস পাসওয়ার্ড Vault ছাড়া প্লেবুকে লিখলে কী ঝুঁকি? (৩) `become_user: postgres` কেন লাগে? (৪) MongoDB-তে প্রমাণীকরণ চালু না করলে কী ঝুঁকি? (৫) ডেটাবেস সার্ভারে পোর্ট 5432 ইন্টারনেটে খোলা উচিত কি না, এবং কেন?

Lab ২০ • ক্লাউড VM কনফিগারেশন

Lab ২০ — ক্লাউড VM কনফিগারেশন অটোমেশন

> **WARNING — খরচ সতর্কতা:** এই ল্যাবটি ক্লাউডে VM তৈরি করলে খরচ হতে পারে। DigitalOcean, Hetzner বা AWS-এ ছোট VM (1 vCPU, 1 GB RAM) ঘণ্টায় কয়েক সেন্ট থেকে কয়েক ডলার/মাস খরচ করে। কাজ শেষে VM মুছে ফেলুন, নইলে বিল চলতে থাকবে। খরচ এড়াতে চাইলে এই ল্যাবটি লোকাল VM (Multipass/KVM)-এ অনুশীলন করুন — ধারণা একই, শুধু প্রভিশনিং ধাপটি বদলে যায়।

২০.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) ক্লাউড VM-এর জন্য ডায়নামিক ইনভেন্টরি ধারণা বুঝতে; (২) নতুন ক্লাউড VM-কে Ansible দিয়ে বেসলাইন কনফিগার করতে; (৩) ক্লাউড API কী নিরাপদে Vault-এ রাখতে; (৪) VM তৈরি ও মুছে ফেলার জীবনচক্র পরিচালনা করতে।

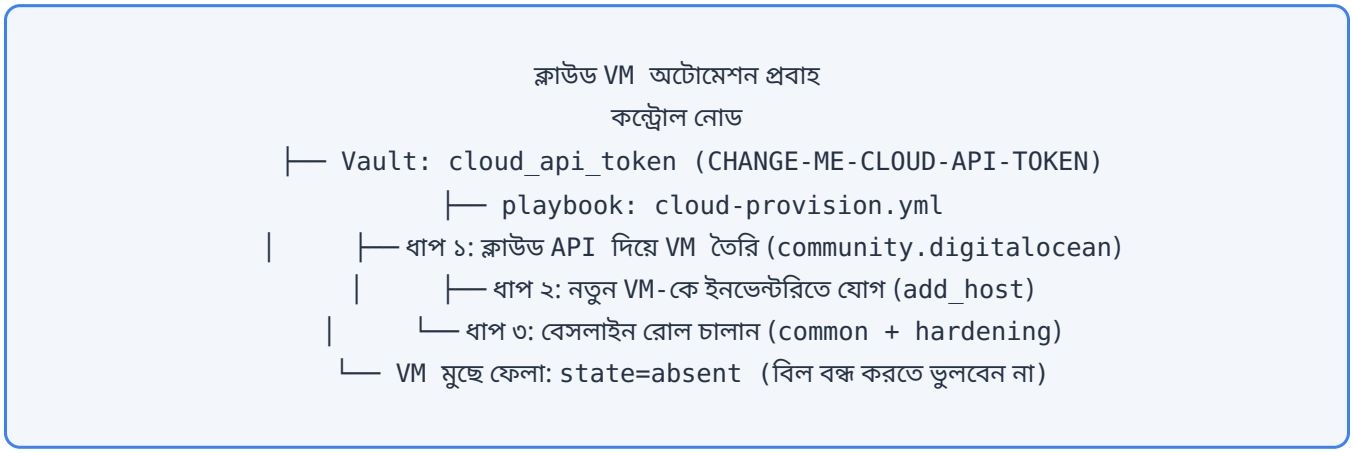
২০.২ প্রয়োজনীয় টুলস

Lab ১–১৯ সম্পন্ন কন্ট্রোল নোড; ক্লাউড অ্যাকাউন্ট (ঐচ্ছিক — লোকাল VM-এও অনুশীলন করা যায়); ক্লাউড API টোকেন (Vault-এ)।

২০.৩ পূর্বশর্ত

বেসলাইন প্লেবুক (`site.yml` , Lab ১৪) কাজ করছে; Vault-এর ব্যবহার জানা (Lab ১৬)।

২০.৪ আর্কিটেকচার



২০.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — ক্লাউড API টোকেন Vault-এ রাখুন।

```
ansible-vault encrypt_string 'CHANGE-ME-CLOUD-API-TOKEN' --name 'cloud_api_token' --vault-
password-file ~/.vault_pass
# আউটপুটটি group_vars/all.yml-এ যোগ করুন (Lab ১৬-এর ধাপ ৩ দেখুন)
```

ধাপ ২ — ক্লাউড VM প্রভিশনিং প্লেবুক লিখুন (DigitalOcean উদাহরণ)।

```
cat > ~/ansible-labs/playbooks/cloud-provision.yml << 'EOF'
---
- name: ক্লাউড VM তৈরি করুন (খরচ হতে পারে — সতর্কতা পড়ুন)
  hosts: localhost
  gather_facts: false
  vars:
    cloud_region: "blr1"
    cloud_size: "s-1vcpu-1gb"   # সবচেয়ে ছোট আকার — খরচ কম
    cloud_image: "ubuntu-24-04-x64"
  tasks:
    - name: DigitalOcean ড্রপলেট তৈরি করুন
      community.digitalocean.digital_ocean_droplet:
        state: present
        name: technova-web-cloud
        oauth_token: "{{ cloud_api_token }}"
        size: "{{ cloud_size }}"
        region: "{{ cloud_region }}"
        image: "{{ cloud_image }}"
        ssh_keys: ["CHANGE-ME-SSH-KEY-ID"]
        tags: ["technova", "lab"]
      register: droplet
      no_log: true

    - name: নতুন VM-কে ইনভেন্টরিতে যোগ করুন
      ansible.builtin.add_host:
        name: cloud-web
        ansible_host: "{{ droplet.data.droplet.networks.v4[0].ip_address }}"
        ansible_user: root
        groups: cloud_webservers

    - name: SSH প্রস্তুত হওয়া পর্যন্ত অপেক্ষা করুন
      ansible.builtin.wait_for:
        host: "{{ droplet.data.droplet.networks.v4[0].ip_address }}"
        port: 22
        delay: 10
        timeout: 300

- name: নতুন ক্লাউড VM-এ বেসলাইন কনফিগ করুন
  hosts: cloud_webservers
  become: true
  roles:
    - common
EOF
echo "সতর্কতা: এই প্লেবুক চালালে ক্লাউডে বিল হবে। চালানোর আগে size ও region যাচাই করুন।"
```

ধাপ ৩ — লোকাল VM-এ একই ধারণা অনুশীলন করুন (বিনামূল্যে বিকল্প)।

```
# Multipass দিয়ে লোকাল VM তৈরি করুন:
multipass launch 24.04 --name technova-lab --cpus 1 --memory 1G --disk 10G
multipass list
```

```
# নতুন VM-এর IP ইনভেন্টরিতে যোগ করে Lab ১৪-এর site.yml চালান:
ansible-playbook ~/ansible-labs/playbooks/site.yml --limit webservers
```

ধাপ ৪ — ক্লাউড VM মুছে ফেলুন (বিল বন্ধ করতে)।

```
cat > ~/ansible-labs/playbooks/cloud-destroy.yml << 'EOF'
---
- name: ক্লাউড VM মুছে ফেলুন
  hosts: localhost
  gather_facts: false
  tasks:
    - name: ড্রপলেট মুছে ফেলুন
      community.digitalocean.digital_ocean_droplet:
        state: absent
        name: technova-web-cloud
        oauth_token: "{{ cloud_api_token }}"
      no_log: true
EOF
echo "কাজ শেষে অবশ্যই cloud-destroy.yml চালান, অথবা ক্লাউড কনসোল থেকে VM মুছে ফেলুন।"
```

> **CONCEPT:** ক্লাউডে VM-এর IP প্রতিবার বদলায় — তাই স্ট্যাটিক ইনভেন্টরির বদলে ডায়নামিক ইনভেন্টরি (ক্লাউড API থেকে হোস্ট তালিকা) বা `add_host` ব্যবহার করা হয়। প্রভিশনিং (VM তৈরি) ও কনফিগারেশন (Ansible দিয়ে সাজানো) দুটি আলাদা ধাপ — Terraform দিয়ে প্রথমটি, Ansible দিয়ে দ্বিতীয়টি করাই প্রচলিত প্যাটার্ন।

২০.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ multipass list
Name           State  IPv4           Image
technova-lab   Running  10.20.30.40  Ubuntu 24.04 LTS
$ ansible-playbook playbooks/site.yml --limit webservers
PLAY RECAP: web : ok=8 changed=0 unreachable=0 failed=0
```

২০.৭ যাচাইকরণ (Verification)

নতুন VM-এ `common` রোলের প্যাকেজ ও টাইমজোন সঠিক হবে; ক্লাউড কনসোলে VM-এর অবস্থা ও বিল দেখা যাবে; কাজ শেষে VM মুছে ফেলার পর কনসোলে আর কোনো চলমান VM থাকবে না।

২০.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
API টোকেন ভুল (`401 Unauthorized`)	Vault থেকে টোকেন ঠিক আসছে কি না দেখুন; টোকেনের মেয়াদ ও অনুমতি যাচাই করুন
SSH সংযোগে দেরি	VM বুট হতে সময় লাগে — `wait_for`-এর `timeout` বাড়ান
বিল চলছে, VM খুঁজে পাচ্ছেন না	

	ক্লাউড কনসোলে সব রিজিয়নে দেখুন; ভলিউম ও স্ন্যাপশটও বিল করে — সেগুলোও মুছুন

২০.৯ ক্লিনআপ

ক্লাউড VM ও লোকাল Multipass VM — দুটিই মুছে ফেলুন (`multipass delete --purge technova-lab`)। API টোকেন Vault-এ রেখে দিন, তবে ব্যবহার শেষে ক্লাউড কনসোল থেকে টোকেনটি বাতিল (revoke) করুন।

২০.১০ রিভিউ প্রশ্ন

(১) প্রভিশনিং ও কনফিগারেশনের পার্থক্য কী? (২) ক্লাউড VM-এ স্ট্যাটিক ইনভেন্টরি কেন অসুবিধাজনক? (৩) `add_host` কী করে? (৪) API টোকেন Vault ছাড়া রাখলে কী ঝুঁকি? (৫) কাজ শেষে VM না মুছলে কী হবে?

Lab ২১ • Docker হোস্ট

Lab ২১ — Docker হোস্ট কনফিগারেশন

২১.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) Docker Engine ইনস্টল ও কনফিগার করতে; (২) ব্যবহারকারীকে `docker` গ্রুপে যোগ করতে; (৩) `community.docker` মডিউল দিয়ে কন্টেইনার চালাতে; (৪) Docker ডেমন কনফিগ (`daemon.json`) পরিচালনা করতে।

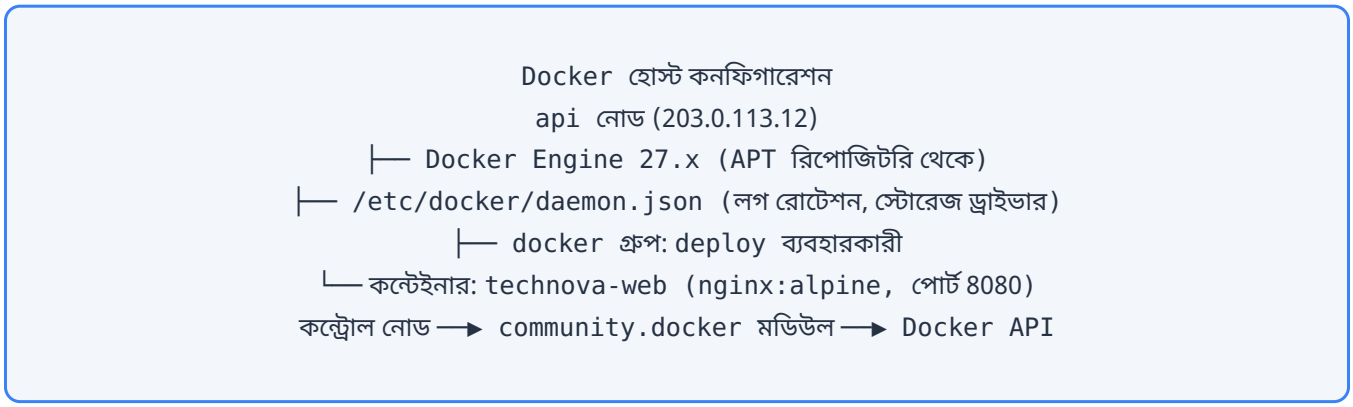
২১.২ প্রয়োজনীয় টুলস

Lab ১–২০ সম্পন্ন কন্ট্রোল নোড, `community.docker` কালেকশন (Lab ১৫), api নোডে Docker হোস্ট হিসেবে ব্যবহার।

২১.৩ পূর্বশর্ত

api নোডে ইন্টারনেট সংযোগ; Docker-এর প্রাথমিক ধারণা (Volume 06)।

২১.৪ আর্কিটেকচার



২১.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Docker Engine ইনস্টল করুন।

```
cat > ~/ansible-labs/playbooks/docker-host.yml << 'EOF'
---
- name: Docker হোস্ট কনফিগার করুন
  hosts: appservers
  become: true
  tasks:
    - name: প্রয়োজনীয় প্যাকেজ ইনস্টল করুন
      ansible.builtin.apt:
        name:
          - ca-certificates
          - curl
          - gnupg
        state: present
        update_cache: true
```

```
    - name: Docker GPG কী যোগ করুন
      ansible.builtin.apt_key:
        url: https://download.docker.com/linux/ubuntu/gpg
        state: present
        keyring: /etc/apt/keyrings/docker.gpg

    - name: Docker রিপোজিটরি যোগ করুন
      ansible.builtin.apt_repository:
        repo: "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] https://
download.docker.com/linux/ubuntu {{ ansible_facts['distribution_release'] }} stable"
        state: present
        filename: docker

    - name: Docker Engine ইনস্টল করুন
      ansible.builtin.apt:
        name:
          - docker-ce
          - docker-ce-cli
          - containerd.io
          - docker-buildx-plugin
          - docker-compose-plugin
        state: present
        update_cache: true

    - name: Docker সার্ভিস চালু ও সক্রিয় করুন
      ansible.builtin.service:
        name: docker
        state: started
        enabled: true

    - name: deploy ব্যবহারকারীকে docker গ্রুপে যোগ করুন
      ansible.builtin.user:
        name: deploy
        groups: docker
        append: true

    - name: Docker ডেমন কনফিগ বসান (লগ রোটেশন)
      ansible.builtin.copy:
        content: |
          {
            "log-driver": "json-file",
            "log-opts": { "max-size": "10m", "max-file": "3" },
            "storage-driver": "overlay2"
          }
        dest: /etc/docker/daemon.json
        mode: "0644"
      notify: Docker পুনরায় চালু করুন

  handlers:
    - name: Docker পুনরায় চালু করুন
      ansible.builtin.service:
        name: docker
        state: restarted
EOF
ansible-playbook ~/ansible-labs/playbooks/docker-host.yml
```

ধাপ ২ — কন্টেইনার চালান (`community.docker` মডিউল)।

```
cat > ~/ansible-labs/playbooks/docker-container.yml << 'EOF'
---
- name: পরীক্ষামূলক কন্টেইনার চালান
  hosts: appservers
  become: true
  tasks:
    - name: Nginx কন্টেইনার চালু করুন
      community.docker.docker_container:
        name: technova-web
        image: nginx:alpine
        state: started
        restart_policy: unless-stopped
        published_ports:
          - "8080:80"
        volumes:
          - /var/www/technova:/usr/share/nginx/html:ro
EOF
ansible-playbook ~/ansible-labs/playbooks/docker-container.yml
curl -s http://203.0.113.12:8080/ | head -3
```

TIP: `docker` গ্রুপে যোগ করার পর ব্যবহারকারীকে আবার লগইন করতে হয় — গ্রুপ সদস্যপদ নতুন সেশনে কার্যকর হয়। আর মনে রাখবেন: `docker` গ্রুপের সদস্য কার্যত রুটের সমান ক্ষমতা পায় (কন্টেইনারে হোস্টের রুট ফাইলসিস্টেম মাউন্ট করা যায়) — তাই শুধু বিশ্বস্ত ব্যবহারকারীকে এই গ্রুপে রাখুন।

২১.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible api -m shell -a 'docker ps --format "{{.Names}} {{.Status}} {{.Ports}}"' --become
api | CHANGED | rc=0 >> technova-web Up 2 minutes 0.0.0.0:8080->80/tcp
$ curl -s http://203.0.113.12:8080/
<h1>TechNova — স্বাগতম</h1>
```

২১.৭ যাচাইকরণ (Verification)

`docker --version` ও `docker ps` api নোডে কাজ করবে; `technova-web` কন্টেইনার `Up` অবস্থায় থাকবে; `curl http://203.0.113.12:8080/` পাতা দেবে; `daemon.json`-এ লগ রোটেশন সেটিং থাকবে।

২১.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`permission denied while trying to connect to Docker daemon`	ব্যবহারকারী `docker` গ্রুপে নেই বা আবার লগইন করেননি — গ্রুপ যাচাই করে নতুন সেশন খুলুন
`port is already allocated`	পোর্ট 8080 অন্য কিছু ব্যবহার করছে — `ss -tlnp | grep 8080` দিয়ে দেখুন
Docker ইনস্টলের পর সার্ভিস চলছে না	

	`journalctl -u docker -n 50` দিয়ে লগ দেখুন; `daemon.json`-এর JSON সিনট্যাক্স যাচাই করুন

২১.৯ ক্লিনআপ

```
ansible api -m shell -a 'docker stop technova-web && docker rm technova-web' --become
# Docker Engine রেখে দিন — Lab ২২-এ Kubernetes প্রস্তুতিতে containerd লাগবে
```

২১.১০ রিভিউ প্রশ্ন

(১) `docker` গ্রুপের সদস্যপদ কেন রুটের সমান ক্ষমতা দেয়? (২) `daemon.json`-এ লগ রোটেশন কেন জরুরি? (৩) `community.docker.docker_container` মডিউল `shell`-এ `docker run` চালানোর চেয়ে ভালো কেন? (৪) `restart_policy: unless-stopped` কী করে? (৫) Docker রিপোজিটরি যোগ করতে GPG কী কেন লাগে?

Lab ২২ • Kubernetes প্রস্তুতি

Lab ২২ — Kubernetes পরিবেশ প্রস্তুতি

২২.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) Kubernetes নোডের পূর্বশর্ত (swap বন্ধ, কার্নেল মডিউল, sysctl) কনফিগার করতে; (২) containerd রানটাইম ইনস্টল করতে; (৩) kubeadm, kubelet ও kubectl ইনস্টল করতে; (৪) একক-নোড ক্লাস্টার প্রস্তুতি যাচাই করতে।

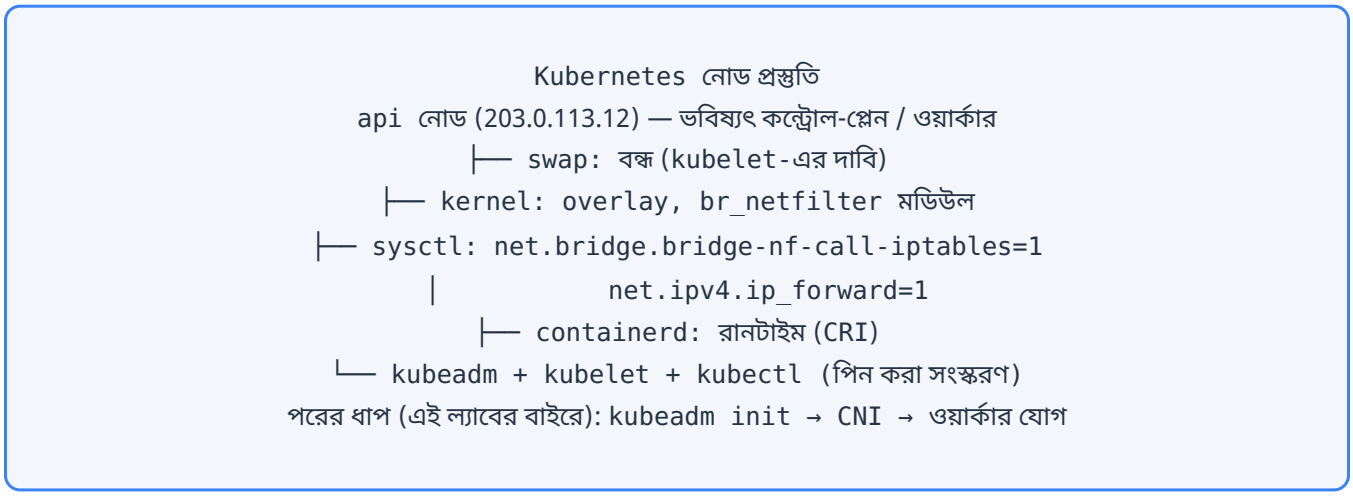
২২.২ প্রয়োজনীয় টুলস

Lab ১–২১ সম্পন্ন কন্ট্রোল নোড, ইনভেন্টরি; api নোডে অন্তত 2 vCPU ও 2 GB RAM (কম হলে kubelet সতর্কতা দেবে)।

২২.৩ পূর্বশর্ত

Docker/containerd-এর ধারণা (Lab ২১); Linux কার্নেল মডিউলের প্রাথমিক ধারণা।

২২.৪ আর্কিটেকচার



২২.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — Swap বন্ধ ও কার্নেল প্রস্তুত করুন।

```
cat > ~/ansible-labs/playbooks/k8s-prepare.yml << 'EOF'
---
- name: Kubernetes নোড প্রস্তুত করুন
  hosts: appservers
  become: true
  vars:
    k8s_version: "1.31"
  tasks:
    - name: Swap বন্ধ করুন (বর্তমান সেশন)
      ansible.builtin.command: swapoff -a
      changed_when: false
```

```
    - name: Swap স্থায়ীভাবে বন্ধ করুন (fstab থেকে সরান)
      ansible.builtin.replace:
        path: /etc/fstab
        regexp: '^([^#].*\sswap\s.*)$'
        replace: '# \1'

    - name: প্রয়োজনীয় কার্নেল মডিউল লোড করুন
      community.general.modprobe:
        name: "{{ item }}"
        state: present
        persistent: present
      loop:
        - overlay
        - br_netfilter

    - name: Kubernetes sysctl সেটিংস বসান
      ansible.posix.sysctl:
        name: "{{ item.key }}"
        value: "{{ item.value }}"
        state: present
        reload: true
      loop:
        - { key: "net.bridge.bridge-nf-call-iptables", value: "1" }
        - { key: "net.bridge.bridge-nf-call-ip6tables", value: "1" }
        - { key: "net.ipv4.ip_forward", value: "1" }

    - name: containerd ইনস্টল করুন
      ansible.builtin.apt:
        name: containerd
        state: present
        update_cache: true

    - name: containerd কনফিগ তৈরি করুন
      ansible.builtin.shell: containerd config default > /etc/containerd/config.toml
      args:
        creates: /etc/containerd/config.toml

    - name: containerd-এ SystemdCgroup চালু করুন
      ansible.builtin.replace:
        path: /etc/containerd/config.toml
        regexp: 'SystemdCgroup = false'
        replace: 'SystemdCgroup = true'
      notify: containerd পুনরায় চালু করুন

    - name: Kubernetes APT কী ও রিপোজিটরি যোগ করুন
      ansible.builtin.apt_key:
        url: "https://pkgs.k8s.io/core:/stable:/v{{ k8s_version }}/deb/Release.key"
        state: present
        keyring: /etc/apt/keyrings/kubernetes-apt-keyring.gpg
    - name: Kubernetes রিপোজিটরি যোগ করুন
      ansible.builtin.apt_repository:
        repo: "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://
pkgs.k8s.io/core:/stable:/v{{ k8s_version }}/deb/ /"
        state: present
        filename: kubernetes

    - name: kubeadm, kubelet ও kubectl ইনস্টল করুন
```

```
      ansible.builtin.apt:
        name:
          - kubeadm
          - kubelet
          - kubectl
        state: present
        update_cache: true

    - name: Kubernetes প্যাকেজ হোল্ড করুন (অনিচ্ছাকৃত আপগ্রেড ঠেকাতে)
      ansible.builtin.dpkg_selections:
        name: "{{ item }}"
        selection: hold
      loop:
        - kubeadm
        - kubelet
        - kubectl

    - name: kubelet চালু ও সক্রিয় করুন
      ansible.builtin.service:
        name: kubelet
        state: started
        enabled: true

  handlers:
    - name: containerd পুনরায় চালু করুন
      ansible.builtin.service:
        name: containerd
        state: restarted
EOF
ansible-playbook ~/ansible-labs/playbooks/k8s-prepare.yml
```

ধাপ ২ — প্রস্তুতি যাচাই করুন।

```
ansible api -m shell -a 'swapoff --show; echo "swap খালি = সঠিক"; lsmod | grep -E "overlay|
br_netfilter"; sysctl net.ipv4.ip_forward; kubeadm version; kubectl version --client' --become
```

> **WARNING:** এই ল্যাব শুধু প্রস্তুতি — `kubeadm init` এখানে চালানো হয়নি। একক ল্যাব VM-এ পূর্ণ ক্লাস্টার চালালে RAM/CPU চাপে পড়ে; প্রোডাকশন ক্লাস্টারের জন্য আলাদা কন্ট্রোল-প্লেন ও ওয়ার্কার নোড লাগে। `kubeadm init` চালাতে চাইলে প্রথমে VM-এর স্ন্যাপশট নিন এবং CNI (যেমন Flannel বা Calico) ইনস্টলের ধাপটি আগে পড়ে নিন।

২২.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"31", ...}
$ kubectl version --client
Client Version: v1.31.0
$ sysctl net.ipv4.ip_forward
net.ipv4.ip_forward = 1
```

২২.৭ যাচাইকরণ (Verification)

`swapoff --show` খালি হবে; `overlay` ও `br_netfilter` মডিউল লোড থাকবে; `net.ipv4.ip_forward = 1` হবে; `kubeadm version` ও `kubectl version --client` 1.31 দেখাবে; `dpkg --get-selections | grep kube`-এ তিনটি প্যাকেজ `hold` থাকবে।

২২.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`kubelet` বারবার রিস্টার্ট হচ্ছে	ক্লাস্টার এখনো `init` হয়নি বলে স্বাভাবিক — `kubeadm init`-এর আগে kubelet স্থির থাকে না
`SystemdCgroup` সেটিং মিলছে না	`containerd config default` আবার তৈরি করে `replace` চালান; containerd পুনরায় চালু করুন
APT কী/রিপোজিটরি ত্রুটি	Kubernetes সংস্করণ (`k8s_version`) ও URL যাচাই করুন; pkgs.k8s.io পৌঁছানো যায় কি না দেখুন

২২.৯ ক্লিনআপ

কিছু মুছবেন না — প্রস্তুতি স্থায়ী। `kubeadm init` চালিয়ে থাকলে `kubeadm reset` দিয়ে ক্লাস্টার মুছে ফেলুন, তারপর আবার প্রস্তুতি যাচাই করুন।

২২.১০ রিভিউ প্রশ্ন

(১) kubelet-এর জন্য swap বন্ধ করা জরুরি কেন? (২) `SystemdCgroup = true` কী নিশ্চিত করে? (৩) `net.ipv4.ip_forward=1` কেন লাগে? (৪) kubeadm/kubelet/kubectl `hold` করা হয় কেন? (৫) `kubeadm init`-এর পর CNI কেন লাগে?

Lab ২৩ • Ansible CI/CD

Lab ২৩ — Ansible CI/CD এক্সিকিউশন

২৩.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) Ansible প্রজেক্টকে Git রিপোজিটরিতে সাজাতে; (২) `ansible-lint` দিয়ে প্লেবুক যাচাই করতে; (৩) GitHub Actions-এ প্লেবুকের সিনট্যাক্স ও লিন্ট চেক চালাতে; (৪) `--check` মোডে নিরাপদ CI ধাপ সাজাতে।

২৩.২ প্রয়োজনীয় টুলস

Lab ১–২২ সম্পন্ন কন্ট্রোল নোড, Git, GitHub অ্যাকাউন্ট (ঐচ্ছিক — লোকালেও অনুশীলন করা যায়), `ansible-lint`।

২৩.৩ পূর্বশর্ত

প্লেবুক ও রোলের অভিজ্ঞতা; Git-এ কমিট করার ধারণা।

২৩.৪ আর্কিটেকচার



২৩.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — প্রজেক্টকে Git রিপোজিটরিতে সাজান।

```
cd ~/ansible-labs
cat > .gitignore << 'EOF'
.vault_pass
*.retry
__pycache__/
.vault*
EOF
cat > README.md << 'EOF'
# TechNova Ansible Automation
```

```
কন্ট্রোল নোড: Ubuntu 24.04, Ansible Core 2.17+
চালানোর নিয়ম: `ansible-playbook playbooks/site.yml --vault-password-file ~/.vault_pass`
EOF
git init
git add .
git commit -m "TechNova Ansible automation — baseline, roles, vault"
```

ধাপ ২ — `ansible-lint` ইনস্টল ও চালান।

```
pipx install ansible-lint
ansible-lint playbooks/site.yml
ansible-lint playbooks/ --format plain | head -40
```

ধাপ ৩ — GitHub Actions ওয়ার্কফ্লো লিখুন।

```
mkdir -p .github/workflows
cat > .github/workflows/ansible-ci.yml << 'EOF'
name: Ansible CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - name: Python ও Ansible ইনস্টল করুন
        run: |
          sudo apt-get update
          sudo apt-get install -y python3-pip
          pip install --user ansible-core ansible-lint
      - name: Lint চালান
        run: |
          ~/.local/bin/ansible-lint playbooks/ roles/
          ~/.local/bin/ansible-playbook playbooks/site.yml --syntax-check

  dry-run:
    runs-on: ubuntu-24.04
    needs: lint
    steps:
      - uses: actions/checkout@v4
      - name: Python ও Ansible ইনস্টল করুন
        run: |
          sudo apt-get update
          sudo apt-get install -y python3-pip
          pip install --user ansible-core
      - name: Check মোডে প্লেবুক চালান (কিছু বদলাবে না)
        run: |
          ~/.local/bin/ansible-playbook playbooks/site.yml \
            --check --diff \
            -i inventory/hosts.ini \
            --vault-password-file /dev/null || true
        env:
```

```
          ANSIBLE_HOST_KEY_CHECKING: "False"
EOF
git add .github/workflows/ansible-ci.yml
git commit -m "CI: ansible-lint ও dry-run ওয়ার্কফ্লো যোগ করুন"
```

> **WARNING:** CI-তে কখনো আসল Vault পাসওয়ার্ড বা ক্লাউড API টোকেন সরাসরি লিখবেন না — GitHub Secrets ব্যবহার করুন। আর `--check` মোডে সব মডিউল সঠিক ফল দেয় না (যেমন `command` / `shell`); তাই dry-run শুধু প্রাথমিক নিরাপত্তা, চূড়ান্ত যাচাই নয়। প্রোডাকশন রান সবসময় ম্যানুয়াল অনুমোদনের পর হওয়া উচিত।

ধাপ ৪ — লোকালি CI ধাপগুলো অনুকরণ করুন।

```
ansible-lint playbooks/site.yml roles/common/ roles/nginx/
ansible-playbook playbooks/site.yml --syntax-check
ansible-playbook playbooks/site.yml --check --diff --vault-password-file ~/.vault_pass | tail
-20
```

২৩.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ ansible-lint playbooks/site.yml
Passed: 0 failure(s), 0 warning(s) on 1 files.
$ ansible-playbook playbooks/site.yml --check --diff
PLAY RECAP: web ok=8 changed=0 unreachable=0 failed=0
```

২৩.৭ যাচাইকরণ (Verification)

`ansible-lint` 0 failure দেবে (সতর্কতা থাকলে ঠিক করুন); `--syntax-check` সফল হবে; `--check` রানে `changed=0` হবে (পরিবেশ ইতিমধ্যে কনফিগার করা); GitHub-এ পুশ করলে Actions ট্যাবে দুটি জব সবুজ হবে।

২৩.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান
`ansible-lint` নামকরণের অভিযোগ করছে	টাস্কের `name` অনুপস্থিত বা FQCN ব্যবহার হয়নি — প্রতিটি টাস্কে নাম ও `ansible.builtin.` উপসর্গ দিন
CI-তে `ansible-playbook: command not found`	`pip install --user`-এর বাইনারি `~/.local/bin`-এ থাকে — PATH-এ যোগ করুন বা পুরো পথ দিন
dry-run-এ Vault ত্রুটি	CI-তে Vault ফাইল নেই — dry-run জবে `--ask-vault-pass` নয়, GitHub Secrets থেকে পাসওয়ার্ড ফাইল তৈরি করুন

২৩.৯ ক্লিনআপ

Git রিপোজিটরি ও ওয়ার্কফ্লো রেখে দিন — Project ০৩ ও ক্যাপস্টোনে এগুলোই ব্যবহৃত হবে। `.vault_pass` Git-এ যায়নি কি না `git status` দিয়ে যাচাই করুন।

২৩.১০ রিভিউ প্রশ্ন

(১) `ansible-lint` কী ধরনের ভুল ধরে, যা `--syntax-check` ধরে না? (২) CI-তে `--check` মোডের সীমা কী? (৩) Vault পাসওয়ার্ড CI-তে নিরাপদে দেওয়ার উপায় কী? (৪) প্রোডাকশন রানে ম্যানুয়াল অনুমোদন কেন জরুরি? (৫) `.gitignore`-এ `.vault_pass` না থাকলে কী বিপদ?

Lab ২৪ • মাল্টি-সার্ভার প্রোডাকশন

Lab ২৪ — মাল্টি-সার্ভার প্রোডাকশন অটোমেশন

২৪.১ উদ্দেশ্য

এই ল্যাব শেষে আপনি পারবেন: (১) পরিবেশ আলাদা করতে (dev/staging/prod ইনভেন্টরি); (২) রোলিং আপডেট কৌশলে (`serial`) সার্ভিস আপডেট করতে; (৩) ডেপ্লয়ের আগে ও পরে হেলথ চেক চালাতে; (৪) ব্যর্থ হলে রোলব্যাক করতে; (৫) পুরো স্ট্যাক এক কমান্ডে যাচাই করতে।

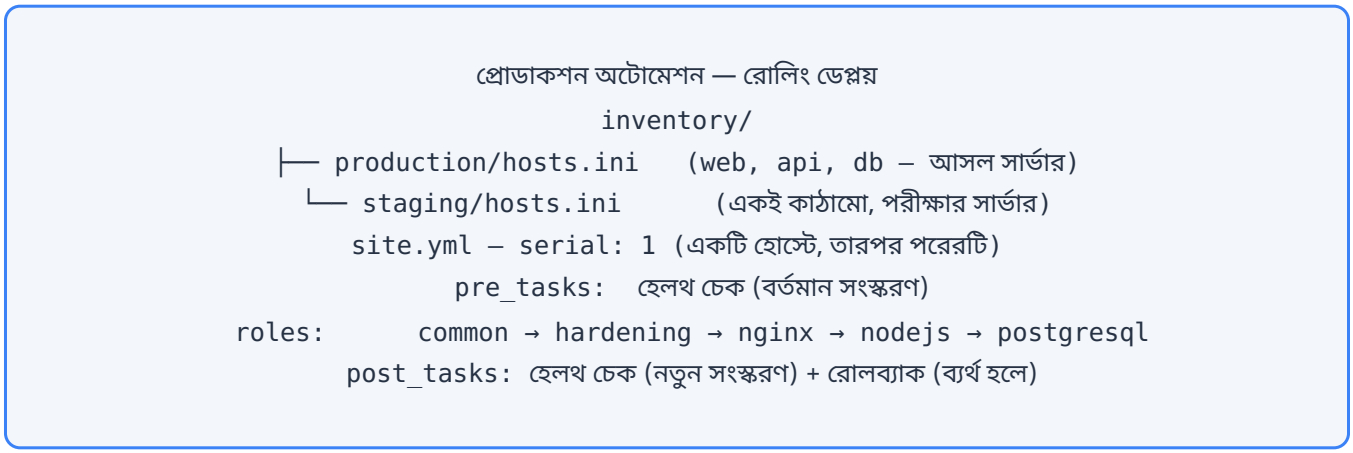
২৪.২ প্রয়োজনীয় টুলস

Lab ১–২৩ সম্পন্ন কন্ট্রোল নোড, তিনটি নোডে চলমান স্ট্যাক (Nginx + Node.js API + PostgreSQL), Vault, Git।

২৪.৩ পূর্বশর্ত

Lab ১৮-এর API ও Lab ১৯-এর ডেটাবেস চলছে; Lab ৮-এর Nginx প্রক্সি কাজ করছে।

২৪.৪ আর্কিটেকচার



২৪.৫ ধাপে ধাপে নির্দেশনা

ধাপ ১ — পরিবেশ আলাদা করুন।

```
mkdir -p ~/ansible-labs/inventory/production ~/ansible-labs/inventory/staging
cp ~/ansible-labs/inventory/hosts.ini ~/ansible-labs/inventory/production/hosts.ini
cp ~/ansible-labs/inventory/hosts.ini ~/ansible-labs/inventory/staging/hosts.ini
cat > ~/ansible-labs/inventory/staging/group_vars_all.yml << 'EOF'
environment: staging
node_env: staging
EOF
echo "প্রোডাকশন ইনভেন্টরি আলাদা রাখুন — ভুলে staging কমান্ড prod-এ চালানো সবচেয়ে সাধারণ দুর্ঘটনা।"
```

ধাপ ২ — রোলিং আপডেট প্লেবুক লিখুন।

```
cat > ~/ansible-labs/playbooks/production-deploy.yml << 'EOF'
---
- name: প্রোডাকশন রোলিং ডেপ্লয়
  hosts: appservers
  become: true
  serial: 1
  max_fail_percentage: 0
  vars:
    app_dir: /opt/technova-api
  pre_tasks:
    - name: ডেপ্লয়ের আগে হেলথ চেক করুন
      ansible.builtin.uri:
        url: http://localhost:3000/health
        status_code: 200
      register: pre_health
      retries: 3
      delay: 2
      until: pre_health.status == 200

  roles:
    - common

  tasks:
    - name: অ্যাপের নতুন কোড কপি করুন
      ansible.builtin.copy:
        src: /tmp/technova-api/
        dest: "{{ app_dir }}/"
        owner: deploy
        group: deploy
        mode: "0644"
      notify: API পুনরায় চালু করুন

    - name: নির্ভরতা আপডেট করুন
      community.general.npm:
        path: "{{ app_dir }}"
        state: present
      become_user: deploy

    - name: হ্যান্ডলার এখনই চালান (হেলথ চেকের আগে)
      ansible.builtin.meta: flush_handlers

  post_tasks:
    - name: ডেপ্লয়ের পর হেলথ চেক করুন
      ansible.builtin.uri:
        url: http://localhost:3000/health
        status_code: 200
      register: post_health
      retries: 5
      delay: 3
      until: post_health.status == 200

    - name: ডেপ্লয় সফল হয়েছে জানান
      ansible.builtin.debug:
        msg: "{{ inventory_hostname }}-এ ডেপ্লয় সফল — হেলথ চেক 200"

  handlers:
    - name: API পুনরায় চালু করুন
      ansible.builtin.systemd:
```

```
        name: technova-api
        state: restarted
EOF
```

ধাপ ৩ — প্রথমে staging-এ, তারপর production-এ চালান।

```
ansible-playbook ~/ansible-labs/playbooks/production-deploy.yml \
  -i ~/ansible-labs/inventory/staging/hosts.ini \
  --vault-password-file ~/.vault_pass --check --diff
ansible-playbook ~/ansible-labs/playbooks/production-deploy.yml \
  -i ~/ansible-labs/inventory/staging/hosts.ini \
  --vault-password-file ~/.vault_pass
# staging সফল হলে তবেই production:
ansible-playbook ~/ansible-labs/playbooks/production-deploy.yml \
  -i ~/ansible-labs/inventory/production/hosts.ini \
  --vault-password-file ~/.vault_pass
```

ধাপ ৪ — পুরো স্ট্যাক যাচাই করুন।

```
curl -s http://203.0.113.11/ | head -2
curl -s http://203.0.113.11/api/health
curl -s http://203.0.113.11/api/hello
ansible all -m shell -a 'systemctl is-active nginx postgresql technova-api 2>/dev/null; sudo
ufw status | head -3' --become
```

CONCEPT: `serial: 1` মানে একবারে একটি হোস্টে ডেপ্লয় হবে — একটি হোস্টে ব্যর্থ হলে বাকিগুলোতে যাওয়া বন্ধ হয়ে যাবে, আর সার্ভিস কখনো পুরোপুরি বন্ধ থাকবে না। `max_fail_percentage: 0` মানে একটি হোস্ট ব্যর্থ হলেই পুরো প্লে থেমে যাবে। এটাই রোলিং ডেপ্লয়ের নিরাপত্তা।

২৪.৬ প্রত্যাশিত আউটপুট

```
# চিত্রণমূলক আউটপুট
$ curl -s http://203.0.113.11/api/health
{"status":"ok","app":"technova-api"}
$ curl -s http://203.0.113.11/api/hello
{"message":"TechNova API থেকে স্বাগতম"}
PLAY RECAP: api ok=9 changed=2 unreachable=0 failed=0
```

২৪.৭ যাচাইকরণ (Verification)

তিনটি `curl` পরীক্ষাই সফল হবে; PLAY RECAP-এ `failed=0` হবে; সব সার্ভিস `active` থাকবে; staging রান production-এর আগে হয়েছে — এই ক্রমটি লগে দেখা যাবে।

২৪.৮ সমস্যা সমাধান (Troubleshooting)

সমস্যা	কারণ ও সমাধান

ভুল পরিবেশে প্লেবুক চলল	`-i` দিয়ে ভুল ইনভেন্টরি দিয়েছেন — সবসময় `ansible-playbook` -এর আগে ইনভেন্টরি পথটি দুবার পড়ুন; প্রোডাকশন ইনভেন্টরিতে সতর্কতা ব্যানার যোগ করুন
রোলিং ডেপ্লয়ে মাঝপথে থামল	একটি হোস্টে হেলথ চেক ব্যর্থ — ওই হোস্টের লগ দেখুন (`journalctl -u technova-api`), ঠিক করে `--limit` দিয়ে শুধু ওই হোস্টে আবার চালান
ডেপ্লয়ের পর পুরোনো সংস্করণ চলছে	হ্যান্ডলার চলেনি — `flush_handlers` আছে কি না দেখুন; সার্ভিসটি আসলে রিস্টার্ট হয়েছে কি না যাচাই করুন

২৪.৯ ক্লিনআপ

কিছু মুছবেন না — এটি আপনার সম্পূর্ণ প্রোডাকশন স্ট্যাক। পরবর্তী অনুশীলনের জন্য `/tmp/technova-api/server.js` -এ একটি ছোট পরিবর্তন করে আবার রোলিং ডেপ্লয় চালিয়ে দেখুন।

২৪.১০ রিভিউ প্রশ্ন

(১) `serial: 1` ও `max_fail_percentage: 0` একসঙ্গে কী নিশ্চিত করে? (২) staging-এর আগে production-এ চালালে কী ঝুঁকি? (৩) `pre_tasks` ও `post_tasks` হেলথ চেকের ভূমিকা কী? (৪) রোলব্যাকের জন্য কী প্রস্তুতি রাখা উচিত (পুরোনো কোড, ব্যাকআপ)? (৫) এক কমান্ডে পুরো স্ট্যাক যাচাই করার সুবিধা কী?

Project ০১ • Linux Server Automation Toolkit

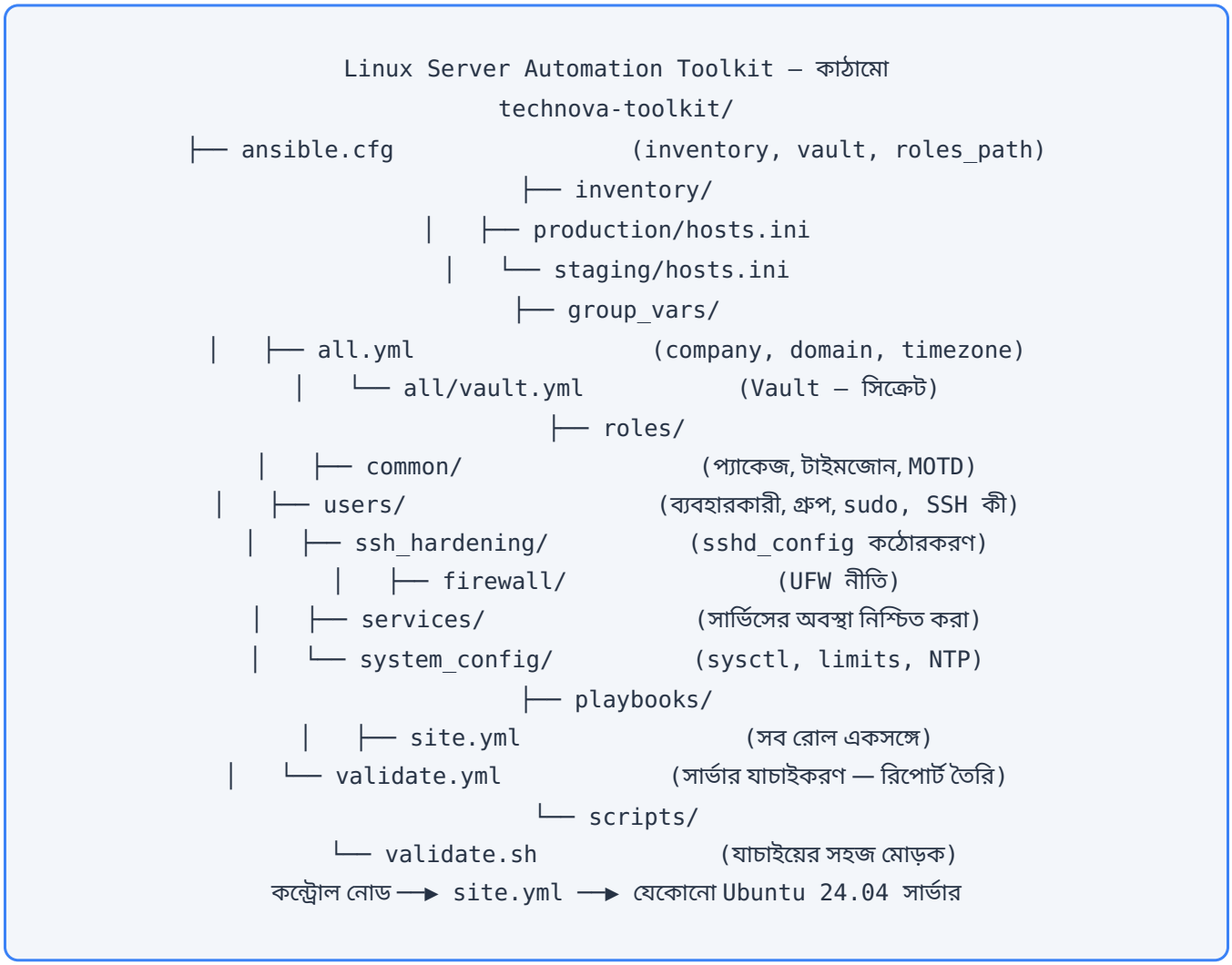
Project ০১ — Linux Server Automation Toolkit

প্রকল্পের পরিচিতি

TechNova-র প্রতিটি নতুন Linux সার্ভার হাতে সাজাতে একজন ইঞ্জিনিয়ারের ২–৩ ঘণ্টা লাগত, আর প্রতিটি সার্ভার একটু আলাদা হতো — কোথাও UFW বন্ধ, কোথাও টাইমজোন ভুল, কোথাও `deploy` ব্যবহারকারী নেই। এই প্রকল্পে আপনি সেই কাজটিকে একটি পুনঃব্যবহারযোগ্য **Linux Server Automation Toolkit** হিসেবে গড়ে তুলবেন: একটি কমান্ডে যেকোনো Ubuntu 24.04 সার্ভারকে TechNova-র মানদণ্ডে (Baseline) আনা যাবে, আর যাচাই করে প্রমাণ করা যাবে যে মানদণ্ডটি মানা হয়েছে। এই টুলকিটটি Lab ১–১৭-এর সব দক্ষতা একত্রিত করে — প্যাকেজ, ব্যবহারকারী, SSH, ফায়ারওয়াল, সার্ভিস, সিস্টেম কনফিগারেশন ও সার্ভার যাচাইকরণ।

> **CONCEPT:** একটি ভালো টুলকিটের তিনটি গুণ থাকে — (১) **Idempotent**: যতবার চালান, ফল একই; (২) **পরিবেশ-নিরপেক্ষ**: একই কোড ল্যাব, স্টেজিং ও প্রোডাকশনে চলে; (৩) **যাচাইযোগ্য**: শেষে একটি রিপোর্ট বলে দেয় কোনটি ঠিক আছে, কোনটি নয়। এই প্রকল্পে তিনটি গুণই তৈরি করবেন।

আর্কিটেকচার



ধাপ ১ — প্রকল্পের কাঠামো তৈরি করুন

```
mkdir -p ~/technova-toolkit/{inventory/production,inventory/
staging,group_vars,roles,playbooks,scripts}
cd ~/technova-toolkit
cp ~/ansible-labs/ansible.cfg .
cp ~/ansible-labs/inventory/hosts.ini inventory/production/hosts.ini
ansible-galaxy init roles/users
ansible-galaxy init roles/ssh_hardening
ansible-galaxy init roles/firewall
ansible-galaxy init roles/services
ansible-galaxy init roles/system_config
cp -r ~/ansible-labs/roles/common roles/common
```

ধাপ ২ — `users` রোল: ব্যবহারকারী, গ্রুপ, SSH কী

এই রোলটি Lab ৬-এর কাজকে পুনঃব্যবহারযোগ্য করে। ভেরিয়েবল দিয়ে ব্যবহারকারীর তালিকা দেওয়া যায়, যাতে প্রতিটি পরিবেশে আলাদা দল বসানো যায়।

```
# roles/users/defaults/main.yml
toolkit_users:
  - name: deploy
    groups: [devops, developers]
    shell: /bin/bash
    sudo: true
    ssh_key: "{{ lookup('file', '~/.ssh/id_ed25519.pub') }}"
  - name: appuser
    groups: [developers]
    shell: /usr/sbin/nologin
    system: true
    sudo: false
toolkit_groups:
  - devops
  - developers
  - deploy

# roles/users/tasks/main.yml
---
- name: গ্রুপ তৈরি করুন
  ansible.builtin.group:
    name: "{{ item }}"
    state: present
  loop: "{{ toolkit_groups }}"

- name: ব্যবহারকারী তৈরি করুন
  ansible.builtin.user:
    name: "{{ item.name }}"
    groups: "{{ item.groups | join(',') }}"
    append: true
    shell: "{{ item.shell }}"
    system: "{{ item.system | default(false) }}"
    create_home: true
  loop: "{{ toolkit_users }}"

- name: sudo অধিকার দিন (শুধু যাদের sudo=true)
  ansible.builtin.copy:
    content: "{{ item.name }} ALL=(ALL) NOPASSWD:ALL\n"
    dest: "/etc/sudoers.d/{{ item.name }}"
    mode: "0440"
    validate: "visudo -cf %s"
  loop: "{{ toolkit_users }}"
  when: item.sudo | default(false)

- name: SSH পাবলিক কী বসান
  ansible.builtin.authorized_key:
    user: "{{ item.name }}"
    state: present
    key: "{{ item.ssh_key }}"
  loop: "{{ toolkit_users }}"
  when: item.ssh_key is defined
```

ধাপ ৩ — `ssh_hardening` ও `firewall` রোল

```
# roles/ssh_hardening/defaults/main.yml
ssh_password_auth: false
```

```
ssh_root_login: false
ssh_empty_passwords: false
ssh_max_auth_tries: 3

# roles/ssh_hardening/tasks/main.yml
---
- name: SSH সেটিংস কঠোর করুন
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: "^#?{{ item.key }}"
    line: "{{ item.key }} {{ item.value }}"
    validate: "/usr/sbin/sshd -t -f %s"
  loop:
    - { key: "PasswordAuthentication", value: "{{ 'yes' if ssh_password_auth else 'no' }}" }
    - { key: "PermitRootLogin", value: "{{ 'yes' if ssh_root_login else 'no' }}" }
    - { key: "PermitEmptyPasswords", value: "{{ 'yes' if ssh_empty_passwords else 'no' }}" }
    - { key: "MaxAuthTries", value: "{{ ssh_max_auth_tries }}" }
  notify: SSH পুনরায় চালু করুন

# roles/ssh_hardening/handlers/main.yml
---
- name: SSH পুনরায় চালু করুন
  ansible.builtin.service:
    name: ssh
    state: restarted

# roles/firewall/defaults/main.yml
firewall_default_policy: deny
firewall_allowed_ports:
  - { port: "22", proto: tcp }
  - { port: "80", proto: tcp, groups: ["webservers"] }
  - { port: "443", proto: tcp, groups: ["webservers"] }

# roles/firewall/tasks/main.yml
---
- name: UFW ইনস্টল করুন
  ansible.builtin.apt:
    name: ufw
    state: present

- name: অনুমোদিত পোর্ট খুলুন
  community.general.ufw:
    rule: allow
    port: "{{ item.port }}"
    proto: "{{ item.proto | default('tcp') }}"
  loop: "{{ firewall_allowed_ports }}"
  when: item.groups is not defined or (item.groups | intersect(group_names) | length > 0)

- name: UFW সক্রিয় করুন (ডিফল্ট deny)
  community.general.ufw:
    state: enabled
    policy: "{{ firewall_default_policy }}"
```

ধাপ ৪ — `services` ও `system_config` রোল

```
# roles/services/defaults/main.yml
toolkit_services_started:
```

```
  - ssh
  - cron
toolkit_services_enabled:
  - ssh
  - cron
  - unattended-upgrades

# roles/services/tasks/main.yml
---
- name: প্রয়োজনীয় সার্ভিস চালু করুন
  ansible.builtin.service:
    name: "{{ item }}"
    state: started
    enabled: true
  loop: "{{ toolkit_services_started }}"

- name: স্বয়ংক্রিয় নিরাপত্তা আপডেট ইনস্টল ও সক্রিয় করুন
  ansible.builtin.apt:
    name: unattended-upgrades
    state: present

# roles/system_config/tasks/main.yml
---
- name: NTP সিঙ্ক চালু করুন
  ansible.builtin.command: timedatectl set-ntp true
  changed_when: false

- name: কার্নেল প্যারামিটার সেট করুন (IP ফরওয়ার্ডিং বন্ধ — ডিফল্ট)
  ansible.posix.sysctl:
    name: net.ipv4.ip_forward
    value: "0"
    state: present
    reload: true

- name: ফাইল ডেসক্রিপ্টর সীমা বাড়ান
  ansible.builtin.lineinfile:
    path: /etc/security/limits.conf
    line: "* soft nofile 65535"
    state: present
```

ধাপ ৫ — মূল প্লেবুক ও যাচাইকরণ প্লেবুক

```
# playbooks/site.yml
---
- name: TechNova Linux Server Automation Toolkit
  hosts: all
  become: true
  roles:
    - common
    - users
    - ssh_hardening
    - firewall
    - services
    - system_config

# playbooks/validate.yml — সার্ভার যাচাইকরণ (পরিবর্তন করে না)
---
```

```
- name: সার্ভার বেসলাইন যাচাই করুন
  hosts: all
  become: true
  gather_facts: true
  tasks:
    - name: SSH হার্ডেনিং যাচাই করুন
      ansible.builtin.shell: |
        grep -q "^PasswordAuthentication no" /etc/ssh/sshd_config &&
        grep -q "^PermitRootLogin no" /etc/ssh/sshd_config &&
        echo PASS || echo FAIL
      register: ssh_check
      changed_when: false

    - name: UFW অবস্থা যাচাই করুন
      ansible.builtin.shell: ufw status | grep -q "Status: active" && echo PASS || echo FAIL
      register: ufw_check
      changed_when: false

    - name: deploy ব্যবহারকারী যাচাই করুন
      ansible.builtin.shell: id deploy && echo PASS || echo FAIL
      register: user_check
      changed_when: false

    - name: যাচাইয়ের সারাংশ দেখান
      ansible.builtin.debug:
        msg: |
          {{ inventory_hostname }} যাচাই:
          SSH হার্ডেনিং: {{ ssh_check.stdout }}
          UFW: {{ ufw_check.stdout }}
          deploy ব্যবহারকারী: {{ user_check.stdout }}
      টাইমজোন: {{ ansible_facts['date_time']['tz'] }}
```

ধাপ ৬ — চালান ও যাচাই করুন

```
cd ~/technova-toolkit
ansible-playbook playbooks/site.yml --vault-password-file ~/.vault_pass
ansible-playbook playbooks/site.yml --vault-password-file ~/.vault_pass  # দ্বিতীয়বার: changed=0
ansible-playbook playbooks/validate.yml
# scripts/validate.sh — এক কমান্ডে যাচাই:
cat > scripts/validate.sh << 'EOF'
#!/bin/bash
set -e
cd "$(dirname "$0")/.."
ansible-playbook playbooks/validate.yml "$@"
EOF
chmod +x scripts/validate.sh
./scripts/validate.sh
```

REAL: TechNova-র মতো ছোট দলে এই টুলকিটের আসল মূল্য নতুন সার্ভার নয় — পুরোনো সার্ভারের ড্রিফট ধরা। সপ্তাহে একবার `validate.yml` চালান: কেউ হাতে UFW বন্ধ করলে, SSH সেটিং বদলালে, বা ব্যবহারকারী যোগ করলে যাচাইয়ে ধরা পড়বে। আর `site.yml` আবার চালালে সার্ভারটি মানদণ্ডে ফিরে আসবে — এটাই Configuration Management-এর মূল প্রতিশ্রুতি।

প্রত্যাশিত ফলাফল

প্রথম রানে সব টাস্ক `changed` বা `ok` হবে, `failed=0` হবে। দ্বিতীয় রানে `changed=0` হবে — Idempotency প্রমাণিত। `validate.yml`-এ তিনটি যাচাইতেই `PASS` আসবে। নতুন একটি VM-এ শুধু ইনভেন্টরিতে IP যোগ করে একই প্লেবুক চালালে সেটিও একই মানদণ্ডে আসবে।

সমস্যা সমাধান

সমস্যা	সমাধান
SSH হার্ডেনিংয়ের পর সংযোগ বন্ধ	কী-লগইন যাচাই না করে প্লেবুক চালিয়েছেন — VM কনসোল থেকে `sshd_config` ঠিক করুন; ভবিষ্যতে আগে `ssh-copy-id` যাচাই করুন
UFW-এ পোর্ট খোলা হয়নি	`firewall_allowed_ports`-এর `groups` শর্তটি দেখুন — হোস্টটি সঠিক গ্রুপে আছে কি না `ansible-inventory --graph` দিয়ে যাচাই করুন
দ্বিতীয় রানেও `changed` আসছে	কোন টাস্কটি বদলাচ্ছে তা `--diff` দিয়ে দেখুন — সাধারণত `command` / `shell` টাস্কে `changed_when: false` অনুপস্থিত

রিভিউ প্রশ্ন

(১) টুলকিটে নতুন একটি রোল (যেমন `monitoring`) যোগ করতে কী কী করতে হবে? (২) `validate.yml` কেন কোনো পরিবর্তন করে না, এবং এটি কেন জরুরি? (৩) একই টুলকিট staging ও production-এ চালাতে কী বদলাতে হবে? (৪) Idempotency না থাকলে সপ্তাহে একবার টুলকিট চালানো কেন বিপজ্জনক? (৫) `toolkit_users` তালিকায় নতুন ব্যবহারকারী যোগ করলে কী হবে?

Project ০২ • Automated MERN Production Deployment

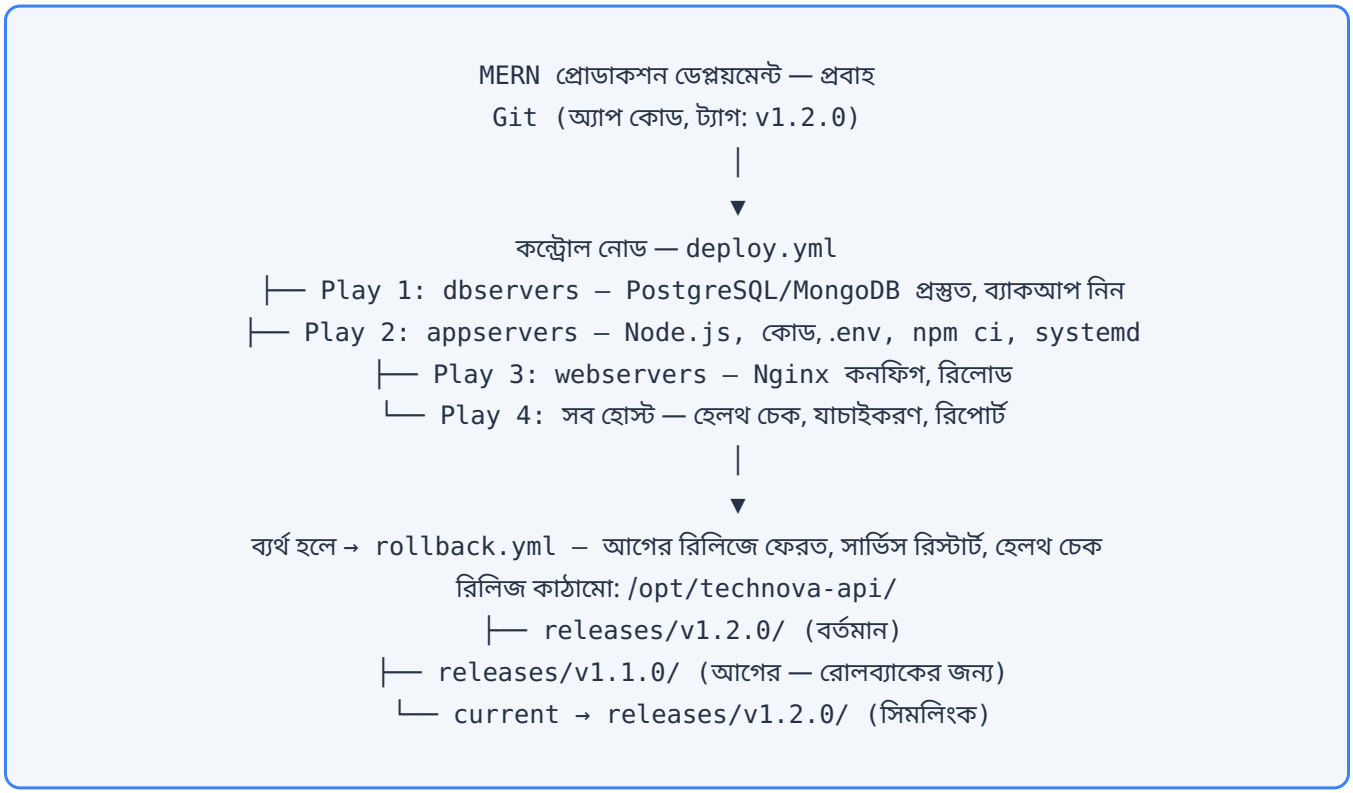
Project ০২ — Automated MERN Production Deployment

প্রকল্পের পরিচিতি

TechNova-র MERN অ্যাপ (MongoDB + Express + React + Node.js) এখনো হাতে ডেপ্লয় করা হয়: একজন ইঞ্জিনিয়ার সার্ভারে ঢুকে Git pull করেন, `npm install` করেন, `.env` হাতে বসান, PM2 রিস্টার্ট করেন — আর মাঝে মাঝে ভুল `.env` বা ভুলে যাওয়া ধাপে সাইট বন্ধ হয়ে যায়। এই প্রকল্পে আপনি সেই পুরো ডেপ্লয়মেন্টকে একটি Ansible-চালিত, যাচাইকৃত, রোলব্যাক-সক্ষম পাইপলাইনে রূপান্তর করবেন: Nginx ইনস্টল/কনফিগ, Node.js রানটাইম, ডেটাবেস কনফিগ, অ্যাপ ডেপ্লয়, পরিবেশ কনফিগ, সার্ভিস ব্যবস্থাপনা, হেলথ চেক, ডেপ্লয় যাচাইকরণ ও নিরাপদ রোলব্যাক — সবকিছু এক কমান্ডে।

> **CONCEPT:** একটি ভালো ডেপ্লয়মেন্ট পাইপলাইনের চারটি গুণ থাকে — (১) **পুনরুৎপাদনযোগ্য**: একই কমান্ডে একই ফল; (২) **যাচাইকৃত**: প্রতিটি ধাপের পর হেলথ চেক; (৩) **পরমাণু-সদৃশ**: হয় পুরোটা সফল, নয়তো আগের অবস্থায় ফেরত; (৪) **পর্যবেক্ষণযোগ্য**: প্রতিটি ধাপের লগ ও ফলাফল দেখা যায়। এই প্রকল্পে চারটি গুণই তৈরি করবেন।

আর্কিটেকচার



ধাপ ১ — প্রকল্পের কাঠামো তৈরি করুন

```
mkdir -p ~/mern-deploy/{playbooks,roles/{nodejs,app_deploy,database,nginx,healthcheck}/
tasks,templates,inventory/production}
cd ~/mern-deploy
cp ~/ansible-labs/ansible.cfg .
cp ~/ansible-labs/inventory/hosts.ini inventory/production/hosts.ini
# Vault সিক্রেট (Lab ১৬-এর মতো):
ansible-vault create group_vars/vault.yml --vault-password-file ~/.vault_pass
# ভেতরে লিখুন: db_app_password, api_jwt_secret (CHANGE-ME-... মান)
```

ধাপ ২ — রিলিজ-ভিত্তিক অ্যাপ ডেপ্লয় রোল

রোলব্যাকের চাবিকাঠি হলো রিলিজ ডিরেক্টরি: প্রতিটি ডেপ্লয় নতুন ফোল্ডারে যায়, `current` সিমলিংক শুধু সফল হেলথ চেকের পর বদলায়। আগের রিলিজ মুছে ফেলা হয় না — তাই রোলব্যাক মানে শুধু সিমলিংক ফেরত দেওয়া।

```
# roles/app_deploy/defaults/main.yml
app_name: technova-api
app_dir: /opt/technova-api
app_user: deploy
app_version: "v1.2.0"
app_repo: "https://github.com/technova/technova-api.git"
app_port: 3000
keep_releases: 3

# roles/app_deploy/tasks/main.yml
---
- name: রিলিজ ডিরেক্টরি তৈরি করুন
  ansible.builtin.file:
    path: "{{ app_dir }}/releases/{{ app_version }}"
    state: directory
    owner: "{{ app_user }}"
    group: "{{ app_user }}"
    mode: "0755"

- name: অ্যাপের কোড Git থেকে টানুন
  ansible.builtin.git:
    repo: "{{ app_repo }}"
    dest: "{{ app_dir }}/releases/{{ app_version }}"
    version: "{{ app_version }}"
    force: true
  become_user: "{{ app_user }}"

- name: নির্ভরতা ইনস্টল করুন (npm ci)
  community.general.npm:
    path: "{{ app_dir }}/releases/{{ app_version }}"
    state: present
  become_user: "{{ app_user }}"

- name: পরিবেশ ফাইল (.env) তৈরি করুন
  ansible.builtin.template:
    src: dotenv.j2
    dest: "{{ app_dir }}/releases/{{ app_version }}/.env"
    owner: "{{ app_user }}"
    group: "{{ app_user }}"
```

```
    mode: "0600"
  no_log: true

- name: current সিমলিংক নতুন রিলিজে নির্দেশ করুন
  ansible.builtin.file:
    src: "{{ app_dir }}/releases/{{ app_version }}"
    dest: "{{ app_dir }}/current"
    state: link
    owner: "{{ app_user }}"
    group: "{{ app_user }}"
  notify: API পুনরায় চালু করুন

- name: পুরোনো রিলিজ পরিষ্কার করুন (শেষ ৩টি রাখুন)
  ansible.builtin.shell: |
    ls -dt {{ app_dir }}/releases/*/ | tail -n +{{ keep_releases + 1 }} | xargs -r rm -rf
  args:
    executable: /bin/bash
  changed_when: false
```

ধাপ ৩ — ডেটাবেস, Nginx ও হেলথ চেক

```
# roles/database/tasks/main.yml (PostgreSQL; MongoDB হলে community.mongodb ব্যবহার করুন)
---
- name: PostgreSQL চালু আছে কি না যাচাই করুন
  ansible.builtin.service:
    name: postgresql
    state: started
    enabled: true

- name: ডেপ্লয়ের আগে ডেটাবেস ব্যাকআপ নিন
  ansible.builtin.shell: |
    pg_dump -U postgres {{ db_name }} > /var/backups/{{ db_name }}-
{{ ansible_date_time.iso8601_basic_short }}.sql
  become_user: postgres
  changed_when: false

- name: অ্যাপ ব্যবহারকারী ও অধিকার নিশ্চিত করুন
  community.postgresql.postgresql_user:
    name: "{{ db_name }}"
    password: "{{ db_app_password }}"
    db: "{{ db_name }}"
    priv: "ALL"
    state: present
  become_user: postgres
  no_log: true

# roles/healthcheck/tasks/main.yml
---
- name: API হেলথ চেক করুন
  ansible.builtin.uri:
    url: "http://localhost:{{ app_port }}/health"
    status_code: 200
  register: api_health
  retries: 5
  delay: 3
  until: api_health.status == 200
```

```
- name: ওয়েবসাইট (Nginx প্রক্সি) হেলথ চেক করুন
  ansible.builtin.uri:
    url: "http://{{ inventory_hostname }}/api/health"
    status_code: 200
  register: web_health
  retries: 3
  delay: 2
  until: web_health.status == 200
  when: "'webservers' in group_names"

- name: হেলথ চেকের ফল দেখান
  ansible.builtin.debug:
    msg: "API: {{ api_health.status }}, Web: {{ web_health.status | default('N/A') }}"
```

ধাপ ৪ — মূল ডেপ্লয় প্লেবুক ও রোলব্যাক প্লেবুক

```
# playbooks/deploy.yml
---
- name: ডেটাবেস প্রস্তুত করুন (ব্যাকআপসহ)
  hosts: dbservers
  become: true
  roles:
    - database

- name: অ্যাপ ডেপ্লয় করুন
  hosts: appservers
  become: true
  serial: 1
  roles:
    - nodejs
    - app_deploy

- name: Nginx কনফিগ আপডেট ও রিলোড করুন
  hosts: webservers
  become: true
  roles:
    - nginx

- name: ডেপ্লয় যাচাই করুন
  hosts: all
  become: true
  roles:
    - healthcheck

# playbooks/rollback.yml — আগের রিলিজে ফেরত
---
- name: নিরাপদ রোলব্যাক
  hosts: appservers
  become: true
  vars:
    previous_version: "v1.1.0"
  tasks:
    - name: আগের রিলিজ আছে কি না যাচাই করুন
      ansible.builtin.stat:
        path: "{{ app_dir }}/releases/{{ previous_version }}"
      register: prev_release
```

```
    - name: আগের রিলিজ না থাকলে থামুন
      ansible.builtin.fail:
        msg: "আগের রিলিজ {{ previous_version }} পাওয়া যায়নি — রোলব্যাক সম্ভব নয়"
      when: not prev_release.stat.exists

    - name: current সিমলিংক আগের রিলিজে ফেরত দিন
      ansible.builtin.file:
        src: "{{ app_dir }}/releases/{{ previous_version }}"
        dest: "{{ app_dir }}/current"
        state: link
        force: true
      notify: API পুনরায় চালু করুন

    - name: হ্যান্ডলার এখনই চালান
      ansible.builtin.meta: flush_handlers

    - name: রোলব্যাকের পর হেলথ চেক করুন
      ansible.builtin.uri:
        url: "http://localhost:{{ app_port }}/health"
        status_code: 200
      retries: 5
      delay: 3
      until: true

  handlers:
    - name: API পুনরায় চালু করুন
      ansible.builtin.systemd:
        name: technova-api
        state: restarted
```

ধাপ ৫ — চালান, যাচাই করুন, রোলব্যাক পরীক্ষা করুন

```
cd ~/mern-deploy
# ১. শুকনো রান (কিছু বদলাবে না):
ansible-playbook playbooks/deploy.yml -i inventory/production/hosts.ini --check --diff --
vault-password-file ~/.vault_pass
# ২. আসল ডেপ্লয়:
ansible-playbook playbooks/deploy.yml -i inventory/production/hosts.ini --vault-password-file
~/.vault_pass
# ৩. যাচাই:
curl -s http://203.0.113.11/api/health
curl -s http://203.0.113.11/api/hello
# ৪. রোলব্যাকের মহড়া (প্রোডাকশনে করার আগে staging-এ):
ansible-playbook playbooks/rollback.yml -i inventory/production/hosts.ini -e
"previous_version=v1.1.0" --vault-password-file ~/.vault_pass
curl -s http://203.0.113.11/api/health
```

> **REAL:** রোলব্যাকের মহড়া ডেপ্লয়ের মতোই গুরুত্বপূর্ণ — যে রোলব্যাক কখনো পরীক্ষা করা হয়নি, সেটি দরকারের সময় কাজ করবে বলে ধরে নেওয়া যায় না। TechNova-র নিয়ম: প্রতিটি নতুন রিলিজের পর staging-এ একবার রোলব্যাক চালিয়ে দেখুন যে আগের সংস্করণে ফেরত যাওয়া যায়, তারপরই প্রোডাকশনে ডেপ্লয় করুন। আর ডেটাবেস মাইগ্রেশন থাকলে রোলব্যাকের আগে মাইগ্রেশনের রিভার্স পরিকল্পনাও রাখুন — কোড ফেরত গেলেও ডেটাবেস স্কিমা নিজে ফেরত যায় না।

প্রত্যাশিত ফলাফল

ডেপ্লয়ের পর `/opt/technova-api/releases/v1.2.0/` থাকবে, `current` সেটিকে নির্দেশ করবে, `/var/backups/`-এ ডেটাবেস ব্যাকআপ থাকবে, আর তিনটি হেলথ চেক (API সরাসরি, Nginx প্রক্সি, ওয়েবসাইট) সফল হবে। রোলব্যাকের পর `current` `v1.1.0`-কে নির্দেশ করবে এবং হেলথ চেক আবার সফল হবে।

সমস্যা সমাধান

সমস্যা	সমাধান
Git ক্লোনে প্রমাণীকরণ ব্যর্থ	প্রাইভেট রিপোর জন্য Deploy Key বা HTTPS টোকেন ব্যবহার করুন; টোকেন Vault-এ রাখুন, প্লেবুকে নয়
`npm ci` ব্যর্থ	`package-lock.json` Git-এ আছে কি না দেখুন; Node.js সংস্করণ `package.json`-এর `engines`-এর সঙ্গে মিলছে কি না দেখুন
ডেপ্লয়ের পর 502	API সার্ভিস চালু হয়নি — `journalctl -u technova-api -n 50` দেখুন; `.env`-এর `PORT` ও Nginx প্রক্সি পোর্ট মিলছে কি না দেখুন
রোলব্যাকে `current` বদলাচ্ছে না	`file` মডিউলে `force: true` আছে কি না দেখুন; আগের রিলিজটি `keep_releases` পরিষ্কারে মুছে যায়নি তো

রিভিউ প্রশ্ন

(১) রিলিজ ডিরেক্টরি ও `current` সিমলিংক রোলব্যাককে সহজ করে কীভাবে? (২) ডেপ্লয়ের আগে ডেটাবেস ব্যাকআপ নেওয়া কেন জরুরি? (৩) `serial: 1` এই প্রকল্পে কী নিরাপত্তা দেয়? (৪) ডেটাবেস মাইগ্রেশন থাকলে রোলব্যাক কেন শুধু কোড ফেরত দিলেই হয় না? (৫) `.env` ফাইলে `no_log: true` না দিলে কী ঝুঁকি?

Project ০৩ • TechNova Configuration Management Platform

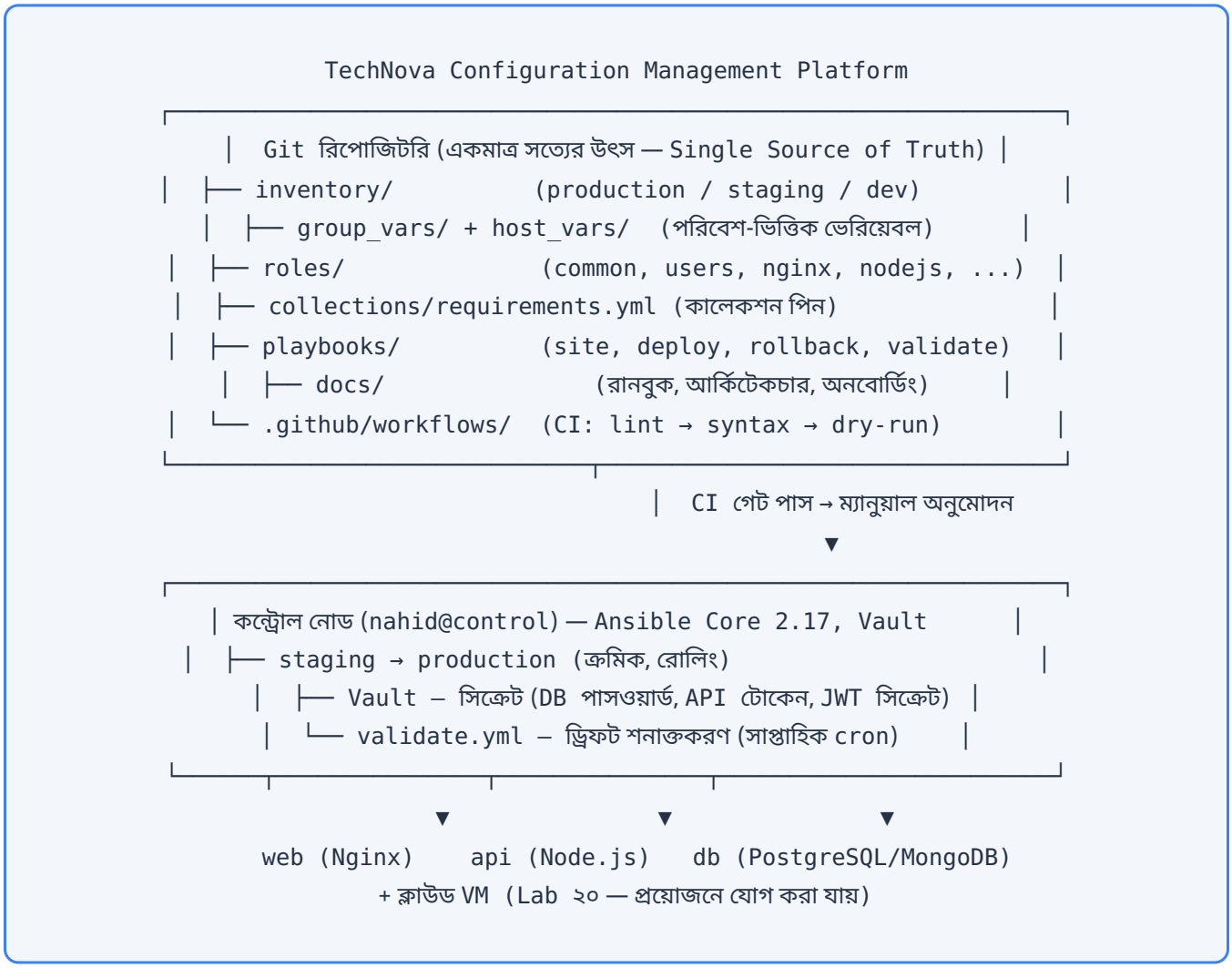
Project ০৩ — TechNova Configuration Management Platform

প্রকল্পের পরিচিতি

Project ০১-এর টুলকিট একটি সার্ভার সাজায়, Project ০২ একটি অ্যাপ ডেপ্লয় করে — কিন্তু TechNova-র এখন দরকার একটি পূর্ণাঙ্গ **Configuration Management Platform**: এক জায়গা থেকে সব সার্ভারের ইনভেন্টরি, পরিবেশ, রোল, সিক্রেট, ক্লাউড, হার্ডেনিং, CI/CD, যাচাইকরণ ও ডকুমেন্টেশন পরিচালনা করা। এই প্রকল্পে আপনি সেই প্ল্যাটফর্মটি গড়ে তুলবেন — Lab ১–২৪-এর সবকিছু একত্রিত করে, যাতে নতুন সার্ভার যোগ করা, নতুন পরিবেশ তৈরি করা, বা নতুন দলের সদস্যকে অনবোর্ড করা একটি নথিভুক্ত, পুনরুৎপাদনযোগ্য প্রক্রিয়া হয়।

> **CONCEPT:** প্ল্যাটফর্ম ও টুলকিটের পার্থক্য হলো পরিচালনা। টুলকিট একটি কাজ করে; প্ল্যাটফর্ম বলে দেয় কে, কখন, কোন পরিবেশে, কোন অনুমোদনে, কী পরিবর্তন করল — এবং পরিবর্তনটি ভুল হলে কীভাবে ফেরত যাওয়া যায়। এই প্রকল্পের মূল বিষয় তাই কোড নয়, প্রক্রিয়া: ইনভেন্টরি আর্কিটেকচার, পরিবেশ পৃথকীকরণ, সিক্রেট ব্যবস্থাপনা, CI/CD গেট ও রানবুক।

আর্কিটেকচার



ধাপ ১ — ইনভেন্টরি আর্কিটেকচার ও পরিবেশ পৃথকীকরণ

প্রতিটি পরিবেশের নিজস্ব ইনভেন্টরি ও ভেরিয়েবল থাকবে; কোড (রোল ও প্লেবুক) সবার জন্য একই। এতে staging-এ পরীক্ষা করা কোডই production-এ চলে — পরিবেশের পার্থক্য শুধু ভেরিয়েবলে।

```
technova-platform/
├── inventory/
│   ├── dev/hosts.ini
│   ├── staging/hosts.ini
│   └── production/hosts.ini
├── group_vars/
│   ├── all.yml
│   ├── all/vault.yml           # Vault এনক্রিপ্টেড
│   ├── dev.yml
│   ├── staging.yml
│   └── production.yml
├── host_vars/
│   ├── web.yml
│   ├── api.yml
│   └── db.yml
├── roles/                      # Lab ১৪, Project ০১–০২-এর রোল
```

```
├── collections/requirements.yml
├── playbooks/
│   ├── site.yml
│   ├── deploy.yml
│   ├── rollback.yml
│   ├── validate.yml
│   └── drift-check.yml
├── docs/
│   ├── architecture.md
│   ├── runbooks/
│   └── onboarding.md
└── .github/workflows/ansible-ci.yml

# group_vars/production.yml — প্রোডাকশনের কঠোর মান
environment: production
node_env: production
firewall_default_policy: deny
ssh_max_auth_tries: 3
app_version: "v1.2.0"
keep_releases: 5

# group_vars/staging.yml — স্টেজিংয়ের শিথিল মান
environment: staging
node_env: staging
app_version: "v1.2.0-rc1"
keep_releases: 2
```

ধাপ ২ — রোল, কালেকশন ও সিক্রেট ব্যবস্থাপনা

```
# collections/requirements.yml
---
collections:
  - name: community.general
    version: ">=9.0.0"
  - name: community.docker
    version: ">=3.0.0"
  - name: community.postgresql
    version: ">=3.0.0"
  - name: ansible.posix
    version: ">=1.5.0"
  - name: community.digitalocean
    version: ">=1.0.0"

# সিক্রেট ব্যবস্থাপনার নিয়ম (docs/secrets.md-এ নথিভুক্ত):
# ১. সব সিক্রেট Vault-এ (group_vars/all/vault.yml) — প্লেইন টেক্সটে কখনো নয়
# ২. ভল্ট পাসওয়ার্ড ফাইল ~/.vault_pass-এ, Git-এ নয় (.gitignore-এ আছে)
# ৩. CI-তে সিক্রেট GitHub Secrets থেকে আসে
# ৪. প্রতি ৯০ দিনে ভল্ট পাসওয়ার্ড rekey করুন: ansible-vault rekey

# ansible.cfg — প্ল্যাটফর্মের কনফিগ
[defaults]
inventory = ./inventory/staging/hosts.ini
host_key_checking = False
retry_files_enabled = False
collections_path = ~/.ansible/collections
roles_path = ./roles
vault_password_file = ~/.vault_pass
```

```
forks = 10
[privilege_escalation]
become = True
```

ধাপ ৩ — সার্ভার হার্ডেনিং ও CI/CD ইন্টিগ্রেশন

হার্ডেনিং Project ০১-এর `ssh_hardening` ও `firewall` রোল থেকে আসে; CI/CD Lab ২৩-এর ওয়ার্কফ্লো থেকে। প্ল্যাটফর্মে একটি নতুন ধাপ যোগ করুন — **ড্রিফট চেক**: সাপ্তাহিক cron-এ `validate.yml` চালিয়ে দেখুন কোনো সার্ভার মানদণ্ড থেকে সরে গেছে কি না।

```
# playbooks/drift-check.yml
---
- name: ড্রিফট শনাক্তকরণ (পরিবর্তন করে না — শুধু রিপোর্ট)
  hosts: all
  become: true
  gather_facts: true
  tasks:
    - name: SSH হার্ডেনিং যাচাই করুন
      ansible.builtin.shell: |
        grep -q "^PasswordAuthentication no" /etc/ssh/sshd_config &&
        grep -q "^PermitRootLogin no" /etc/ssh/sshd_config && echo PASS || echo FAIL
      register: drift_ssh
      changed_when: false

    - name: UFW যাচাই করুন
      ansible.builtin.shell: ufw status | grep -q "Status: active" && echo PASS || echo FAIL
      register: drift_ufw
      changed_when: false

    - name: অপ্রত্যাশিত ব্যবহারকারী আছে কি না দেখুন
      ansible.builtin.shell: |
        getent passwd | awk -F: '$3 >= 1000 {print $1}' | sort
      register: drift_users
      changed_when: false

    - name: ড্রিফট রিপোর্ট দেখান
      ansible.builtin.debug:
        msg: |
          {{ inventory_hostname }} — ড্রিফট রিপোর্ট:
          SSH: {{ drift_ssh.stdout }} | UFW: {{ drift_ufw.stdout }}
    ব্যবহারকারী (UID>=1000): {{ drift_users.stdout_lines | join(', ') }}

# সাপ্তাহিক cron (কন্ট্রোল নোডে):
# 0 3 * * 0 cd ~/technova-platform && ansible-playbook playbooks/drift-check.yml -i inventory/
production/hosts.ini >> /var/log/technova-drift.log 2>&1
```

ধাপ ৪ — যাচাইকরণ, ডকুমেন্টেশন ও অপারেশনাল রানবুক

প্ল্যাটফর্মের ডকুমেন্টেশন কোডের মতোই গুরুত্বপূর্ণ। `docs/`-এ অন্তত এই ফাইলগুলো লিখুন:

```
# docs/architecture.md — এক পাতায় পুরো প্ল্যাটফর্মের চিত্র ও সিদ্ধান্ত
#   - ইনভেন্টরি কাঠামো, পরিবেশের পার্থক্য, রোলের তালিকা, সিক্রেট নীতি
# docs/runbooks/new-server.md — নতুন সার্ভার যোগ করার রানবুক:
#   ১. VM তৈরি করুন (Lab ২০) → ২. ইনভেন্টরিতে IP যোগ করুন
```

```
#   ৩. ssh-copy-id করুন → ৪. site.yml চালান → ৫. validate.yml দিয়ে যাচাই করুন
# docs/runbooks/deploy.md — ডেপ্লয়ের রানবুক:
#   ১. Git-এ ট্যাগ করুন → ২. staging-এ deploy.yml → ৩. হেলথ চেক
#   ৪. অনুমোদন নিন → ৫. production-এ deploy.yml → ৬. হেলথ চেক
#   ৭. ব্যর্থ হলে rollback.yml (আগের ট্যাগ)
# docs/runbooks/incident.md — সার্ভার বন্ধ হলে করণীয়:
#   ১. validate.yml দিয়ে অবস্থা দেখুন → ২. journalctl দিয়ে লগ দেখুন
#   ৩. site.yml আবার চালান (ড্রিফট ঠিক হবে) → ৪. না হলে রোলব্যাক
# docs/onboarding.md — নতুন দলের সদস্যের চেকলিস্ট:
#   ১. SSH কী তৈরি ও কন্ট্রোল নোডে যোগ → ২. Vault পাসওয়ার্ড পান
#   ৩. Lab ১–৯ শেষ করুন → ৪. staging-এ প্রথম ডেপ্লয় (তত্ত্বাবধানে)

# রানবুকের মহড়া — প্রতিটি রানবুক staging-এ একবার চালিয়ে দেখুন:
ansible-playbook playbooks/site.yml -i inventory/staging/hosts.ini --check --diff
ansible-playbook playbooks/validate.yml -i inventory/staging/hosts.ini
ansible-playbook playbooks/drift-check.yml -i inventory/staging/hosts.ini
```

> **REAL:** একটি প্ল্যাটফর্মের সাফল্য মাপা হয় নতুন একজন সদস্য কত দ্রুত প্রথম নিরাপদ ডেপ্লয় করতে পারে, তা দিয়ে — কোডের লাইন দিয়ে নয়। TechNova-তে নতুন সদস্য `docs/onboarding.md` পড়ে, staging-এ একটি ডেপ্লয় করে, তারপরই production-এর অনুমতি পায়। ডকুমেন্টেশন পুরোনো হলে প্ল্যাটফর্মটি ধীরে ধীরে "শুধু একজন জানে" এমন অবস্থায় ফিরে যায় — তাই প্রতিটি পরিবর্তনের সঙ্গে ডকুমেন্টেশন আপডেট করাই প্ল্যাটফর্মের নিয়ম।

প্রত্যাশিত ফলাফল

তিনটি পরিবেশের ইনভেন্টরি থেকে একই `site.yml` চলবে; staging-এ ডেপ্লয় সফল হলে তবেই production-এ চালানো হবে; `drift-check.yml` সাপ্তাহিক রিপোর্ট দেবে; `docs/`-এর রানবুক পড়ে নতুন সদস্য একা একটি সার্ভার যোগ করতে পারবে। CI-তে lint, syntax ও dry-run — তিনটি জবই সবুজ হবে।

সমস্যা সমাধান

সমস্যা	সমাধান
staging ও production-এর আচরণ আলাদা	ভেরিয়েবল ফাইলের পার্থক্য দেখুন (`group_vars/staging.yml` বনাম `production.yml`); কোড একই, শুধু ভেরিয়েবল আলাদা হওয়া উচিত
ড্রিফট রিপোর্টে অপ্রত্যাশিত ব্যবহারকারী	কেউ হাতে ব্যবহারকারী তৈরি করেছে — `users` রোলে যোগ করুন অথবা মুছে ফেলুন; হাতে পরিবর্তন নয়, কোডে পরিবর্তনই নিয়ম
CI dry-run-এ Vault ত্রুটি	GitHub Secrets থেকে পাসওয়ার্ড ফাইল তৈরি করুন; আসল Vault ফাইল CI-তে কমিট করবেন না

রিভিউ প্রশ্ন

(১) "একমাত্র সত্যের উৎস" (Single Source of Truth) বলতে কী বোঝায়, এবং Git কেন সেই ভূমিকায়? (২) পরিবেশ আলাদা করতে কোড নয়, ভেরিয়েবল বদলানো হয় কেন? (৩) ড্রিফট কী, এবং এটি নিয়মিত যাচাই করা কেন জরুরি? (৪) রানবুক না থাকলে প্ল্যাটফর্মের কী ক্ষতি? (৫) নতুন দলের সদস্যকে production-এর অনুমতি দেওয়ার আগে কী যাচাই করবেন?

Capstone • TechNova Enterprise Server Automation Platform

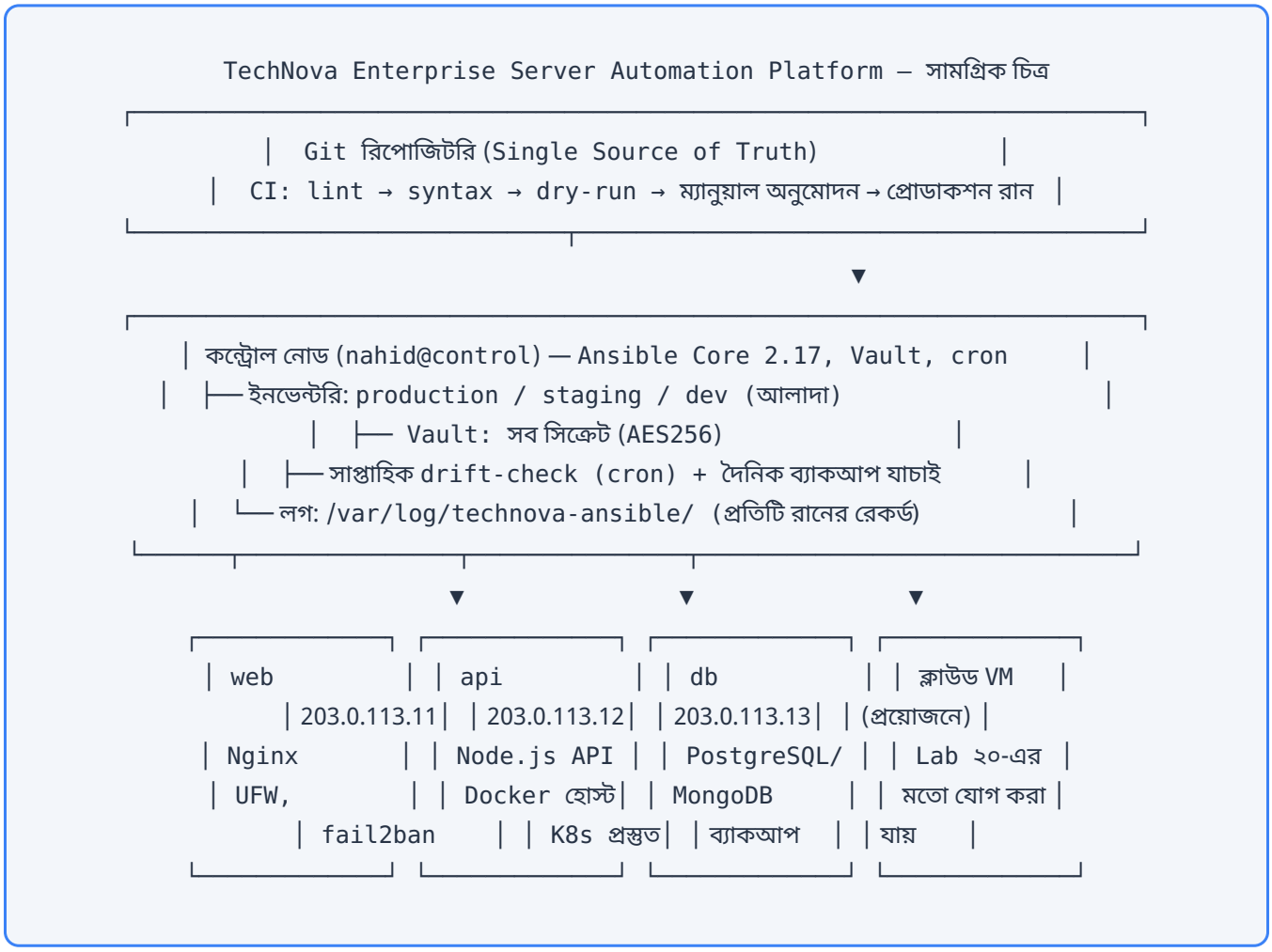
Capstone — TechNova Enterprise Server Automation Platform

ক্যাপস্টোনের পরিচিতি

এই ক্যাপস্টোনে আপনি Volume 09-এর সবকিছু একত্রিত করে TechNova-র জন্য একটি সম্পূর্ণ **Enterprise Server Automation Platform** গড়ে তুলবেন — এমন একটি প্ল্যাটফর্ম যা একাধিক Linux সার্ভারকে এক জায়গা থেকে পরিচালনা করে: সার্ভার ইনভেন্টরি, মানদণ্ডভিত্তিক বেসলাইন কনফিগারেশন, ব্যবহারকারী ও SSH ব্যবস্থাপনা, প্যাকেজ ব্যবস্থাপনা, সার্ভিস ব্যবস্থাপনা, নিরাপত্তা হার্ডেনিং, অ্যাপ্লিকেশন ডেপ্লয়মেন্ট, সিক্রেট ব্যবস্থাপনা, কনফিগারেশন যাচাইকরণ, CI/CD ইন্টিগ্রেশন, ড্রিফট শনাক্তকরণ, ব্যর্থতা থেকে পুনরুদ্ধার, ডকুমেন্টেশন ও অপারেশনাল রানবুক। এই ক্যাপস্টোনটি কোনো একক ল্যাবের পুনরাবৃত্তি নয় — এটি প্রমাণ যে আপনি একটি বাস্তব অবকাঠামোর দায়িত্ব নিতে পারেন।

> **CONCEPT:** এন্টারপ্রাইজ প্ল্যাটফর্মের মানদণ্ড চারটি: (১) **সম্পূর্ণতা** — সার্ভারের জীবনচক্রের প্রতিটি ধাপ (তৈরি → কনফিগার → ডেপ্লয় → যাচাই → পুনরুদ্ধার) অটোমেটেড; (২) **নিরাপত্তা** — সিক্রেট কখনো প্লেইন টেক্সটে নয়, সর্বনিম্ন অধিকার (Least Privilege); (৩) **পুনরুদ্ধারযোগ্যতা** — যেকোনো ব্যর্থতা থেকে নির্দিষ্ট সময়ে ফেরত যাওয়া যায়; (৪) **হস্তান্তরযোগ্যতা** — ডকুমেন্টেশন ও রানবুক পড়ে অন্য কেউ প্ল্যাটফর্মটি চালাতে পারে। গ্রেডিং রুব্রিক এই চারটি মানদণ্ডেই সাজানো হয়েছে।

ক্যাপস্টোনের আর্কিটেকচার



সম্পূর্ণ রেফারেন্স সমাধানের রূপরেখা

নিচের ডিরেক্টরি কাঠামোটিই ক্যাপস্টোনের রেফারেন্স সমাধান। প্রতিটি ফাইলের কাজ সংক্ষেপে দেওয়া হয়েছে — আপনার জমা দেওয়া প্রকল্পে এই কাঠামোর সব অংশ থাকতে হবে।

```
technova-enterprise-platform/
├── README.md
│     # প্ল্যাটফর্মের পরিচিতি, দ্রুত শুরু, পরিবেশের তালিকা
├── ansible.cfg
│     # inventory=./inventory/staging/hosts.ini (ডিফল্ট: staging — নিরাপদ)
│     # vault_password_file, roles_path, collections_path, forks=10
├── .gitignore
│     # .vault_pass, *.retry, __pycache__/, /var/log/
├── .github/workflows/ansible-ci.yml
│     # lint → syntax-check → dry-run (Lab ২৩)
├── collections/requirements.yml
│     # community.general, community.docker, community.postgresql,
│     # ansible.posix, community.digitalocean (সংস্করণ পিন করা)
├── inventory/
│   ├── dev/hosts.ini
│   ├── staging/hosts.ini
│   └── production/hosts.ini
│     # প্রতিটি পরিবেশে webservers / appservers / dbservers গ্রুপ
```

```
├── group_vars/
│   ├── all.yml
│   │   # company: TechNova, domain: technova.example, timezone: Asia/Dhaka
│   ├── all/vault.yml
│   │   # Vault এনক্রিপ্টেড: db_app_password, api_jwt_secret, cloud_api_token
│   ├── dev.yml
│   ├── staging.yml
│   └── production.yml
│     # পরিবেশ-ভিত্তিক মান (app_version, keep_releases, firewall নীতি)
├── host_vars/
│   ├── web.yml        # app_name, http_port, server_name
│   ├── api.yml        # app_port, node_env
│   └── db.yml         # db_engine, db_name
├── roles/
│   ├── common/            # প্যাকেজ, টাইমজোন, MOTD (Lab ১৪)
│   ├── users/             # ব্যবহারকারী, গ্রুপ, sudo, SSH কী (Project ০১)
│   ├── ssh_hardening/    # sshd_config কঠোরকরণ (Lab ১৭)
│   ├── firewall/          # UFW নীতি (Lab ১৫, ১৭)
│   ├── services/          # সার্ভিসের অবস্থা (Lab ৭)
│   ├── system_config/     # sysctl, limits, NTP (Project ০১)
│   ├── nginx/             # Nginx টেমপ্লেট + হ্যান্ডলার (Lab ১২–১৪)
│   ├── nodejs/            # Node.js 20 LTS ইনস্টল (Lab ১৮)
│   ├── app_deploy/        # রিলিজ-ভিত্তিক ডেপ্লয় + রোলব্যাক (Project ০২)
│   ├── database/          # PostgreSQL/MongoDB + ব্যাকআপ (Lab ১৯)
│   ├── docker_host/       # Docker Engine + ডেমন কনফিগ (Lab ২১)
│   ├── k8s_prepare/       # swap, sysctl, containerd, kubeadm (Lab ২২)
│   └── healthcheck/       # API + ওয়েব হেলথ চেক (Project ০২)
├── playbooks/
│   ├── site.yml           # সব সার্ভারে বেসলাইন (সব রোল)
│   ├── deploy.yml         # MERN ডেপ্লয়: db → api → web → যাচাই
│   ├── rollback.yml       # আগের রিলিজে ফেরত + হেলথ চেক
│   ├── validate.yml       # বেসলাইন যাচাই (পরিবর্তন করে না)
│   ├── drift-check.yml    # সাপ্তাহিক ড্রিফট রিপোর্ট
│   └── cloud-provision.yml # ক্লাউড VM তৈরি (খরচ সতর্কতাসহ, Lab ২০)
├── templates/
│   ├── nginx.conf.j2      # Nginx সাইট কনফিগ
│   ├── dotenv.j2          # .env (Vault সিক্রেটসহ)
│   └── technova-api.service.j2  # systemd ইউনিট
├── scripts/
│   ├── bootstrap.sh       # নতুন কন্ট্রোল নোড সেটআপ (Ansible ইনস্টল)
│   ├── validate.sh        # validate.yml-এর মোড়ক
│   └── backup-verify.sh   # ডেটাবেস ব্যাকআপ আছে কি না যাচাই
└── docs/
    ├── architecture.md    # প্ল্যাটফর্মের চিত্র ও সিদ্ধান্তের নথি
    ├── secrets.md         # সিক্রেট নীতি (Vault, rekey, GitHub Secrets)
    ├── onboarding.md      # নতুন সদস্যের চেকলিস্ট
    └── runbooks/
        ├── new-server.md  # নতুন সার্ভার যোগ করা
        ├── deploy.md      # ডেপ্লয় ও অনুমোদনের ধাপ
        ├── rollback.md    # রোলব্যাকের ধাপ
        ├── incident.md    # সার্ভার/সার্ভিস বন্ধ হলে করণীয়
        └── recovery.md    # সম্পূর্ণ ব্যর্থতা থেকে পুনরুদ্ধার
```

মূল ফাইলসমূহ (সংক্ষিপ্ত)

`playbooks/site.yml` — সব সার্ভারের বেসলাইন; `common → users → ssh_hardening → firewall → services → system_config` রোলগুলো ক্রমানুসারে চলে; গ্রুপ-ভিত্তিক রোল (`nginx` শুধু webservers, `nodejs` ও `docker_host` শুধু appservers, `database` শুধু dbservers) আলাদা প্লেতে থাকে।

`playbooks/deploy.yml` — Project ০২-এর ডেপ্লয় প্লেবুক: প্রথমে dbservers-এ ব্যাকআপ, তারপর appservers-এ `serial: 1` রোলিং ডেপ্লয়, তারপর webservers-এ Nginx রিলোড, শেষে সব হোস্টে হেলথ চেক।

`playbooks/rollback.yml` — আগের রিলিজের অস্তিত্ব যাচাই করে `current` সিমলিংক ফেরত দেয়, সার্ভিস রিস্টার্ট করে, হেলথ চেক করে; আগের রিলিজ না থাকলে `fail` দিয়ে থেমে যায়।

`playbooks/drift-check.yml` — কোনো পরিবর্তন না করে SSH, UFW, ব্যবহারকারী ও প্যাকেজের অবস্থা রিপোর্ট করে; সাপ্তাহিক cron-এ চলে এবং ফলাফল `/var/log/technova-drift.log`-এ সংরক্ষিত হয়।

`docs/runbooks/recovery.md` — সম্পূর্ণ ব্যর্থতা থেকে পুনরুদ্ধারের ধাপ: (১) কোন সার্ভার/সার্ভিস বন্ধ তা `validate.yml` দিয়ে শনাক্ত করুন; (২) ডেটাবেস ব্যাকআপ থেকে পুনরুদ্ধার করুন (`pg_restore` / `mongorestore`); (৩) `site.yml` দিয়ে সার্ভারটি বেসলাইনে ফিরিয়ে আনুন; (৪) `deploy.yml` দিয়ে শেষ পরিচিত ভালো সংস্করণ ডেপ্লয় করুন; (৫) হেলথ চেক ও ড্রিফট চেক দিয়ে নিশ্চিত করুন।

> **WARNING:** ক্যাপস্টোনের ডেমোতে কখনো আসল ক্লাউড অ্যাকাউন্টে বড় VM তৈরি করবেন না — Lab ২০-এর খরচ সতর্কতা এখানেও প্রযোজ্য। ক্লাউড অংশটি লোকাল VM (Multipass/KVM)-এ দেখান, অথবা ছোট VM তৈরি করে কাজ শেষে সঙ্গে সঙ্গে মুছে ফেলুন। আসল পাসওয়ার্ড, টোকেন বা API কী ক্যাপস্টোনের কোনো ফাইলে লিখবেন না — সব জায়গায় `CHANGE-ME-...` প্লেসহোল্ডার ও Vault ব্যবহার করুন।

বাস্তবায়নের ধাপ (সুপারিশকৃত ক্রম)

(১) Lab ১–২৪ শেষ করে `~/ansible-labs/`-এর কাজগুলো যাচাই করুন। (২) Project ০১–০৩ শেষ করে রোল ও প্লেবুকগুলো সংগ্রহ করুন। (৩) উপরের কাঠামোতে রিপোজিটরি তৈরি করে সবকিছু এক জায়গায় আনুন। (৪) staging ইনভেন্টরিতে `site.yml` ও `deploy.yml` চালিয়ে শেষ-থেকে-শেষ পরীক্ষা করুন। (৫) `drift-check.yml` চালিয়ে ইচ্ছাকৃত একটি ড্রিফট (যেমন UFW-এ একটি পোর্ট হাতে খোলা) ধরা পড়ে কি না পরীক্ষা করুন, তারপর `site.yml` দিয়ে ঠিক করুন। (৬) staging-এ রোলব্যাকের মহড়া করুন। (৭) `docs/`-এর সব রানবুক লিখে একজন সহপাঠীকে দিয়ে পড়িয়ে দেখুন — সে একা একটি সার্ভার যোগ করতে পারে কি না। (৮) CI ওয়ার্কফ্লো GitHub-এ পুশ করে সবুজ জব নিশ্চিত করুন।

গ্রেডিং রুব্রিক

মানদণ্ড	ওজন	পাসের মানদণ্ড (Pass Standard)
সার্ভার ইনভেন্টরি ও পরিবেশ পৃথকীকরণ	১০%	dev/staging/production ইনভেন্টরি আলাদা; একই কোড তিনটি পরিবেশে চলে; ভুল পরিবেশে চালানো ঠেকাতে সতর্কতা বা আলাদা ডিফল্ট আছে
স্ট্যান্ডার্ড বেসলাইন কনফিগারেশন	১০%	`site.yml` চালালে যেকোনো Ubuntu 24.04 সার্ভার বেসলাইনে আসে; দ্বিতীয়বার চালালে `changed=0` (Idempotent)
	১০%	

ব্যবহারকারী ও SSH ব্যবস্থাপনা		ব্যবহারকারী/গ্রুপ/sudo/SSH কী রোল দিয়ে পরিচালিত; পাসওয়ার্ড লগইন বন্ধ; রুট লগইন বন্ধ; কী-লগইন যাচাইকৃত
প্যাকেজ ও সার্ভিস ব্যবস্থাপনা	৫%	প্যাকেজ ও সার্ভিসের কাঙ্ক্ষিত অবস্থা ঘোষণামূলকভাবে সংজ্ঞায়িত; সার্ভিসের অবস্থা হেলথ চেক দিয়ে যাচাই করা হয়
নিরাপত্তা হার্ডেনিং	১০%	SSH কঠোরকরণ, UFW (ডিফল্ট deny), fail2ban, স্বয়ংক্রিয় নিরাপত্তা আপডেট — সবই রোল দিয়ে প্রয়োগ করা ও যাচাইকৃত
অ্যাপ্লিকেশন ডেপ্লয়মেন্ট	১৫%	MERN অ্যাপ এক কমান্ডে ডেপ্লয় হয় (Nginx + Node.js + ডেটাবেস); রিলিজ-ভিত্তিক কাঠামো; ডেপ্লয়ের পর হেলথ চেক সফল
সিক্রেট ব্যবস্থাপনা	১০%	সব সিক্রেট Vault-এ; প্লেইন টেক্সটে কোনো সিক্রেট নেই; `no_log: true` সিক্রেট টাস্কে; ভল্ট পাসওয়ার্ড ফাইল Git-এ নেই
কনফিগারেশন যাচাইকরণ ও ড্রিফট শনাক্তকরণ	১০%	`validate.yml` ও `drift-check.yml` কাজ করে; ইচ্ছাকৃত ড্রিফট ধরা পড়ে; `site.yml` আবার চালালে ড্রিফট ঠিক হয়
CI/CD ইন্টিগ্রেশন	৫%	GitHub Actions-এ lint, syntax-check ও dry-run জব সবুজ; প্রোডাকশন রান ম্যানুয়াল অনুমোদনের পর হয়
ব্যর্থতা থেকে পুনরুদ্ধার (রোলব্যাক ও রিকভারি)	১০%	রোলব্যাকের মহড়া staging-এ সফল; ডেটাবেস ব্যাকআপ নেওয়া ও পুনরুদ্ধারের ধাপ নথিভুক্ত ও পরীক্ষিত
ডকুমেন্টেশন ও অপারেশনাল রানবুক	৫%	`docs/` -এ আর্কিটেকচার, সিক্রেট নীতি, অনবোর্ডিং ও পাঁচটি রানবুক সম্পূর্ণ; সহপাঠী রানবুক পড়ে একা কাজটি করতে পারে
মোট	**১০০%**	**পাস করতে হলে মোট ৭০% বা তার বেশি পেতে হবে, এবং "অ্যাপ্লিকেশন ডেপ্লয়মেন্ট", "সিক্রেট ব্যবস্থাপনা" ও "ব্যর্থতা থেকে পুনরুদ্ধার" — এই তিনটি মানদণ্ডের প্রতিটিতে অন্তত অর্ধেক নম্বর পেতে হবে।**

ক্যাপস্টোন জমা দেওয়ার চেকলিস্ট

(১) Git রিপোজিটরির লিংক (সব ফাইল কমিট করা, `.vault_pass` ছাড়া)। (২) staging-এ `site.yml` ও `deploy.yml` চালানোর লগ (PLAY RECAP-সহ)। (৩) ড্রিফট চেকের রিপোর্ট ও ড্রিফট ঠিক করার প্রমাণ। (৪) রোলব্যাক মহড়ার লগ। (৫) CI-এর সবুজ জবের স্ক্রিনশট বা লিংক। (৬) `docs/` -এর সব ফাইল সম্পূর্ণ। (৭) একটি ছোট ভিডিও বা লিখিত বর্ণনা: প্ল্যাটফর্মটি কী করে, কোন সিদ্ধান্ত কেন নেওয়া হয়েছে, এবং কী উন্নতি করা যায়।

> **REAL:** ক্যাপস্টোন শেষ করার পর আপনার হাতে শুধু একটি প্রকল্প নয় — একটি পোর্টফোলিও আছে। চাকরির সাক্ষাৎকারে "Ansible জানেন?" প্রশ্নের উত্তরে এই রিপোজিটরিটি দেখান: ইনভেন্টরি, রোল, Vault, CI, ড্রিফট চেক, রোলব্যাক — প্রতিটি অংশ একটি করে গল্প বলে। আর TechNova-র জন্য এর আসল মূল্য হলো: নতুন সার্ভার, নতুন পরিবেশ, নতুন দলের সদস্য — সবকিছু এখন একটি নথিভুক্ত প্রক্রিয়া, একজন ব্যক্তির স্মৃতির ওপর নির্ভরশীল নয়।

ওয়ার্কবুকের সমাপ্তি

আপনি ২৪টি ল্যাব, তিনটি প্রজেক্ট ও একটি ক্যাপস্টোন শেষ করেছেন — Ansible ইনস্টল থেকে এন্টারপ্রাইজ প্ল্যাটফর্ম পর্যন্ত। পরের ধাপ: এই প্ল্যাটফর্মটি নিজের হোম ল্যাবে নিয়মিত চালান, প্রতিটি পরিবর্তন Git-এ কমিট করুন, এবং Volume

10-এর Kubernetes ও Volume 11-এর মনিটরিং অধ্যায়ে এই একই প্ল্যাটফর্মকে আরও এগিয়ে নিন। শুভকামনা — TechNova-র সার্ভারগুলো এখন আপনার হাতে নিরাপদ।